

电子科技大学
UNIVERSITY OF ELECTRONIC SCIENCE AND TECHNOLOGY OF CHINA

专业学位硕士学位论文

MASTER THESIS FOR PROFESSIONAL DEGREE



论文题目 基于深度学习的源代码漏洞检测
技术研究

专业学位类别 工程硕士

学 号 201922081124

作者姓名 张浩杰

指导教师 李玉军 副教授

学 院 计算机科学与工程学院

分类号 TP311 密级 公开
UDC ^{注 1} 004.02

学 位 论 文

基于深度学习的源代码漏洞检测技术研究

(题名和副题名)

张浩杰

(作者姓名)

指导教师 李玉军 副教授
电子科技大学 成 都

(姓名、职称、单位名称)

申请学位级别 硕士 专业学位类别 工程硕士
专业学位领域 计算机技术
提交论文日期 2021 年 3 月 17 日 论文答辩日期 2021 年 5 月 13 日
学位授予单位和日期 电子科技大学 2021 年 6 月
答辩委员会主席 赵志为
评阅人 _____

注 1: 注明《国际十进分类法 UDC》的类号。

Research on Source Code Vulnerability Detection Based on Deep Learning

A Master Thesis Submitted to
University of Electronic Science and Technology of China

Discipline **Master of Engineering**

Student ID **201922081124**

Author **Zhang Haojie**

Supervisor **A.Prof. Li YuJun**

School **School of Electronic Science and Engineering**

独创性声明

本人声明所呈交的学位论文是本人在导师指导下进行的研究工作及取得的研究成果。据我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得电子科技大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示谢意。

作者签名： 张浩杰

日期：2022年6月1日

论文使用授权

本学位论文作者完全了解电子科技大学有关保留、使用学位论文的规定，有权保留并向国家有关部门或机构送交论文的复印件和磁盘，允许论文被查阅和借阅。本人授权电子科技大学可以将学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编学位论文。

（保密的学位论文在解密后应遵守此规定）

作者签名： 张浩杰

导师签名： 李东军

日期：2022年6月1日

摘 要

随着计算机技术的飞速发展，软件数量呈现爆发式增长。大部分软件开发周期短，开发不规范，导致软件漏洞数量呈现上升趋势。如何解决目前漏洞检测技术中存在的漏报率高、误报率高、严重依赖领域专家知识、检测粒度粗、以及漏洞定位困难等诸多问题，已成为亟须解决的问题。

为此本文提出了一种基于关联代码属性图和图注意力网络的源代码漏洞检测及定位方法。关联代码属性图是本文提出的一种改进的代码属性图，通过将调用函数和被调用函数对应的代码属性图关联起来用以实现跨函数检测。基于漏洞代码和无漏洞代码对应的关联代码属性图间的差异性，本文将源代码漏洞检测归纳为对关联代码属性图进行图分类的问题，并构建了一种基于图注意力网络的漏洞检测模型 DMGGAT。该模型使用 VecCPG 向量化方法对关联代码属性图进行向量化，提取源代码的词汇、语法、语义信息形成特征矩阵，提取 AST、CFG、DDG 边信息形成三种邻接矩阵。DMGGAT 模型包含三层多头 GAT 层和一个分类模块。DMGGAT 模型通过在每个多头 GAT 层输入不同的邻接矩阵对特征向量进行更新降维，再输入到分类模块实现图的二分类。基于注意力机制，可以获得检测过程中模型对节点的平均注意力值，平均注意力值高的节点对图分类的作用大。基于此思想，本文构建了一种基于注意力机制的漏洞定位模型 LDMGGAT。LDMGGAT 模型通过每层的注意力矩阵和 IQR 算法，获取节点注意力值正常区间实现漏洞定位效果。

本文从 Juliet 数据集中选择了 CWE121、CWE122、CWE369、CWE416 和 CWE476 进行了漏洞检测与定位实验。漏洞检测实验结果显示 DMGGAT 模型在五种 CWE 漏洞类型上的 F1 分数分别为 95.99%、90.36%、95.86%、96% 和 96.62%。定位检测实验结果显示 LDMGGAT 模型在五种 CWE 漏洞类型上的定位准确率均在 80% 以上。上述实验均验证了 DMGGAT 漏洞检测模型和 LDMGGAT 漏洞定位模型的有效性。最后，本文实现了一套基于深度学习的源代码漏洞检测原型系统，并进行了相关测试。

关键词：漏洞检测，代码属性图，图注意力网络，注意力机制

ABSTRACT

With the rapid development of computer technology, the number of software shows an explosive growth. Most software development cycles are short and the development is not standardized, resulting in an upward trend in the number of software vulnerabilities. How to solve many problems such as high false negative rate, high false positive rate, heavy dependence on domain expert knowledge, coarse detection granularity, and difficulty in vulnerability location in current vulnerability detection technology has become an urgent problem to be solved.

In order to solve these problems, this thesis proposes a source code vulnerability detection and localization method based on associated code property graph and graph attention network. The associated code property graph is an improved code property graph proposed in this thesis, which enables cross-functional detection by associating the corresponding code property graphs of the calling function and the called function. Based on the difference between the corresponding associated code property graphs of vulnerable and non-vulnerable code, this thesis converts the source code vulnerability detection into the graph classification problem of associated code property graphs, and constructs a vulnerability detection model DMGGAT based on graph attention network. The model uses VecCPG vectorization method to vectorize the associated code property graph, by extracting the lexical, syntactic, and semantic information of the source code to form a feature matrix, and extracting the AST, CFG, and DDG edge information to form three adjacency matrices. The DMGGAT model contains three multi-headed GAT layers and a classification module. The DMGGAT model updates the feature vector by inputting different adjacency matrices at each multi-headed GAT layer to reduce the dimensionality, and then input to the classification module to achieve graph binary classification. Based on the attention mechanism, the average attention value of the model to the nodes during detection can be obtained. The nodes with high average attention value are useful for graph classification. Based on this idea, this thesis constructs a vulnerability location model LDMGGAT based on the attention mechanism. LDMGGAT model achieves the vulnerability location effect by obtaining the normal interval of node attention value through the attention matrix of each layer and IQR algorithm.

This thesis selects CWE121, CWE122, CWE369, CWE416 and CWE476 from the

Juliet dataset for vulnerability detection and localization experiments. The results of the vulnerability detection experiments show that the F1 scores of the DMGGAT model on the five CWE vulnerability types are 95.99%, 90.36%, 95.86%, 96% and 96.62%, respectively. The results of the location detection experiments show that the LDMGGAT model has an accuracy of over 80% on all five CWE vulnerability types. All of the above experiments validate the effectiveness of the DMGGAT vulnerability detection model and the LDMGGAT vulnerability location model. Finally, this thesis implements a prototype system for source code vulnerability detection based on deep learning and conducts relevant tests.

Keywords: Vulnerability Detection, Code Property Graph, Graph Attention Network, Attention Mechanism

目 录

第一章 绪 论	1
1.1 课题背景与研究意义	1
1.2 漏洞检测的国内外研究现状	3
1.2.1 静态检测方法国内外研究现状	3
1.2.1.1 基于代码相似性的源代码漏洞检测	3
1.2.1.2 基于规则的源代码漏洞检测	4
1.2.1.3 基于机器学习的源代码漏洞检测	6
1.2.2 动态检测方法国内外研究现状	8
1.2.3 动静结合检测方法国内外研究现状	9
1.3 主要研究内容	11
1.4 本文的主要贡献与创新	11
1.5 本论文的结构安排	12
第二章 理论基础与关键技术	13
2.1 编译	13
2.1.1 词法分析	13
2.1.1.1 状态转换图	14
2.1.1.2 有限自动机	15
2.1.2 语法分析	16
2.1.2.1 自上而下的语法分析方法	16
2.1.2.2 自下而上的语法分析方法	16
2.2 图相关理论	17
2.2.1 图定义及遍历方法	17
2.2.2 属性图	18
2.2.3 代码属性图	19
2.3 本章小结	19
第三章 关联代码属性图构建技术研究	20
3.1 代码属性图构建技术	20
3.1.1 代码属性图构建技术方案	20
3.1.2 抽象语法树 AST 属性图	21
3.1.3 控制流图 CFG 属性图	22

3.1.4 程序依赖图 PDG 属性图	25
3.1.4.1 控制依赖图 CDG.....	26
3.1.4.2 数据依赖图 DDG	26
3.1.5 代码属性图	26
3.2 代码属性图关联技术	27
3.2.1 关联代码属性图	27
3.2.2 函数调用图 FCG 属性图添加技术.....	28
3.2.3 控制流图 CFG 关联技术.....	29
3.2.4 数据依赖图 DDG 关联技术.....	30
3.3 关联代码属性图存储技术	31
3.3.1 关联代码属性图内容信息	32
3.3.2 关联代码属性图存储方案	34
3.3.3 关联代码属性图可视化展示	35
3.4 本章小结	36
第四章 基于深度学习的源代码漏洞检测技术	37
4.1 源代码漏洞检测技术原理	37
4.1.1 基于关联代码属性图的源代码漏洞检测技术原理	37
4.1.2 基于注意力机制的源代码漏洞定位技术原理	39
4.2 基于深度学习的源代码漏洞检测框架	40
4.3 基于关联代码属性图的 VECCPG 向量化技术.....	41
4.3.1 特征矩阵	41
4.3.2 邻接矩阵	45
4.4 基于图注意力网络的 DMGGAT 漏洞检测模型构建	46
4.4.1 GAT 模块.....	47
4.4.2 分类模块	49
4.5 基于图注意力网络的 LDMGGAT 漏洞定位模型构建.....	50
4.5.1 LDMGGAT 漏洞定位模型结构.....	50
4.5.2 注意力矩阵	52
4.5.2 IQR 算法	53
4.6 实验验证	54
4.6.1 实验数据集	54
4.6.2 评价指标	56
4.6.2.1 漏洞检测效果评价指标	56

4.6.2.2 漏洞定位效果评价指标	57
4.6.3 实验结果分析	58
4.6.3.1 漏洞检测实验结果分析	58
4.6.3.2 漏洞定位实验结果分析	62
4.7 本章小结	63
第五章 基于深度学习的源代码漏洞检测系统设计与实现	64
5.1 系统设计	64
5.1.1 需求分析	64
5.1.2 总体框架设计	65
5.1.3 功能模块设计	65
5.2 基于深度学习的源代码漏洞检测系统实现	67
5.2.1 关联代码属性图模块的实现	67
5.2.2 漏洞检测及定位模块的实现	71
5.3 基于深度学习的源代码漏洞检测系统测试	72
5.3.1 基于深度学习的源代码漏洞检测系统的运行环境	72
5.3.2 基于深度学习的源代码漏洞检测系统的系统展示	73
5.3.2.1 用户管理界面	73
5.3.2.2 文件管理界面	74
5.3.2.3 漏洞检测定位界面	74
5.4 本章小结	76
第六章 全文总结与展望	77
6.1 全文总结	77
6.2 后续工作展望	78
致 谢	79
参考文献	80
攻读硕士 学位期间取得的成果	85

第一章 绪论

1.1 课题背景与研究意义

随着科技的飞速发展，互联网和计算机技术在通信、教育、办公、医疗等领域得到了广泛应用，人们的生活也随之发生了翻天覆地的变化。中国互联网络信息中心（CNNIC）发布的第 48 次中国互联网络发展状况统计报告中指出，截至 2021 年 6 月，我国即时通信用户规模达 9.83 亿，较 2020 年 12 月增长 218 万，占网民整体的 97.3%^[1]。新冠疫情以来，互联网在即时通信、在线办公、网络交易、网络娱乐、公共服务等方面起着越来越重要的作用，因而软件数量呈现爆炸式增长。其中大量软件基于开源的组件式和模块化开发，开发周期短，开发不规范，软件中漏洞数量呈现上升趋势。诸多软件漏洞导致安全事件频发，这对个人、企业、国家和社会都造成了极其恶劣的影响。

软件漏洞是由软件设计、开发或配置过程中的错误导致的缺陷^[2]。这些缺陷将会威胁到软件系统及其应用数据的机密性、完整性、可用性。正常情况下，这些缺陷一般不会影响程序的正常执行。但黑客会根据程序中存在的漏洞，使用相应的攻击手段，在未经允许的情况下访问或者破坏系统，使程序崩溃，进而窃取账户密码，应用数据等隐私信息，从而对企业的软件运行和用户使用造成不利影响。

软件漏洞主要存在于软件开发过程中的三个主要阶段：架构设计，编码实现和实际运行^[3]。软件开发的关键在于架构设计，正确且合理的架构设计可以在开发之初大幅度的降低出现漏洞的概率，同时也会降低后期漏洞维护的成本，因此架构设计的好坏将直接影响到整个软件的安全性和可用性。在拥有一个好的系统架构设计后，就需要分模块进行编码实现。程序员在代码编写的过程中，可能会由于经验不足或者疏忽留下了一些安全隐患。这些安全隐患虽然不会影响各个模块的关联和系统的运行，但还是存在产生漏洞的风险。在系统开发完善之后，将会在实际环境中进行部署使用。在安装使用的过程中，可能会存在软件配置错误和认证安全错位等导致漏洞。并且在开发的过程中，测试环境不能够真正取代真实生产环境，会存在一些意想不到的情况发生，这些都会导致漏洞的产生。在实际开发中，任何阶段都不可能尽善尽美，一丁点的出错都会导致漏洞的产生。漏洞难以避免，只能在设计和实现的过程中尽可能的降低漏洞出现的概率，并且在软件上线使用之前，结合一些专门的代码测试工具，通过人工审计的方式对软件代码进行安全测试。

美国标准与技术研究所（NIST）发布了 2001 年到 2021 年 20 年间报告的漏洞可视化数据统计情况，如图 1 所示。其中 2021 年共报告 18378 个安全漏洞，其中高危漏洞 3646 个，中危漏洞 11767 个，低危漏洞 2965 个。与 2020 年相比，漏洞总数增加 203 个，其中高危漏洞数量减少 684 个，中危漏洞增加 675 个，低危漏洞增加 212 个。

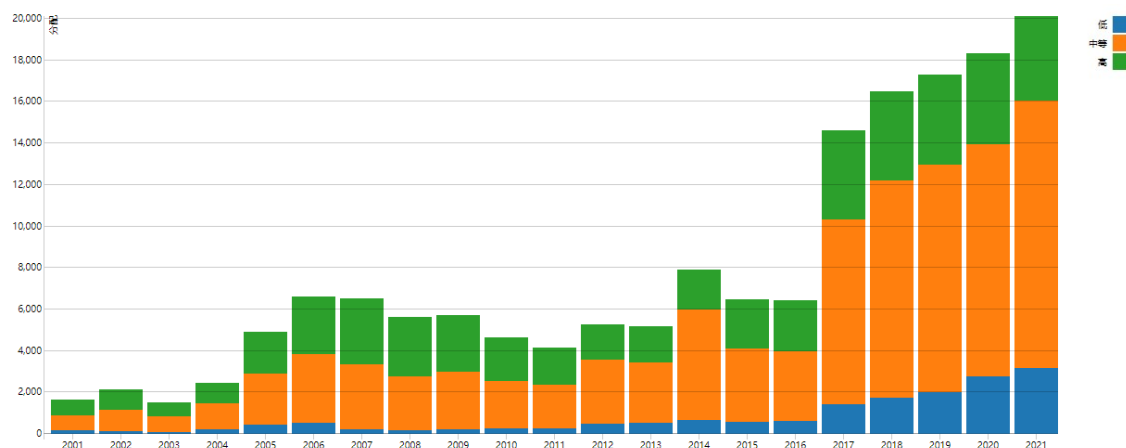


图 1 2001-2021 年漏洞可视化数据统计

快速增长的漏洞数量导致各种安全事件不断发生。黑客会利用软件漏洞，绕过系统防护措施，实现资源窃取、非法利用、破坏设施等攻击目的。2021 年 4 月，微信 PC 版客户端被披露出一个高危 0day 远程代码执行漏洞。由于微信使用默认关闭沙箱的 google 内核，因此只要微信用户点击一个特制的恶意 web 链接，攻击者就可以获取当前登陆微信的电脑的权限，用以窃取个人隐私信息或公司内部数据资料等。2021 年 5 月，高通芯片被披露出存在缓冲区溢出漏洞 CWE-2020-11292。不法分子可以利用该漏洞实现对手机用户的短信、通话记录、通话内容等信息的窃取，甚至远程进行 SIM 卡解锁操作，用以贩卖个人信息，实施诈骗等犯罪行为。2021 年 12 月，阿里云团队发现了 Apache Log4j2 远程代码执行漏洞（CVE-2021-44228/CVE-2021-45046）。Apache Log4j2 某些功能存在递归解析代码，未经身份验证的攻击者通过发送特定恶意数据包，触发该漏洞并在目标服务器上执行一些恶意代码。由于其触发方式简单、使用范围广泛，因此漏洞危害极大，堪比“永恒之蓝”。

从上述论述中可以看出，软件漏洞已经严重侵犯了个人隐私，企业财产，对国家安全、社会稳定和经济发展造成了重大影响。因此，针对软件漏洞的检测技术研究刻不容缓。但目前软件漏洞检测技术还存在着检测规则简单、漏报率高、误报率高、检测粒度过粗、无法定位或者定位不准确等诸多问题，因此如何及时发现、快速检测、精准定位软件漏洞，已成为亟须解决的问题。

1.2 漏洞检测的国内外研究现状

目前针对漏洞检测有多种方法，根据是否需要运行代码，可以分为三种方法：静态检测方法、动态检测方法、动静结合检测方法。

1.2.1 静态检测方法国内外研究现状

静态检测方法不需要运行程序代码，直接对程序源代码进行分析，通过数据流或控制流分析、相似性分析、相应安全规则检查等技术扫描源代码中的变量、操作符、传参等内容，进而发现程序中潜在的漏洞。

源代码漏洞静态检测方法一般是将源代码转化为 token、树、图等代码中间表示形式，结合不同的检测算法和检测模型进行检测。因此基于不同的检测原理，本文将静态检测方法分为基于代码相似的源代码漏洞检测、基于规则的源代码漏洞检测、基于机器学习的源代码漏洞检测。

1.2.1.1 基于代码相似性的源代码漏洞检测

基于代码相似性进行漏洞检测的核心思想是与漏洞代码相似的代码段极有可能包含相同的漏洞类型，即从代码段中抽取代码表征并比较判断它们的相似性，用以判断是否存在相同漏洞。

Sun 等人提出一种基于代码相似性的漏洞检测方法 VDSimilar^[4]。VDSimilar 使用包含漏洞函数和补丁函数的 CVE brunch 数据集，通过对比漏洞与漏洞间、漏洞与补丁间的差异性来进行漏洞检测。

Lei Cui 等人提出了一种基于 WFG 相似性的代码漏洞检测方法^[5]。该方法通过找寻漏洞关键词来获取漏洞函数，并提取出漏洞函数对应的控制流图 CFG。通过对 CFG 图切片，保留与漏洞关键词相关的程序依赖部分，进而生成权重特征图 WFG。最终通过对比 WFG 图的相似性来进行代码相似性漏洞检测。

李珍等人基于代码相似性研发了 Vackerability Pecker(VulPecker)系统，该系统可自动检测一段软件源代码是否包含给定的漏洞^[6]。VulPecker 的主要思想是使用基本特征和修补特征组成的文件来描述漏洞及补丁信息和设计的多种代码相似性检测算法来进行相似代码的漏洞检测。VulPecker 会根据漏洞检测结果和准确率阈值选出效果最好的代码相似性检测算法，并结合漏洞类型输出 CVE-算法映射表。

吴鹏等人使用代码属性图提取源代码特征信息，通过计算检测代码与易受攻击代码间的相似度，判断检测代码中是否存在漏洞^[7]。通过使用补丁代码，该方法可以减少误报。作者在 LibTIFF 和 Linux 内核上使用此方法可以有效地发现漏洞并减少误报。

四川大学的黄诚等人提出一种基于函数级代码相似性的漏洞检测方法^[8]。首先使用自定义的语法抽象化和规范化规则对开源漏洞函数和待检测函数源代码进行预处理；然后利用漏洞函数体和相应 patch 文件中的增加行代码和删除行代码生成漏洞函数指纹和待检测函数指纹；最后基于 WagnerFischer 算法的模糊匹配和基于 Aho-Corasick 算法的多模态精确匹配，实现函数指纹的漏洞检测。该方法避免生成复杂的中间表示，同时保留了基本的语法结构，保证了检测模型的性能，尤其是检测精度不受语法上无意义的修改的影响。在保证低误报率和漏报率的同时，提升了漏洞检测的可扩展性。

Pham 等人提出 SecureSync 自动化检测方法^[9]。该方法定义了 xASTs 和 xGRUMs 两种模型来描述代码复用和 API 复用导致的漏洞，通过对比检测代码和漏洞代码的 xASTs 和 xGRUMs 差异性来检测重复性软件漏洞。

Li 等人设计了一种基于子图匹配同构的代码相似性漏洞检测系统 CBCD^[10]。该系统提取漏洞代码对应的程序依赖图 PDG 中包含漏洞信息的子图，使用图的同构匹配算法来查找检测代码中与漏洞子图相似的代码片段，以此来进行代码相似性判断。

Yamaguchi 等人将漏洞归纳为特定的 API 模式，将漏洞代码相似性检测转化为 API 模式相似性检测^[11]。Yamaguchi 首先抽取函数中的 API 标识，定义 API 使用模式。然后使用机器学习的方法对检测代码的 API 模式与漏洞代码的 API 模型进行向量相似性比较，以此来进行漏洞检测。

基于代码相似性的源代码漏洞检测虽然可以检测出由于相似代码导致相同漏洞的出现，但是需要已知大量漏洞代码段，处理较为麻烦，并且相同漏洞也可能是由不相似代码导致的，局限性高。

1.2.1.2 基于规则的源代码漏洞检测

基于规则的源代码漏洞检测的核心思想是对漏洞进行模式匹配，针对特定的漏洞，总结漏洞特征，制定对应的规则来进行检测。

Jang 提出了一种基于规则的代码审计系统^[12]。该系统可以检测代码中存在的恶意代码和缓冲区溢出、格式字符串错误、竞争条件等漏洞。系统使用预定义的规则来分析检测代码中的漏洞并生成中间代码形式。基于生成的代码中间形式创建点文件并更改图形结构，通过 FSA 语法来定义相关的检测规则。

Shahriar 等人针对缓冲区漏洞提出了一套基于规则的漏洞检测与修复方案^[13]。该方案对缓冲区漏洞代码进行了大量的研究，将漏洞产生原因归纳为 ANSI C 库函数调用、索引变量、空字符、指针运算等，并设计相应规则以实现 BOF 漏洞检

测。此外，该方案还设计了八种缓冲区漏洞修复规则方案。

刘春燕设计了一种基于规则的 C/C++ 代码自动化检测系统 CIS^[14]。CIS 系统内置了一个基于 Lex 和 Yacc 工具的静态信息解析器，该解析器可以将源代码转化为一种去除冗余信息的关系语法树。然后使用一种可扩展的 XML 中间数据存储模型对关系语法树进行存储。基于《GJB 5369-2005 航天型号软件 C 语言安全子集》，通过 Xquery 查询表达式查询 XML 模型中违反安全规则的漏洞节点。

Lint 是一款针对 C/C++ 源代码进行检测分析的软件。Lint 比大多数 C 语言编译器在执行 C 的类型规则时要更加严格，主要进行强制类型检查、边界检测、赋值顺序检查、变量值跟踪和格式检查等操作^[15]。Lint 可以对代码文件进行关联，并进行类型组合、变量使用、代码可达性等更加广泛的错误分析，较为灵活。Lint 可以检测出的漏洞主要包含空指针漏洞、符号丢失、printf/scanf 的格式检查、异常表达式、拼写错误等。

Klocwork 静态代码分析工具基于 C/C++ 的编码规则对源代码自动扫描分析，检测源代码中的错误^[16]。它可以扩展到任何规模的项目，并且可以在 DevOps 周期内开展有效地工作。Klocwork 可以在开发早期检测出漏洞，提升代码质量，加速开发，通过报告与指标来监控代码质量，消除安全漏洞，并且将误报率保持在很低的水平。

PMD 是一种基于 Java 语言的开源静态分析检测工具^[17]。该工具通过对源代码进行词法、语法分析，将源代码转化为抽象语法树，制定的安全规则会检测树的相应节点，分析其属性或结构，从而找出违反规定的部分，进而找出漏洞所在。使用 PMD 可以在开发初期就进行代码的自我检查，及时发现代码中的错误或者缺陷进行修改，并通过规则，统一代码规范、提高代码质量，但是检测的效率相对较低。

Fortify 是一个基于规则的软件源代码安全测试工具，支持 26 种语言的漏洞检测^[18]。Fortify 首先通过内置的编译器将检测代码转化为一种 NST 的代码中间表示形式；然后将检测代码间的调用关系，执行环境，上下文等通过数据流分析、控制流分析、语义分析、结构分析、配置流分析等技术进行分析；最后使用其内置的安全规则编码包进行全面地漏洞匹配、漏洞查找。Fortify 可以自定义检测规则，集成性强，但是 Fortify 检测速度较慢，需要对源代码进行编译才能进行检测，不支持增量检测。

基于规则的源代码漏洞检测使用特定的规则进行漏洞匹配，具有误报率低，查准率高的特点。但是前期需要人工对大量的漏洞代码进行分析，制定规则，耗时耗力，并且制定的规则的好坏和覆盖面将直接影响到漏洞检测的结果。

1.2.1.3 基于机器学习的源代码漏洞检测

近年来,机器学习在自然语言处理、图像识别、语言识别等领域得到了广泛的应用。随着对网络安全的逐步重视,机器学习也逐渐应用到漏洞检测方面,目前已取得了一定的成果。

根据训练方式的不同,机器学习可以分为三种:监督学习、无监督学习和强化学习。监督学习意味着在训练过程中,输入的数据集中包含向量数据与对应的标签信息。常用的方法有多层感知机 MLP、支持向量机 SVM、k 近邻等。无监督学习则是给模型输入不带有标签信息的向量数据集,模型通过训练自行对无标签数据进行分类。常见的方法有聚类方法、奇异值分解 SVD、主成分分析 PCA 等。强化学习是不给定数据,根据模型对动态环境的反应动作给予不同的奖励和惩罚,使模型自行学习信息并不断更新参数,以达到某个训练目标。常用的方法有 Q-learning、sarsa、deep Q Network、policy Gradient、Actor Critic 等

本文按照是否需要人类专家定义特征和机器学习中的不同方法,将漏洞检测技术分为基于传统机器学习方法和基于深度学习方法。

(1) 基于传统机器学习的源代码漏洞检测

传统的机器学习漏洞检测方法需要人为抽取漏洞的特征属性,然后采用机器学习方法,如随机森林、支持向量机、朴素贝叶斯、k 近邻等进行分类。

Yamaguchi 等人基于漏洞函数中的 API 符号,将其映射到向量空间中,使用 PCA 方法获取漏洞函数对应的 API 使用模型,进而用于辅助代码检测^[11]。

Chernis 等人提取了 C 源代码中函数的简单特征和复杂特征,采用了机器学习中的朴素贝叶斯、k 近邻和 k 均值等分类器进行实验^[19]。实验结果显示基于机器学习的函数特征检测方法是有效的,可以用于判断检测函数中是否存在漏洞。

Medeiros 等人提出一种基于 AST 的 Web 漏洞检测方法^[20]。该方法首先需要对代码进行编译,生成抽象语法树 AST。然后使用污点分析方法对 AST 进行遍历,获取漏洞相关子树。最终使用 MLP、朴素贝叶斯、逻辑回归、随机森林等 10 种机器学习分类器进行训练验证。实验结果显示逻辑回归方法分类效果最好。

Lin 等人基于 AST 和 CBOW 模型将源代码转化为 100 维的向量,选用 23 个代码度量指标提取代码特征,随后使用机器学习中的随机森林分类器进行训练。实验结果表明该方法提取的代码特征针对漏洞检测十分有效^[21]。

Zhong 等人提出了一种基于自然语言处理的漏洞检测方法 Doc2Spec^[22]。Doc2Spec 方法使用隐马尔可夫模型对 API 文档进行分析与实体识别,获取代码词性特征用于漏洞检测。该方法具有较高的准确率、查全率和 F1 分数,但是误报较高。

Padmanabhuni 等人提出了一种基于缓冲区溢出漏洞的静态漏洞检测方法^[23]。该方法通过提取潜在漏洞的控制流信息与数据依赖信息，使用机器学习中的 MLP、SVM、朴素贝叶斯、J48 决策树、简单逻辑回归方法构建检测模型，进行训练验证。实验表明在这五种方法中，J48 决策树具有较高的召回率和精确度，漏洞检测效果最佳。

Feng 等人提出了一种基于聚类算法的源代码漏洞检测方法^[24]。该方法将源代码转化为具有结构属性和统计属性的属性控制流图 ACFG，使用聚类算法进行训练，实现了漏洞搜索引擎 Genius。

Xu 等人设计了一种基于 Structure2vec 图嵌入网络的 Siamese 架构用于漏洞检测^[25]。Siamese 架构首先提取检测代码和漏洞代码属性控制流图 ACFG 中的代码特征。然后将其输入到 Structure2vec 模型中去，产生两个高维向量。最后根据这两个向量间的 cosine 距离来衡量检测代码和漏洞代码的相似度，以此来判断检测代码是否存在漏洞。

Harer 等人从代码对应的控制流图 CFG 中提取执行操作特征、变量信息等代码特性，基于这些代码特性定义了一个 116 维的特征向量，然后将这些代码特征输入到随机森林中进行分类^[26]。

基于传统机器学习的源代码漏洞检测方法前期对数据的处理较为麻烦，需要专家手工定义代码特征，并且这类方法大多数都是基于函数进行漏洞检测的，检测粒度较粗，无法定位到漏洞具体所在行。

(2) 基于深度学习的源代码漏洞检测

深度学习具有学习能力强，覆盖范围广、适应性好等优点，因此将其应用到漏洞检测领域。基于深度学习的源代码漏洞检测方法告别了特征工程，可以自动提取代码特征，自动生成漏洞模式，检测效果也比传统的机器学习方法好。

段旭等人提出了基于代码属性图及注意力双向 LSTM 的漏洞挖掘方法^[27]。根据代码属性图对源代码进行切片，转化为 144 维的特征向量，基于双向 LSTM 和注意力机制的神经网络进行训练，在 SARD 的两种漏洞类型上 F1 分数达 80%，能够有效地识别单个函数中是否存在漏洞。

李珍等人设计了一种基于深度学习的漏洞检测系统 VulDeePecker^[28]。VulDeePecker 定义了一种代码中间表示 code gadget，代表着语义上存在控制依赖关系或者数据依赖关系的代码语句的集合。VulDeePecker 首先利用程序中的 key point 将代码转化为 code gadget，然后使用 word2vec 将 code gadget 转化为具有语义信息的向量，最后基于双向 LSTM 网络进行特征提取与漏洞二分类检测。2021 年，李珍等人又提出了改进的漏洞检测系统 uVulDeePecker^[29]。uVulDeePecker 系

统与 VulDeePecker 系统相比, 添加了用于漏洞类型检测的 code attention, 并使用双向 LSTM 提取局部的 code attention 特征, 最后实现了漏洞多分类效果。

Rebecca 等人设计了一个 C/C++ 词法分析器, 对源代码进行词法分析, 获取关键词的含义, 并舍弃一些非关键信息。然后使用 word2vec 对这些关键词向量化, 分别使用 CNN 和 RNN 来提取近距离和远距离特征依赖, 然后融合特征进行 softmax 或者 RF 的二分类检测^[30]。

Zhou 等人提出一种基于 GNN 的源代码漏洞检测模型 Devign^[31]。该模型首先将源代码转化为包含 AST、CFG、DFG 的联合图, 基于联合图提取复合代码特征。然后使用 word2vec 模型对这些复合代码特征向量化, 将其输入到构建的 GNN 模型中去聚合节点特征, 最终使用 Conv 模块分类。

王松等人将源代码转化为抽象语法树, 选取其中声明节点和控制流节点, 基于 CLNI 算法去掉噪音后将节点 token 映射为整数。然后将其输入 DBN 深度置信网络后得到输出特征, 通过分类器对特征进行二分类, 以此判断检测代码是否存在漏洞^[32]。

Hoa Khanh Dam 使用 LSTM 网络学习代码的语法和语义特征以用于 Java 源代码漏洞检测^[33]。该方法使用 LSTM 提取 Java 源文件的特征表示, 将其聚合池化形成局部特征。然后使用 kmeans 对整个程序中的所有 Java 文件的 token 进行聚类, 形成全局特征。最后基于全局特征和局部特征, 通过分类器进行漏洞的分类检测。

文敏等人提出了一种基于关系图卷积网络的源代码漏洞检测方法^[34]。该方法首先将源代码转化为代码属性图, 然后使用 DGL 将代码属性图转化为图数据结构 DGLGraph。随后在其中挑选了 69 种类型的节点和 5 种语法语义边, 使用 word2vec 模型进行向量化。最后将向量化后的数据输入到 RGCN 模型中来进行漏洞分类。

基于深度学习的诸多检测方法在各自数据集上的检测效果较好, 但还存在许多不足。目前大多数深度学习的检测粒度为函数级别, 不能对代码中的漏洞实现具体定位。漏洞检测工作缺少一个统一且全面的数据集, 该数据集需要包含各种类型的漏洞, 且有多种编程语言版本。深度学习方法在漏洞检测粒度、漏洞定位、模型解释等方面有待深入研究。

1.2.2 动态检测方法国内外研究现状

动态检测方法则需要运行代码或者直接对系统进行检测。除了开源软件之外, 无法获取到其他软件的源代码, 此时就需要动态检测方法进行漏洞检测。动态检测方法需要人为的构造一些标准数据和非标准数据, 作为相应的测试数据进行输

入，根据系统功能或数据流向，检查运行结果的异常。对比实际输出结果与预期结果，分析程序的正确性、健壮性等性能，以判断被测软件系统是否存在漏洞。

动态检测方法主要分为三种：模糊测试、动态符号执行和动态污点分析。模糊测试是软件测试中的常用技术，通过向目标程序中输入大量随机有效数据，检测程序是否会有异常，以此来判断目标程序是否存在漏洞。动态符号执行是将程序的输入使用符号来代替，该符号表示所有可能输入的值。通过不同的程序执行路径，将程序的输出转化为多个路径函数。每个函数对应着一条从输入到输出的程序执行路径。动态污点分析的分析对象是程序中的污点信息流，通过在程序的执行过程中，结合上下文信息跟踪污点信息的执行顺序和流动方向进行漏洞检测。

朱飏凯等人基于动态模糊测试和机器学习实现了对智能合约代码的漏洞检测^[35]。通过自定义爬虫爬取 Solidity 智能合约代码和其他语言代码作为实验数据集，将其输入到基于随机森林算法的漏洞检测模型，并使用 k 折交叉检验来控制模型的超参数。最终使用动态模糊测试对模型的结果进行验证。

倪萍等人设计了一个基于模糊测试的反射型跨站脚本漏洞检测系统^[36]。该系统依据潜在的用户注入点和攻击载荷的语法特征，构造出大量的模糊测试数据。通过设置权重和注入探针向量来提取有效的测试数据，将其作为请求参数对网站进行相应分析。该系统与 ZAP、Wapiti 系统相比，降低了漏洞的误报率和漏报率。

赵伟等人提出了一种基于符号执行的智能合约漏洞检测方案^[37]。该方案使用 solc 编译器获取源代码对应的汇编代码，再反编译生成以太坊合约字节码。通过对编译源代码形成控制流图，再利用约束求解器对控制流约束求解，最后基于合约漏洞的特点，设计出整形溢出、权限控制、Call 注入、重入攻击所对应的路径约束条件，以此进行漏洞检测。

朱正欣等人提出一种二进制程序的动态符号化污点分析方案^[38]。该方案是基于符号执行中的符号化思想，将污点信息进行符号化，制定相应的规则，通过查看污点信息对应的符号是否违背定制的规则，以此来对漏洞进行检测。

动态检测方法具有准确率高的优点，但效率较低，需要不断的输入数据来进行判断。根据输出结果的差异性，只能判断是否存在漏洞和大概判断漏洞存在的范围，需要进一步的数据追踪，逐步分析才能得到具体的漏洞信息，耗时耗力，效率低下。因此动态检测方法不适用于大型的软件系统，只能小规模进行检测。

1.2.3 动静结合检测方法国内外研究现状

动静结合检测方法有效的结合了静态检测方法和动态检测方法。动静结合检测方法先使用静态检测方法对大规模的软件源代码进行检测，根据检测结果程序

的先验知识，对大规模的软件源代码进行切分，有针对性的进行检测。再使用动态检测方法对已划分的程序代码进行数据输入，根据数据流向来判断漏洞是否存在。

李鑫等人基于代码中间表示形式和污点分析技术提出了一种动静结合的 SQL 注入漏洞检测技术^[39]。他们使用开源工具 PHP-Parser，对 Web 应用源代码进行编译获取 CFG。然后借助元数据分析 PDS 构建的数据流，使用污点分析技术获取数据流中污点信息的传递方向，用以检测二阶 SQL 的注入漏洞。最终，使用模糊测试的方法来验证检测结果的正确性。

宁馨等人将基于代码相似性的静态检测方法与模糊测试动态检测方法相结合，提出一种基于深度学习的漏洞挖掘方法^[40]。该方法首先基于程序中的敏感点，对漏洞源代码进行切片操作。随后，对漏洞切片进行宏定义替换、函数名/变量名替换等预处理操作。然后对预处理数据分词，使用自然语言处理中的 Word2Vec 方法将其转化为词向量，将其输入到 Siamese 孪生网络中进行训练。根据训练的结果，将模型检测出存在漏洞的代码进行定向模糊测试，验证漏洞的可触发性。该方法具有一定的效果，但是模型的准确率只有 72.38%，具有较多的漏报和误报。

Zhang 等人提出了一种结合静态分析和动态检测的漏洞检测框架 SDCF^[41]。SDCF 框架在动态污点分析过程中，使用静态分析器对代码对应的控制流图 CFG 进行未执行信息补充，添加目标程序功能中未执行的代码和信息。并且基于 SDCF 框架开发了 LSVD 工具，该工具不仅能检测代码运行时产生的软件漏洞，还能发现可能会导致漏洞但未执行的代码。

Sanjay Rawat 等人提出了一种基于 C 语言的缓冲区溢出漏洞混合检测方法^[42]。该方法使用静态分析技术对程序进行切片，获取用户输入与易受攻击的语句之间的污点依赖序列 TDS。然后基于遗传算法生成输入数据，使用模糊测试的动态方法来检测漏洞。

Li 等人设计了基于动静结合的 ReDos 漏洞检测工具 ReDoSHunter^[43]。ReDoSHunter 将诸多正则表达式归纳与基于嵌套量词模式、指数重叠析取模式、指数重叠相邻模式、多项式重叠相邻模式、从大量词开始模式。基于这五种中模式对正则表达式进行静态检测和动态验证。该工具不仅可以准确的检测漏洞，还会给出详细的检测信息，例如漏洞原因，漏洞定位，漏洞严重程度等。

动静结合检测方法既解决了静态检测方法误报率较高的缺陷，又解决了动态检测方法中效率低下，检测范围低的缺陷，有针对性的进行漏洞检测，提升了检测效率，提高了查全率和查准率。

1.3 主要研究内容

现阶段基于深度学习的漏洞检测方法虽具有一定的效果，但还存在着不能进行跨函数检测、检测粒度较粗、误报率较高等问题。为解决这些问题，本文提出并实现一套基于图注意力网络的源代码漏洞检测方案，主要研究内容包括以下四个方面：

（1）构建关联代码属性图

基于图的代码中间表示形式可以有效的表示源代码的词法、语法信息。因此对源代码进行词法分析、语法分析，构建源代码对应的抽象语法树 AST。随后基于抽象语法树 AST，引入控制流图 CFG、程序依赖图 PDG 形成代码属性图。最后，根据函数间的调用关系，为代码属性图添加函数调用图 FCG，关联控制流图 CFG 和数据依赖图 DDG，形成能够全面反映程序词法信息、语义信息、控制信息、数据依赖信息和函数调用关系等要素关联代码属性图。

（2）构建基于图注意力网络的漏洞检测模型

基于漏洞代码和无漏洞代码对应的关联代码属性图间的差别，将源代码漏洞检测归纳为图分类问题。首先，需要将关联代码属性图向量化，并添加相应的漏洞标签信息作为模型的最终输入数据集。然后，基于图注意力网络构建深度学习漏洞检测模型。该模型使用三层多头 GAT 和分类模块实现对关联代码属性图的二分类。最后，对构建的漏洞检测模型进行训练验证，根据检测结果对模型不断调参和优化，并确定最终模型。

（3）构建基于注意力机制的漏洞定位模型

注意力机制通过给图中邻居节点分配不同的权重，使关注度高的节点成为图分类的依据。基于注意力机制，将检测过程中的注意力矩阵输出，获取关联代码属性图中所有节点的平均注意力值。然后基于 IQR 算法，对节点的平均注意力值进行排序，得到正常注意力区间对应的节点，再结合节点对应的行号信息进行定位。

（4）设计并实现原型系统

基于上述研究成果，设计并实现一套 B/S 架构的源代码漏洞检测原型系统。随后，基于公开测试集对原型系统进行功能、性能测试，用以验证所提出技术方案的可行性及有效性。

1.4 本文的主要贡献与创新

本文提出的基于深度学习的源代码漏洞检测技术的主要贡献与创新为：

（1）改进代码属性图，构建关联代码属性图

本文提出了一种改进的代码属性图——关联代码属性图。关联代码属性图是在代码属性图的基础上，根据函数间的调用关系，添加函数调用图 FCG、调整控制流图 CFG 和数据依赖图 DDG。在进行漏洞检测时，以关联代码属性图为基础，从而实现对跨函数漏洞的检测。

（2）漏洞检测模型 DMGGAT

基于图注意力网络的漏洞检测模型 DMGGAT 使用三层多头注意力网络层，依次输入 AST、CFG、DDG 邻接矩阵来更新特征向量。DMGGAT 模型充分利用了关联代码属性图中节点间的不同边关系，准确率和查全率较高，模型效果明显。

（3）漏洞定位

本文提出了一种基于图注意力网络的漏洞定位模型 LDMGGAT。该模型基于注意力机制，获取检测时关联代码属性图对应的注意力矩阵，计算节点的平均注意力值。随后通过 IQR 算法，结合节点行号信息，实现了漏洞定位功能。

1.5 本论文的结构安排

本文的章节结构安排如下：

第一章：绪论。本章介绍了源代码漏洞检测技术的研究背景和研究必要性，又分别从静态检测、动态检测、动静结合检测方面介绍了目前漏洞检测方法的国内外研究现状。结合已有的研究成果和方法，提出了本文的研究内容。

第二章：理论基础与关键技术。本章主要介绍了编译原理的相关方法和图、属性图、代码属性图的相关理论基础，为后续的研究奠定理论基础和现实依据。

第三章：关联代码属性图构建技术研究。本章在原有代码属性图的基础上，添加函数调用关系，关联控制流图 CFG 和数据依赖图 DDG，提出了新的代码属性图——关联代码属性图，以此来奠定实现跨函数漏洞检测的基础。

第四章：基于深度学习的漏洞检测技术研究。本章详细介绍了针对待检测代码的代码属性图的 VecCPG 向量化方法、漏洞检测模型 DMGGAT 和漏洞定位模型 LDMGGAT。

第五章：基于深度学习的源代码漏洞检测系统的设计与实现。本章主要介绍了源代码漏洞检测系统的整体框架和功能模块的设计与实现，并进行了系统开发和相关测试。

第六章：总结与展望。本章首先对基于深度学习的源代码漏洞检测技术研究的主要工作进行了总结，随后指出了本文技术研究中的不足，以及在未来研究中可以进一步改进的方向。

第二章 理论基础与关键技术

2.1 编译

程序是人们为了方便快捷的解决某些问题而使用编程语言编写并能在计算机上运行代码集合。目前应用软件程序的开发主要是使用计算机语言中的高级编程语言，例如 C、C++、Java、Python 等。与用于计算机识别的低级机器语言相比，这些高级编程语言便于人们理解使用，虽然与日常生活中的语言不同，但其也有基本的语法单位和语法结构，具有较强的表达能力。使用高级编程语言编写的算法可以很好的表示人们解决问题的思路和方法，并且执行速度快。高级编程语言与计算机的硬件结构及指令系统无关，并且高级编程语言编写的程序不能直接在计算机上运行。程序的运行需要将高级语言代码转化为计算机能够识别的二进制机器语言程序，其中将源代码转化为目标程序的过程叫做编译。

如图 2-1 所示，根据编译的逻辑步骤，可以分为词法分析、语法分析、语义分析、优化和目标代码生成五个阶段。下文将主要介绍编译过程中的词法分析与语法分析。

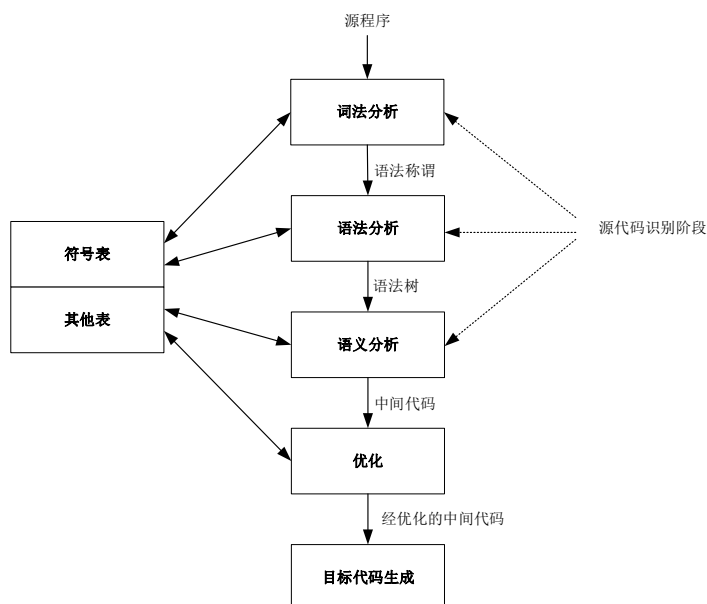


图 2-1 编译过程

2.1.1 词法分析

词法分析是将程序代码中的字符序列依次扫描识别，结合词法规则将其转换

为基本词法单位序列的过程。一个简单的 C 程序文件包含头文件和函数，函数由函数头和函数体组成。函数体是函数功能实现的若干语句，这些语句是由 C 语言的基本语法单位构成，包括标识符、关键字、运算符、常量和边界符。词法分析的目的就是按照从左到右的字符扫描顺序，结合语法规则和符号表，将这些基本语法单位一一识别标记。语句 `int result= 100 / a` 的基本语法单位识别过程如图 2-2 所示，首先将字符按顺序一一读入，然后去除空格等无关字符，最后组合识别。

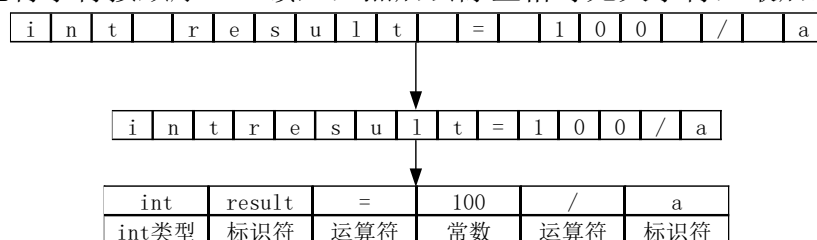


图 2-2 示例语句识别过程

进行词法分析的程序或者函数叫做词法分析器^[44]。词法分析器的构造一般借助状态转换图和有限自动机。

2.1.1.1 状态转换图

状态转换图是一种有限有向图，在词法分析中用以表示识别某个特定字符串的过程。状态转换图中的节点代表词法分析过程中的某个状态，若两个节点间有一个有向边相连，则这两个节点代表的状态具有状态转化关系，反之则无。状态间的转化基于状态转换条件，即有向边上的字符信息。

状态转换图与图论中的图一样，节点集合是一个有限集合，代表有限个状态。有向边对应着一个出点和入点，出点即状态转换的初态，入点即状态转换的终态。一个初态可以对应一个或多个终态。依据不同的转换条件，将初态转换为对应的终态。如图 2-3 所示，状态 1 为初态，状态 2 和状态 3 为终态，转换条件分别为字符 a、字符 b。当状态 1 接收到字符 a 时，会转换到状态 2。当状态 1 接收到字符 b 时，会转换到状态 3。

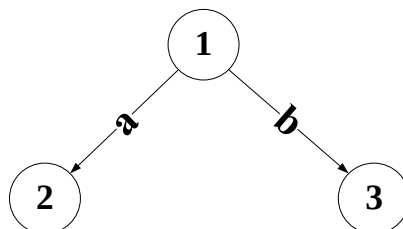


图 2-3 状态转换图

基于以上状态转换图，就可以程序中的标识符、关键字、常数、运算符、界

符单词符号进行状态转换。图 2-4 展示了用于识别标识符的状态转换图，状态 0 为初态，状态 2 为终态。因为在程序中标识符的首字母不能为数字，一般为字母，所以当状态 0 接收的状态转换条件为字母时，则将字母读入存储，并转换为状态 1。在状态 1 中，如果接下来的转换条件为字母或者数字时，则继续维持状态 1，直到转换条件不是字母或者数字时，将其读进来，并转换为状态 2。状态 2 是终态，表示已经得到输入字符串对应的标识符。状态 2 需要将状态 1 与状态 2 之间的转换条件退回，最终输出标识符内容。

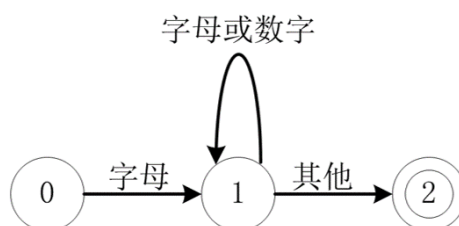


图 2-4 标识符状态转换图

2.1.1.2 有限自动机

有限自动机是一种数学模型，用于状态转换图的形式化表达，两者之间可以等价转换。在词法分析中，有限自动机用于描述识别输入字符串的过程，主要使用确定有限自动机（deterministic finite automaton，DFA）。

如公式 2-1 所示，确定有限自动机 A 是由集合 Q，输入字母表 Σ ，转移函数 δ ，开始状态 s，接收状态集合 F 组合的五元组^[45]。

$$A = (Q, \Sigma, \delta, s, F) \quad (2-1)$$

集合 Q 是一个非空有限的状态集合，表示状态转换图中所有状态。输入字母表 Σ 是一个非空有限的字符集合，即源代码中所有的字符，代表所有的状态转移条件。转移函数 δ 用于计算状态转移，通过接收当前状态 q 和状态转移字符 σ 得到转移的下一状态。

在词法分析中，确定有限自动机 DFA 对给定的字符串按照从左到右的字符顺序一一读入，判断该字符串能否接收。从最初状态开始，接收一个字符便使用转移函数 δ 计算转移状态，直至读取完全部字符。如果最终计算出来的转移状态属于接收状态集合 F，那么确定有限自动机 DFA 接收该字符串，反之则拒绝该字符串。

词法分析中的语法规则一般使用正则表示式描述，因此在词法分析器中使用确定有限自动机 DFA 来识别正则表示式，获得相应的 token。

2.1.2 语法分析

语法分析是在词法分析的基础上，根据源代码的语法规则，对 token 序列进行组合，形成正确语句对应的 token 序列流，并给出相应的语法树。如果在语法分析过程中，存在不符合源代码的语法规则的 token 序列流，则会将其输出并显示相应的错误信息，因此语法分析可以对源代码进行语法检测。语法分析的方法通常分为自上而下的语法分析方法和自下而上的语法分析方法。

2.1.2.1 自上而下的语法分析方法

自上而下的语法分析方法是指对给定的单词符号串，从文法的开始符号出发，寻找该单词符号串对应的一个最左推导序列。从生成语法树的过程来看，就是从根节点出发，自上而下的构造出对应的语法树。自上而下的语法分析主要分为回溯分析法、递归下降分析法和预测分析法。

回溯分析法是一种不确定的分析方法。回溯分析法是在寻找最左推导序列的过程中，由于文法中包含公共左因子、左递归或者 ϵ 产生式等情形时，使用当前最左推导候选式推导失败，就会进行回溯，换用其他候选式继续推导。回溯分析法实际上是一种穷举，效率较低。在语法分析中，一般使用提取公共左因子方法和消除左递归方法来避免回溯。

递归下降分析法式一种确定的分析方法。递归下降分析法根据非终结符号设计一种识别程序，在不含左递归的文法中或者每一个非终结符的所有候选终结首字符集都两两不相交条件下，递归的使用识别程序进行最左推导序列推导的方法。递归下降分析法分析高效，便于实现。

预测分析法是一种表驱动的方法，它由下推栈，预测分析表和控制程序组成。预测分析法的核心思想是从文法开始符号起，将输入的字符压入下推栈中，通过查询预测分析表确定下一步推导所需要的产生式，若产生式不存在，则该字符串不符合语法规范；若存在唯一产生式，则继续利用该产生式进行下一字符的推导，直至结束。预测分析法的关键在于预测分析表的构建，即对文法中各非终结符 FIRST 集和 FOLLOW 集的构建。

2.1.2.2 自下而上的语法分析方法

自下而上的语法分析方法与自上而下的语法分析方法相反，其是从输入的字符串出发，向上规约，直至文法开始符号，寻找到一个最左规约序列。从语法树的角度来看，就是从最底层节点出发，一步一步的构建上一层节点，直至规约到根节点。自下而上的语法分析方法分为算符优先分析法和 LR 分析法。

算符优先分析法也是一种表驱动的分析方法，其核心思想是根据算符优先关系表，判断终结符之间的优先级关系，先对优先级高的终结符进行规约，对句型不断寻找最左素短语，直至获取全部字符串，进而确定语句的合法性。算符优先分析法只适用于算符优先文法，即对表达式的分析。算符优先分析法具有简单高效的优点。

LR 分析法是一种从左到右，从下到上进行规约的表驱动分析方法。LR 分析法由分析栈、分析表、控制程序组成。分析栈包含了存放分析状态信息的状态栈和存放移进归约文法符号信息的符号栈。该分析法的主要思想是对给定字符串依次读取，将读取的字符放入字符栈中并保存状态信息，直至栈顶出现句柄，然后根据分析表采用移进或者规约的处理方法，得到唯一句柄，直至文法开始符。LR 分析法具有文法限制少，分析速度快，适用范围广，分析高效准确等优点，适用于一大类文法。

2.2 图相关理论

2.2.1 图定义及遍历方法

图是指图论中的图，并非是日常生活中的各种图像和图画。图代表着某类具体事物和这些事物之间的联系。图包含两种元素，顶点和边，以节点代表单个的具体事物，以边进行连接代表事物之间的关系。

图论中图 G 定义为一个有序对 (V, E) ，记作 $G=(V, E)$ [46]。图 G 中的所有节点构成的有限点集合为 $V(G)$ ， $V(G)$ 中节点与节点之间的边构成了有限边集合 $E(G)$ 。 $E(G)$ 中的边可以有向边，也可以是无向边，因此可以分为有向图和无向图，如图 2-5 所示。在本文的研究中使用的是有向图的概念。

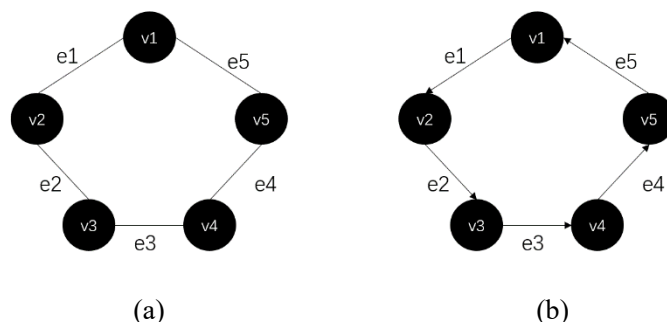


图 2-5 图示例。(a)无向图；(b)有向图

在有向图中，如果节点间存在不止一条边，并且这些边是平行边，那么这种有向图被称为有向多重图(Directed Multi Graph)。

在计算机理论基础中，针对连通图中节点的遍历有两种遍历顺序方案，分别为广度优先遍历和深度优先遍历。

广度优先遍历的主要思想是从一个点出发，尽最大程度的辐射到能覆盖的节点，并对其进行访问。即先选取图 G 中的一个节点 A ，放入遍历集合 V 中去；找出与节点 A 相连的所有节点，放入到放入已知节点集合 V 中去；再分别找出与这些节点相连的不包含 V 中节点的其他节点，将其放入集合 V 中。一直重复此类操作，直至图中所有关联的节点被找到。

深度优先遍历的主要思路是从一个点出发，在与其相连的节点中只选择一个节点进行访问，一直往下访问。即选取图 G 中的一个节点 A ，放入遍历集合 V ；在与节点 A 相连的所有节点中选取一个节点 B ，将节点 B 放入集合 V 中；再寻找与节点 B 相连的一个不在 V 中的节点放入集合 V 中，直至遍历到这条路的尽头，然后回退到尽头节点的上一个节点，进行向下遍历；重复此操作，直至遍历完所有节点。

广度优先遍历需要保留当前遍历的节点，因此占用内存较多但是速度快，而深度优先遍历占用内存小但是需要回溯，速度较慢。在实际运用中，则需要根据实际情况来选择合适的遍历方案。

2.2.2 属性图

属性图是一个有向多重图，由顶点、边、属性和标签组成^[47]。顶点也被称为节点，代表着某个个体或者事物。边代表着节点间的联系关系。上节中的图相比，属性图多出了属性和标签。属性是一个 **key** 和 **value** 的键值对，节点和边都可以拥有多种属性，节点的属性指的是这个个体或者事物拥有某种特征特点，而边的属性可以用来说明节点间关系的特点。标签指的是节点或者边具有某些共性，可以将这些节点或者边归为一类。那么拥有相同标签的节点属于同一个集合。

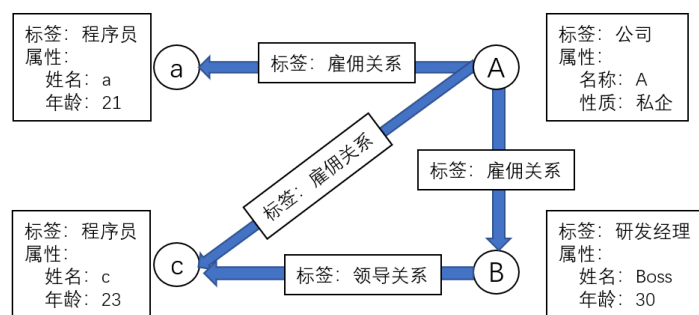


图 2-6 属性图示例

常见的属性图结构如图 2-6 所示，该图描述了员工与公司之间的关系。图中

存在一个节点 A，标签为公司，拥有两种属性名称和性质。图中还有三个节点 a，B，c，代表三个公司员工，其中节点 a 和节点 c 的标签为程序员，节点 B 的标签为研发经理。每个公司员工节点都拥有两种属性：姓名和年龄。图中共有两种边：雇佣关系和领导关系。公司 A 和三位员工拥有着雇佣关系，并且员工 B 是员工 c 的上级领导，存在领导关系。

2.2.3 代码属性图

Yamaguchi 等人于 2014 年首次提出代码属性图，之后不断对其进行补充完善^[48]。代码属性图包含了抽象语法树（Abstract Syntax Code，AST）、控制流图（Control Flow Graph，CFG）、数据依赖图（Data Dependence Graphs，DDG）、控制依赖图（Control Dependence Graphs，CDG）、程序依赖图（Program Dependence Graph，PDG）。代码属性图是一种基于属性图的代码中间表示形式。

通过将源代码进行语法分析、词法分析生成抽象语法树 AST。抽象语法树以树结构表现源代码的语法结构，抽象语法树的节点代表着词法信息，节点与节点间的关系则是对应着语法信息。在抽象语法树 AST 属性图的基础上引入控制流图 CFG、控制依赖图 CDG、数据依赖图 DDG、程序依赖图 PDG 来构建代码属性图。在代码属性图中，顶点表示字面量、变量、函数名、文件名等信息，图中的边表示语法结构、控制流、控制依赖信息，这些顶点和边组成的图结构抽象表示展示了源代码的多种语法、语义信息。

2.3 本章小结

本章详细介绍了编译原理与图相关理论基础。编译原理中的词法分析和语法分析为下一章关联代码属性图的构建提供了技术依据，图相关理论为代码属性图的关联提供理论基础。

第三章 关联代码属性图构建技术研究

源代码的语义信息涉及变量类型、变量声明、各种语句表达式、函数声明、函数定义、注释信息等语法要素，以及顺序结构、分支结构、循环结构、函数调用关系、数据依赖关系等语义要素。而且源代码的这些语法、语义要素蕴含了全面的漏洞信息。本文将这些语法、语义要素从源代码中抽取出来，将之有序组织起来构建以函数为基本单位的代码属性图。为了实现跨函数漏洞检测，本文将具有调用关系的函数关联起来，构建了一种改进的代码属性图——关联代码属性图。关联代码属性图为第四章基于深度学习的源代码漏洞检测技术研究提供图分类数据。本章将从代码属性图构建技术、代码属性图关联技术和关联代码属性图的存储这三个方面详细介绍关联代码属性图构建技术。

3.1 代码属性图构建技术

代码属性图（CPG，Code Property Graph）最早是 Yamaguchi 在 2014 年提出的，是一种基于抽象语法树 AST，结合控制流图 CFG、程序依赖图 CPG、控制依赖图 CDG 和数据依赖图 DDG 构建的属性图^[48]。

相较于 token、树等代码中间表示形式，代码属性图中包含了源代码中更多的语法、语义、语序等信息，因此在漏洞检测方面应用较广。在基于代码属性图的源代码漏洞检测中，既可以使用基于规则的图遍历方法，也可以使用基于代码相似性的子图匹配方法，还可以使用基于深度学习的模型检测方法。

3.1.1 代码属性图构建技术方案

代码属性图的构建技术主要研究如何将源代码中的各种语法、语义、语序等信息抽取出来，以节点和节点间的关系进行表示。

整个过程如图 3-1 所示，首先对源代码进行编译，通过词法分析、语法分析等操作将源代码转化为代码中间表示形式——抽象语法树（AST）。词法分析会对源代码进行从左到右、从上到下的单词扫描，通过内置的词法规则对单词进行识别、分类。语法分析在词法分析的基础上，通过自上而下语法分析方法或自下而上语法分析方法对单词序列进行语法识别，生成语法树。随后，在抽象语法树的基础上，引入控制流图 CFG、控制依赖图 CDG、数据依赖图 DDG 和程序依赖图 PDG，构建源代码对应的代码属性图。

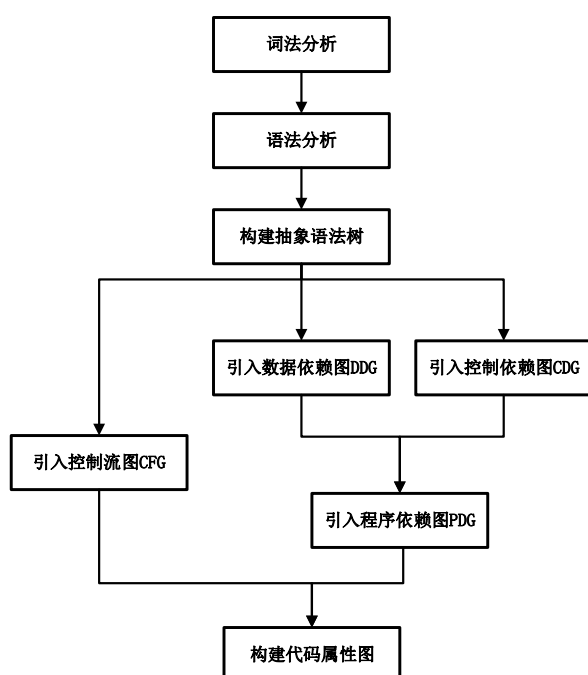


图 3-1 代码属性图构建的技术方案

3.1.2 抽象语法树 AST 属性图

抽象语法树（Abstract Syntax Tree，AST）是对源代码进行词法分析、语法分析后生成的一种代码中间表示^[49]。抽象语法树以树结构表现源代码的语法结构，抽象语法树中的节点代表着源代码中运算符、标识符、变量、返回值等不同的语法结构，节点与节点间的关系则是对应着语法信息。基于抽象语法树可以精准地定位到声明语句、赋值语句、运算语句等，进而实现对源代码的分析、优化、变更等操作。

单个语句、代码段或者整个函数代码都可以用抽象语法树来表示。本文以函数为单位，进行抽象语法树的相关定义。本文定义一个函数的抽象语法树 AST 为图 $G_{AST}=(V_{AST}, E_{AST})$ ，其中 V_{AST} 表示 AST 图中的节点，除了根节点 F 表示函数本身外，每个节点都表示了一个标识符。 E_{AST} 表示 AST 图中的边，如果两个节点之间存在语法关系，则这两个节点就会有一条 AST 有向边进行连接。

抽象语法树 AST 属性图 $G_A=(V_A, E_A, \lambda_A(\cdot), \mu_A(\cdot))$ 是在抽象语法树 AST 的基础上，为 V_A 中的节点添加类型、行号、列号等属性和为 E_A 添加边标签得到的。属性计算函数 $\mu_A: V_A \times K_A \rightarrow S_A$ ，其中 K_A 为属性集合， S_A 为属性值集合。标签计算函数为 $\lambda_A: E_A \rightarrow \Sigma_A$ ，其中类型集合 $\Sigma_A=\{AST\}$ 。表 3-1 示例代码对应的抽象语法树 AST 属性图如图 3-2 所示。

表 3-1 示例代码

```

1: void good(){
2:     int data;
3:     data = 0;
4:     goodsink(data);
5: }
6: void goodsink(int data){
7:     if(data != 0){
8:         int result = 100 / data;
9:         print (result);
10:    }
11: }

```

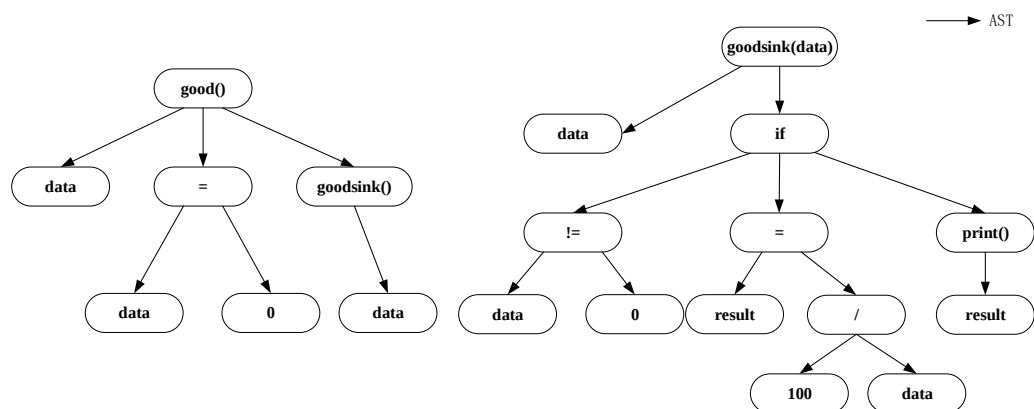


图 3-2 示例代码对应的抽象语法树 AST

图 3-2 中两个抽象语法树属性图分别对应 `good()` 函数和 `goodsink()` 函数，根节点代表着函数节点，与函数节点相连的节点为每一行语句对应的根节点。例如语句 `data = 0` 则可以分成三个节点，变量 `data` 节点，赋值操作符 `=` 节点和常量 `0` 节点。赋值操作符 `=` 节点通过边连接其余两个节点，又向上与根节点 `good()` 函数进行相连。

3.1.3 控制流图 CFG 属性图

控制流图（Control-Flow Graph, CFG）是程序执行过程和代码执行顺序的抽象表现^[50]。控制流图使用图符号和有向边表示程序执行过程中可能经过的所有路径。每个控制流图都存在 2 个指定的块：Entry Block(输入块)，Exit Block(输出块)。控制流图中会用有向边来表示程序中的执行顺序。

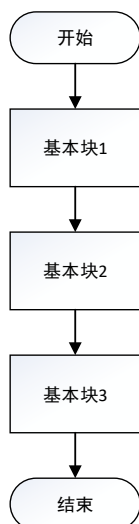


图 3-3 控制流图的顺序结构

控制流图有四种基本结构：顺序结构、选择结构、循环结构和跳转结构。图 3-3 表示控制流图的顺序结构，每一个基本块表示源代码的每一行语句，基本块内无选择、跳转、循环等指令。源代码将按照语句的前后关系进行顺序执行。

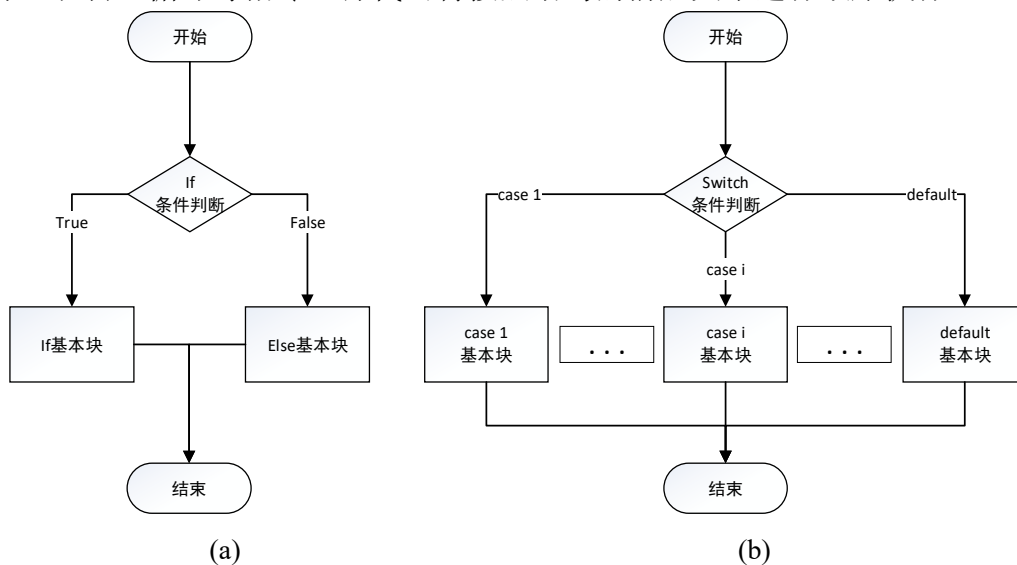


图 3-4 控制流图的选择结构。(a)if 判断；(b)switch 判断

如图 3-4 所示，控制流图的选择结构有两种：if 判断和 switch 判断。当程序执行到 if 代码段或者 switch 代码段时，会根据给定的判断条件，进行逻辑判断，根据得到的 bool 类型，选择不同的分支往下运行。

如图 3-5 所示，控制流图的循环结构有三种：for 循环、while 循环和 do while 循环。当源代码执行到 for 循环和 while 循环时，会先给定的循环变量进行循环条件的判断。如果符合循环条件，则一直进行循环体的代码执行，并对循环遍历进

行修改。当循环遍历的值不再满足循环条件时，就会退出当前循环体代码，执行后续代码或者退出。而 **do while** 循环则是先进行 **do** 循环体的执行，然后再对循环变量进行循环判断，满足条件则继续进行循环，直至不满足循环条件时，退出循环，执行后续代码或者退出。

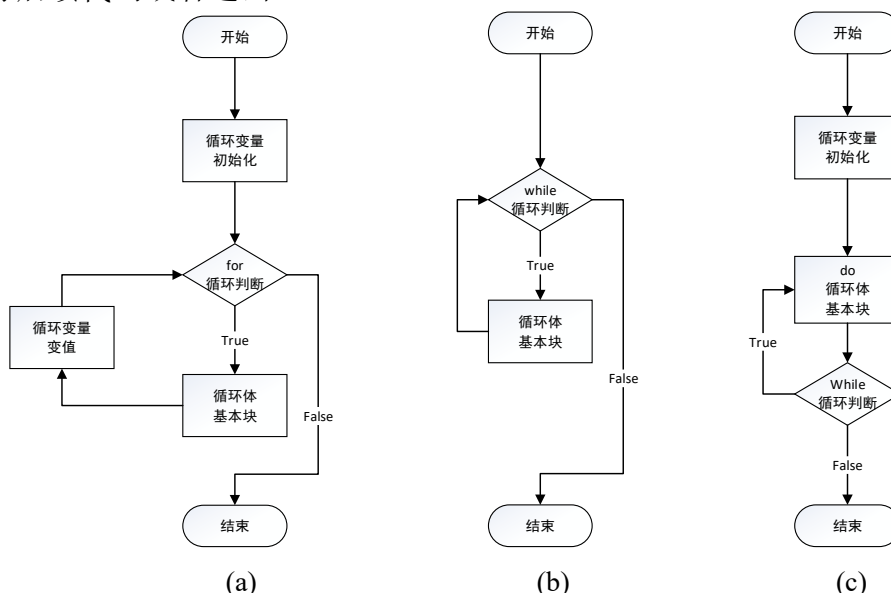


图 3-5 控制流图的循环结构。(a)for 循环结构；(b)while 循环结构；(c)do-while 循环结构

控制流图还有一种结构是跳转结构，包含三种跳转指令：**break**、**continue**、**return**。跳转结构可以与上述三种结构任意结合进行使用。**break** 指令用于跳出单层循环，若有多层循环，则退出 **break** 指令所在的内层循环。**break** 指令也用于结束 **switch** 语句，通常配合 **if** 判断使用。**continue** 指令用于退出循环的一次迭代过程，继续进行下次循环。**return** 用于结束一个方法或者跳出循环体，全部终止。

程序控制流图是分析程序运行情况的重要依据，表示了程序的语法信息，通常和函数的复杂程度有关。本文定义控制流程图 $G_{CFG}=(V_{CFG}, E_{CFG})$ ，其中 V_{CFG} 表示 CFG 中的程序基本块， E_{CFG} 表示 CFG 中的边，如果节点 m 到 n 之间是可达的，说明这两个节点之间存在控制关系。表 3-1 所示代码对应的控制流图 CFG 如图 3-6 所示。

控制流图 CFG 属性图 $G_C=(V_C, E_C, \lambda_C(.), \mu_A(.))$ 是在抽象语法树 AST 属性图的基础上抽取相应的点，基于控制流程 CFG 添加边，并添加标签计算函数 $\lambda_C:E_C \rightarrow \Sigma_C$ 获得的，其中类型集合 $\Sigma_C=\{true, false, \varepsilon\}$ 。表 3-1 所示代码对应的控制流图 CFG 属性图如图 3-7 所示。

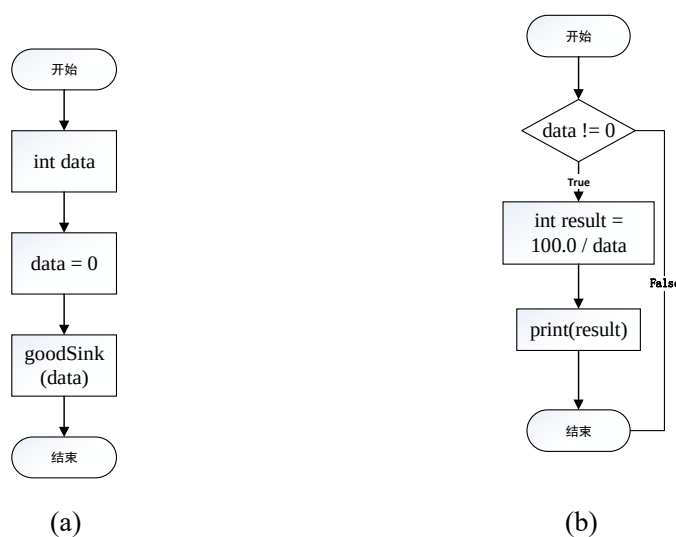


图 3-6 示例代码对应的 CFG。(a)good()函数对应的 CFG；(b)goodsink()函数对应的 CFG

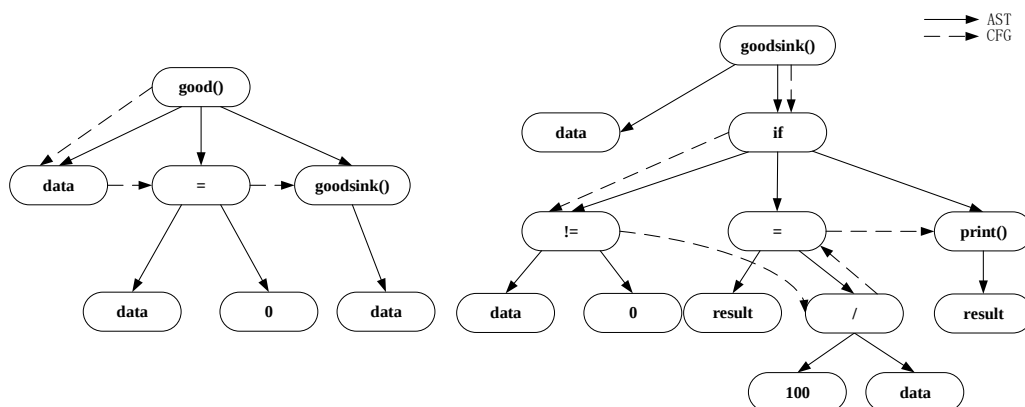


图 3-7 示例代码对应的 CFG 属性图

3.1.4 程序依赖图 PDG 属性图

程序依赖图（Program Dependence Graph，PDG）也是一种基于图的代码中间表示形式，是一种带有程序依赖信息有向多重图^[51]。程序中的依赖关系包含程序的控制依赖和数据依赖关系，因此程序依赖图由控制依赖图和数据依赖图组成。

程序依赖图定义为 $G_{PDG}=(V_{PDG}, E_{PDG})$ ，其中 V_{PDG} 表示 PDG 中的控制节点和标识符， E_{PDG} 表示 PDG 中的边，如果 PDG 图中的节点之间存在边，说明这两个节点之间存在控制依赖关系或者数据依赖关系。

程序依赖图 PDG 属性图 $G_P=(V_P, E_P, \lambda_P(\cdot), \mu_A(\cdot))$ 是在控制流图 CFG 属性图点集 V_C 的基础上，基于程序依赖图 PDG 添加边，并添加标签计算函数 $\lambda_P: E_P \rightarrow \Sigma_P$ ，和属性计算函数 $\mu_P: V_P \times K_P \rightarrow S_P$ 获得的，其中类型集合 $\Sigma_P=\{CDG, DDG\}$ 。

3.1.4.1 控制依赖图 CDG

控制依赖图（Control Dependence Graph, CDG）是以条件判断语句为根节点，以对应条件下的代码段语句为中间节点的有向图。控制依赖图的边代表程序中控制依赖的信息。边主要存在与 if、else、switch 等条件判断节点和 if、else、case 等基本块语句间。与根节点相连的节点代表着在某个条件成立的情况下节点对应的代码会被执行，即这些语句的执行依赖于该条件的成立。

控制依赖图 CDG 与控制流图 CFG 的区别在于，控制依赖图 CDG 只表示源代码中存在条件判断时的控制依赖信息，但不包含语句的前后执行顺序；而控制流图 CFG 不仅包含条件判断及其基本块间的控制流信息，还包含整个程序的控制流信息，并且可以体现语句的前后执行顺序。

3.1.4.2 数据依赖图 DDG

数据依赖表示程序中数据的依赖关系，即对某个变量进行定义、操作间数据的传递。例如在程序中先定义了一个变量 a，随后对变量 a 进行加减乘除运算。在进行加减乘除运算时不能凭空出现变量 a，在使用变量 a 前需要先对其进行定义，此时称加减乘除运算中的变量 a 数据依赖于定义语句中的变量 a，后续对 a 的操作又依赖于加减乘除运算中的变量 a。由变量等节点和数据依赖关系构建的图就是数据依赖图。

将上述控制依赖图 CDG 和数据依赖图 DDG 结合起来就形成了控制依赖图 PDG。表 3-1 示例代码中 goodSink() 所对应的程序依赖图 PDG 属性图如图 3-8 所示。

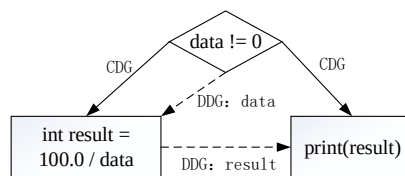


图 3-8 示例代码中 goodSink() 对应的 PDG

3.1.5 代码属性图

代码属性图 $G_{CPG}=(V_{CPG}, E_{CPG}, \lambda_{CPG}(.), \mu_{CPG}(.))$ 由抽象语法树 AST 属性图 $G_A=(V_A, E_A, \lambda_A(.), \mu_A(.))$ 、控制流图 CFG 属性图 $G_c=(V_c, E_c, \lambda_c(.), \mu_c(.))$ 、程序依赖图 PDG 属性图 $G_p=(V_p, E_p, \lambda_p(.), \mu_p(.))$ 通过以下公式 3-1、3-2、3-3、3-4 集合运算获得：

$$V_{CPG} = V_A \cup V_C \cup V_P \quad (3-1)$$

$$E_{CPG} = E_A \cup E_C \cup E_P \quad (3-2)$$

$$\mu_{CPG} = \mu_A \cup \mu_C \cup \mu_P \quad (3-3)$$

$$\lambda_{CPG} = \lambda_A \cup \lambda_C \cup \lambda_P \quad (3-4)$$

其中 V_{CPG} 为代码属性图中的节点集合， E_{CPG} 为边集合， λ_{CPG} 和 μ_{CPG} 分别代表着其标签类型集合和属性集合。表 3-1 示例代码对应的代码属性图如图 3-9 所示。

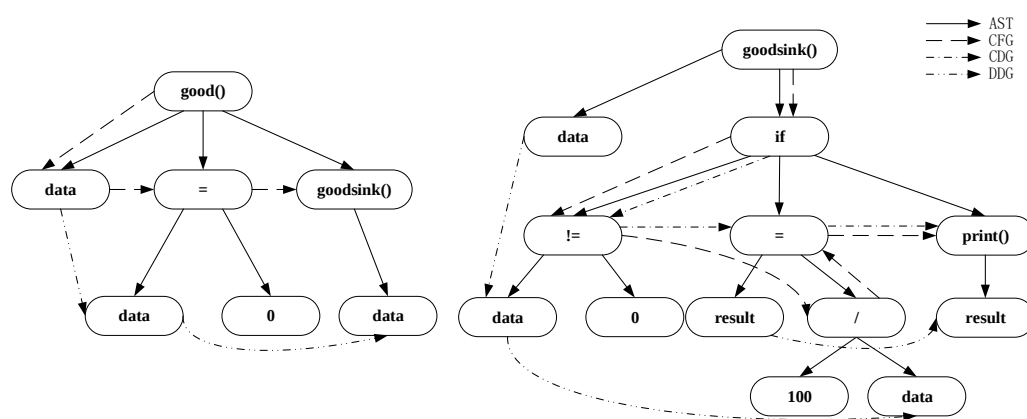


图 3-9 示例代码对应的代码属性图

3.2 代码属性图关联技术

3.2.1 关联代码属性图

上述代码属性图的构建方案是基于单个函数的，函数间并无关联关系。但是实际源代码中各个功能是由基于单个或者多个函数进行实现的，各个函数之间是有关联的，部分函数是存在函数调用关系的。主调函数使用被调函数的功能，称为函数调用。在 C 语言中，只有在函数调用时，函数体中定义的功能才会被执行。因此，只是基于单个函数的代码属性图缺少函数间的参数传递，执行顺序等要素，还不足以展示整个程序逻辑执行过程。并且漏洞不仅仅存在于单个函数中，还存在于调用函数与被调用函数之间，因此需要将调用函数与被调用函数进行函数调用关联和融合。

为了实现跨函数漏洞检测，本文提出一种改进的代码属性图——关联代码属性图（Associated Code Property Graph, ACPG）。关联代码属性图是一种基于代码属性图 CPG，结合函数调用图 FCG，关联调用函数与被调用函数对应代码属性图

的控制流图 CFG 和数据依赖图 DDG 的属性图，其定义为 $G_{ACPG}=(V_{ACPG}, E_{ACPG}, \lambda_{ACPG}(.), \mu_{ACPG}(.))$ 。其中 V_{ACPG} 为关联代码属性图中的节点集合，与代码属性图的节点集合 V_{CPG} 相同。 E_{ACPG} 为边集合，与代码属性图边集合 E_{CPG} 有所不同，更改了 E_{CPG} 中的 CFG 边，添加了 FCG 边和 DDG 边。 λ_{ACPG} 和 μ_{ACPG} 分别代表着其标签类型集合和属性集合。关联代码属性图实质上也是一种有向多重图。

关联代码属性图通过 FCG 边将调用函数与被调用函数关联起来，体现了函数间的调用关系。关联调用函数与被调用函数对应代码属性图的 CFG 边，体现了程序真实执行过程中的控制流信息。关联调用函数与被调用函数对应代码属性图的 DDG 边，体现了函数间参数传递的数据流信息。

在代码属性图的基础上，关联代码属性图的构建主要使用代码属性图关联技术。该技术包括函数调用图 FCG 属性图添加技术，控制流图 CFG 关联技术，数据依赖图 DDG 关联技术三个方面。

3.2.2 函数调用图 FCG 属性图添加技术

函数调用图 (Function Call Graph, FCG) 可以展示源代码中函数之间的调用关系^[52]。在函数调用图 $G_{FCG}=(V_{FCG}, E_{FCG})$ 中， V_{FCG} 表示 FCG 的节点，代表函数节点或者是调用函数节点； E_{FCG} 表示 FCG 中的边，如果 FCG 图中的节点之间存在边，说明这两个函数之间存在调用关系。例如边 (f, g) 表示调用函数节点 f 调用函数 g 。若其中有出现互相调用的环，表示程序中可能有递归调用。

函数调用图 FCG 属性图由 $G_F=(V_F, E_F, \phi, \phi)$ 是在程序依赖图 PDG 属性图点集 V_P 的基础上抽取相应的点（函数调用节点和函数定义节点），添加函数调用边组成。

$$V_F=\{v|v\in V_P, v \text{ 为函数调用节点或函数定义节点}\}$$

$$E_F=\{\langle v_i, v_j \rangle | v_i \in V_F, v_j \in V_F, v_i \text{ 调用 } v_j\}$$

当函数存在入参时，一般会在函数名后写明参数类型，参数名。参数可以有一个或者多个，各参数之间用逗号分隔。而调用函数中的调用语句则是直接在函数名后写参数名。需要注意的是，程序中可能会存在有多个同名函数的情况，这些同名函数功能不同且使用的参数列表不同，即参数的类型、个数或顺序不同。

基于程序的多态性，对函数调用关系 FCG 的添加算法 AddFCG 步骤如下：

(1) 寻找函数 f_0 对应代码属性图中的调用函数节点 a ，得到节点 a 所调用的函数名 f_1 ，参数列表，参数类型信息。

(2) 遍历所有代码属性图的根节点，即入度为 0 的节点，得到该节点对应的

节点信息，进行以下操作：

- a) 比对两个节点的函数名 f1 和 f2 是否相同，若相同，转 b)；若不同，转 d)；
- b) 比对两个节点的参数列表个数是否相同，若相同，转 c)；若不同，转 d)；
- c) 依次比对参数列表的每个参数类型是否相同，若全部相同，转 e)；若有一个或者多个不同，转 d)；
- d) 继续遍历，得到下个函数节点的节点信息；
- e) 输出调用函数节点和被调用函数节点。

(3) 将 (2) 中输出的调用函数节点和被调用函数节点使用 FCG 边进行关联，有向边的方向为调用函数节点指向被调用函数节点。

图 3-10 展示了表 3-1 中示例代码中添加了 FCG 关系的代码属性图。通过比对 good() 函数中 goodsink() 调用节点和 goodsink() 函数节点的参数列表和参数类型，将这两个节点使用 FCG 边进行关联。

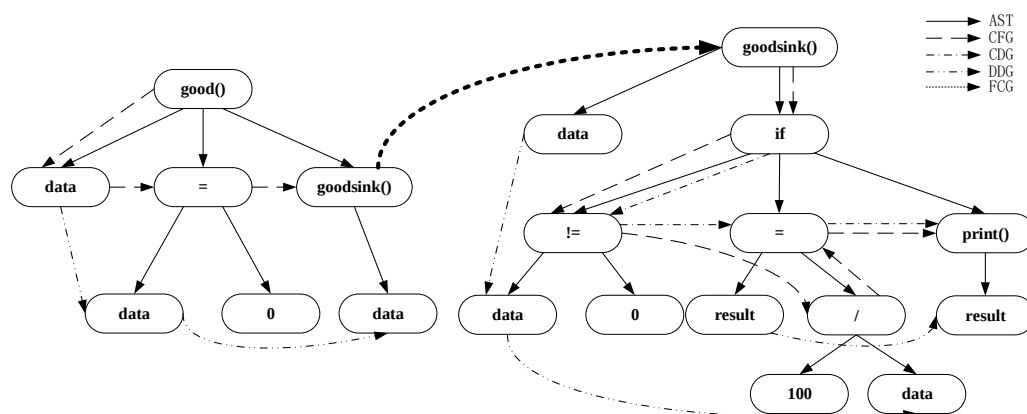


图 3-10 函数调用图 FCG 属性图的添加

3.2.3 控制流图 CFG 关联技术

当程序中存在函数调用时，程序的真实执行过程是从函数调用的节点转向被调用函数，当被调用函数执行完毕之后，再执行调用节点之后的代码程序。因此想要完整体现程序运行时的控制流信息，必须将调用函数和被调用函数对应的代码属性图中的控制流图 CFG 进行关联。

基于上述函数调用图 FCG 属性图添加的 AddFCG 算法，可以得到调用函数 F1 的调用节点 A 和被调用函数 F2 的函数节点 B。控制流图 CFG 关联技术主要是研究如何将节点 A 相关的 CFG 边与节点 B 相关的 CFG 边进行关联。

当源代码中一个函数的执行需要使用到另一个函数，那么当程序执行到调用函数语句时，会进入到被调用函数的代码块进行顺序执行。基于此思想，本文提出对控制流图 CFG 的调整算法 AssCFG 如下：

- (1) 通过代码属性图中的 FCG 边得到调用函数节点 a 和被调用函数节点 b；
- (2) 寻找调用函数节点 a 具有 CFG 关系的边，根据出边和入边得到父节点集 v1，子节点集 v2 和边集 e1；
- (3) 寻找被调用函数节点 b 具有 CFG 关系的边，根据出边得到节点 b 的子节点集 v3；
- (4) 通过 CFG 边遍历被调用函数节点 b 的代码属性图，得到 CFG 图中的最后一个节点 c，即入度为 1，出度为 0 的节点；
- (5) 将调用函数节点 a 的父节点集 v1 依次与被调用函数节点 b 的子节点集 v3 相连；
- (6) 将被调用函数节点 b 的节点 e 与调用函数节点 a 的子节点集 v2 依次相连；
- (7) 删除调用函数节点 a 的边集 e1；

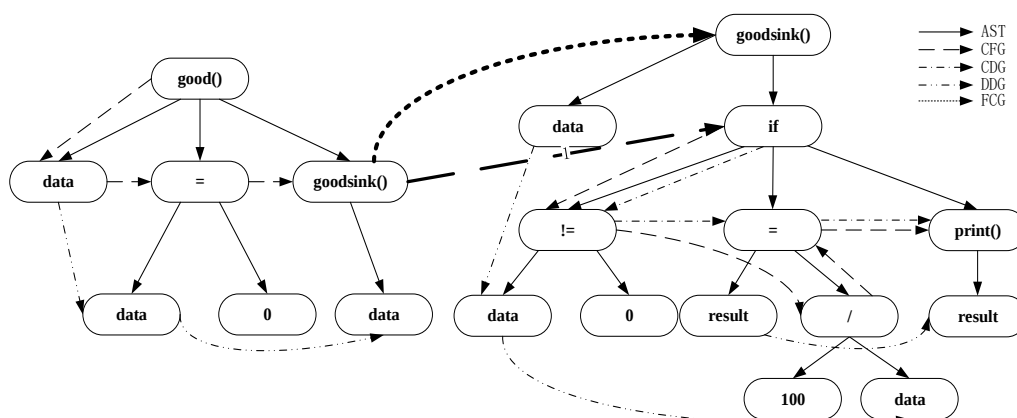


图 3-11 关联控制流图 CFG

对表 3-1 示例代码的控制流图 CFG 关联如图 3-11 所示，当 `good()` 函数执行到函数调用节点 `goodsink()` 时，整个示例代码的控制流从 `goodsink()` 节点流向了 `goodsink()` 函数中的 `if` 节点，代表着程序在真实执行过程中会跳转到 `if` 语句，进而进行 `if` 后续代码操作。

3.2.4 数据依赖图 DDG 关联技术

若调用函数与被调用函数之间存在参数传递，或者被调用函数有返回值，则调用函数和被调用函数对应的代码属性图中的数据依赖图 DDG 同样需要关联。对

数据依赖图 DDG 的关联算法 AssDDG 如下：

- (1) 通过代码属性图中的 FCG 边得到调用函数节点 a 和被调用函数节点 b；
- (2) 寻找调用函数节点 a 的子节点集 V1，即传递的参数节点集；如果调用函数节点的父节点为赋值节点等操作符节点，则将调用函数节点 a 的左兄弟节点记作节点 c。
- (3) 寻找被调用函数节点 b 的子节点集 v3，该子节点集即为被调用函数的参数节点集与其他语句根节点；若被调用函数存在返回值，则获取返回值对应的节点 d；
- (4) 按照对应的参数类型和参数顺序，将调用函数节点 a 的子节点集 v2 依次与被调用函数节点 b 的子节点集 v3 相连；
- (5) 按照返回值的类型，比对被调用函数的返回值节点 c 与调用函数的兄弟节点 c 进行关联；

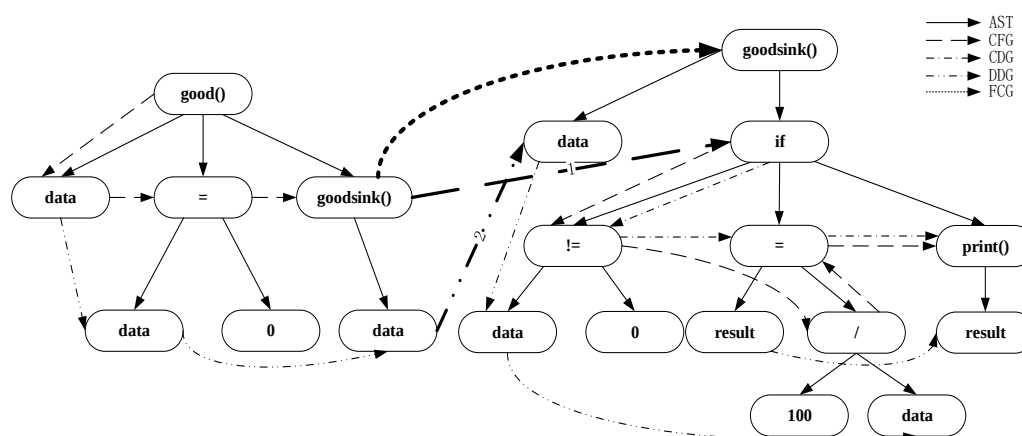


图 3-12 关联代码属性图

通过对代码属性图添加 FCG 函数调用图和调整控制流图 CFG、数据依赖图 DDG，逐步构建关联代码属性图。最终效果如图 3-12 所示，加粗显示的 CFG 边 1 和 DDG 边 2 即为对代码属性图调整的边。示例代码中的数据流通过关联的 DDG 边 2 从 good() 函数中最后一个变量 data 节点传入 goodsink() 函数中的参数 data 节点。这样可以对被调用函数的参数数据进行溯源。

3.3 关联代码属性图存储技术

基于上述代码属性图构建技术和代码属性图关联技术可以得到源代码对应的关联代码属性图。为了方便数据管理查看和为后续基于深度学习的漏洞检测提供

图分类数据，需要将关联代码属性图进行相应的存储。

关联代码属性图存储技术主要考虑如何对关联代码属性图中的多种类型的节点和较为复杂的有向边关系进行存储管理及可视化展示。

3.3.1 关联代码属性图内容信息

本文通过采用开源软件 Joern 对源代码通过词法分析、语法分析生成代码属性图，然后对代码属性图进行函数关联，得到关联代码属性图。

关联代码属性图中每个节点都有一个标签 `label`，表示该节点的类型。每个节点又包含 37 种属性，表示该节点对应类型的具体信息。节点与节点间存在着不同的边关系。图中的边关系共有 5 种，分别表示语法结构、控制流、数据流等信息。这些节点和边关系组成的图结构数据表示展示了源代码的多种语法、语义信息。

表 3-2 关联代码属性图节点 `label`

节点 <code>label</code>	含义
<code>BLOCK</code>	块
<code>CALL</code>	函数
<code>COMMENT</code>	注释
<code>CONTROL_STRUCTURE</code>	控制结构
<code>FIELD_IDENTIFIER</code>	成员变量
<code>FILE</code>	文件
<code>IDENTIFIER</code>	变量
<code>JUMP_TARGET</code>	跳转
<code>LITERAL</code>	常量
<code>LOCAL</code>	本地变量
<code>MEMBER</code>	成员
<code>META_DATA</code>	元数据
<code>METHOD</code>	方法
<code>METHOD_PARAMETER_IN</code>	方法入参
<code>METHOD_PARAMETER_OUT</code>	方法出参
<code>METHOD_RETURN</code>	方法返回
<code>NAMESPACE</code>	命名空间
<code>NAMESPACE_BLOCK</code>	块
<code>RETURN</code>	返回
<code>TYPE</code>	类型
<code>TYPE_DECL</code>	类型声明
<code>UNKNOWN</code>	未知

节点共有块、函数、变量、入参、返回类型等 22 种不同 `label`，具体 `label` 值

和含义如表 3-2 所示。

每个节点都包含节点 id、行号信息、列号信息等 37 种属性，每个节点所有的属性及其具体含义如表 3-3 所示：

表 3-3 关联代码属性图节点属性

属性	含义
ID	节点 id
ALIAS_TYPE_FULL_NAME	别名全程
ARGUMENT_INDEX	参数索引
AST_PARENT_FULL_NAME	父节点全名
AST_PARENT_TYPE	父节点类型
BINARY_SIGNATURE	二进制签名
CANONICAL_NAME	全名
CLOSURE_BINDING_ID	绑定闭包 ID
CODE	代码
COLUMN_NUMBER	节点所在列号
COLUMN_NUMBER_END	节点结束列
DEPTH_FIRST_ORDER	深度优先顺序
DISPATCH_TYPE	分派类型
DYNAMIC_TYPE_HINT_FULL_NAME	动态类型 hint 全名
EVALUATION_STRATEGY	求值策略
FILENAME	文件名
FULL_NAME	全称
HAS_MAPPING	哈希匹配
INHERITS_FROM	内部来源
INTERNAL_FLAGS	内部标志
IS_EXTERNAL	是否为外部
LANGUAGE	程序语言
LINE_NUMBER	节点所在行号
LINE_NUMBER_END	节点结束行号
METHOD_FULL_NAME	节点方法全名
METHOD_INST_FULL_NAME	方法 hint 全名
NAME	命名
ORDER	语句顺序
OVERLAYS	重复占位段
PARSER_TYPE_NAME	解析器全名
POLICY_DIRECTORIES	目录策略

表 3-3 关联代码属性图节点属性（续）

属性	含义
RESOLVED	是否识别
SIGNATURE	标志
SPID	系统进程
TYPE_DECL_FULL_NAME	类型声明全名
TYPE_FULL_NAME	类型全名
VERSION	版本

关联代码属性图包含了抽象语法树 AST、控制流图 CFG、程序依赖图 PDG、控制依赖图 CDG、数据依赖图 DDG 和函数调用图 FCG，因此图中节点间边关系共有六种。边的类型及其具体含义如表 3-4 所示：

表 3-4 关联代码属性图本体关系类型

类型	含义
AST	抽象语法树的边
CFG	控制流图的边
PDG	程序依赖图的边
CDG	控制依赖图的边
DDG	数据依赖图的边
FCG	函数调用图的边

根据开源数据集中的漏洞信息，在构建的全要素代码语义属性图中添加漏洞标签信息 y ， y 为 0 表示函数没有漏洞，为 1 表示函数存在漏洞。最后，将包含标签信息的语义属性图集合存入图数据库，并建立相应的索引，形成高效的源代码关联代码属性图的管理方案。

3.3.2 关联代码属性图存储方案

目前数据库主要分为两种：关系型数据库和非关系型数据库。关系型数据库可以简单认为是由二维表和表间关系组建的数据库。例如可以用学生基本信息表、教师基本信息表、班级信息表、任课信息表等表示学生和老师之间的对应关系。NoSQL 非关系型数据库使用键值对来存储数据，每个元组都包含不同的键值对，结构较为灵活。非关系型数据库在查询多种信息时，只需要根据 key 来取出对应的 value 值即可。

关联代码属性图的存储如果使用关系型数据库会需要多张二维表来进行节点和边关系的存储，结构复杂且表结构不易修改。非关系型数据库的模式设定较为灵活，在存储设计方面相比于关系型数据库更加便捷，因此本文选择非关系型数

数据库作为关联代码属性图的存储方案，具体选用了 Neo4j 图数据库。

Neo4j 图数据库具有存储方便、查询高效、能可视化的优点。Neo4j 图数据库只有节点和边两种数据类型，因此在进行关联代码属性图存储的时候，只需要将节点信息和边信息分别抽取出来形成点表和边表，再使用其导入工具直接命令行导入即可。Neo4j 图数据库在存储的时候会对每一个节点分配一个 id，并且其底层存储中每个节点记录的长度都是固定的，在查询节点时可以根据 id 快速计算存储位置，获取节点信息。Neo4j 图数据库还具有可视化工具 Neo4j Browser，可以在前端页面查看图结构或节点的详细信息。

3.3.3 关联代码属性图可视化展示

通过 Neo4j Browser 可以直观清楚的查看关联代码属性图中的节点和边关系，查找源代码漏洞对应子图，便于后续的分析研究。以表 3-5 的代码片段为例，其所对应的关联代码属性图如图 3-13 所示。

表 3-5 CWE369 示例漏洞代码

```

1: void CWE369_Divide_by_Zero__float_fgets_01_bad(){
2:     float data;
3:     data = 0.0F;
4:     char inputBuffer[CHAR_ARRAY_SIZE];
5:     if (fgets(inputBuffer, CHAR_ARRAY_SIZE, stdin) != NULL){
6:         data = (float)atof(inputBuffer);
7:     } else{
8:         printLine("fgets() failed.");
9:     }
10:    int result = (int)(100.0 / data);
11:    printIntLine(result);
12: }
```

图 3-13 展示了示例代码对应的关联代码属性图信息，图中共有 106 个节点，不同 label 的节点使用不同颜色进行了标注，节点间共有 AST、CFG、CDG、DDG、PDG 五种边信息。在 Neo4j Browser 点击节点，可以在页面下方查看节点的相关信息。例如图中的 data 变量节点，Neo4j 图数据库为其添加的 id 为 17，节点本身的 label 为 local 类型，NAME 属性为 data，TYPE_FULL_NAME 属性为 float，节点自身 id 为 10000104。

3.4 本章小结

36

第四章 基于深度学习的源代码漏洞检测技术

相较于基于传统的机器学习的漏洞分析方法，基于深度学习的漏洞分析可以自动生成漏洞模式，从而减轻了手动定义功能的需求。并且在对大型软件源代码漏洞检测方面，基于深度学习的检测方法可以高效快速的处理庞大的代码量，具有较高的准确率和查全率。

基于关联代码属性图的差异性，本章将源代码漏洞检测归纳为图分类问题。基于深度学习中的图注意力网络和注意力机制，本文提出了 DMGGAT 漏洞检测模型和 LDMGGAT 漏洞定位模型，并进行了相关实验，证明了 DMGGAT 模型和 LDMGGAT 模型的正确性和有效性。

4.1 源代码漏洞检测技术原理

4.1.1 基于关联代码属性图的源代码漏洞检测技术原理

本文通过对 Juliet 数据集中的漏洞代码分析可知，存在漏洞的函数 F1 与不存在漏洞的函数 F2 实现的功能相同，其对应的关联代码属性图存在明显差异。因此源代码的漏洞检测实际上是对其对应的关联代码属性图的检测，可以归纳为图分类问题。

Juliet 数据集中每个漏洞文件中都存在一个 `bad()` 函数和 `good()` 函数。`bad()` 函数和 `good()` 函数代码相似，但 `bad()` 函数存在漏洞，而 `good()` 函数中不存在漏洞。如下文中 CWE369 除 0 漏洞样例代码所示：表 4-1 示例代码是存在漏洞的 `bad()` 函数，在 `100.0/ data` 除法操作时没有对除数做出判断。`data` 的值取决于参数 `Data` 的值，`Data` 可能为 0，会导致除 0 漏洞，其关联代码属性图如图 4-1 所示。表 4-2 示例代码是代码和功能与表 4-1 示例代码 `bad()` 函数相似但不存在漏洞的 `good()` 函数，在进行 `100.0/ data` 除法操作时前进行了非 0 判断，因此不管参数 `Data` 是否为 0，`good()` 函数都不会存在除 0 漏洞，其关联代码属性图如图 4-2 所示。

对比图 4-1 和图 4-2 可以发现，相较于 `good()` 函数，`bad()` 函数对应的关联代码属性图中缺少判断数据是否为零的子图，即控制流图缺少了 `if` 节点和判断边，导致了除零漏洞的出现。这种差异不仅限于除零漏洞，也存在于其他类型的漏洞。只要关联代码属性图涵盖得足够全面的语法、语义要素，所有类型的漏洞都存在一定的差异。根据上述原理，可以将源代码漏洞检测归结为对关联代码属性图进行图分类，即判断关联代码属性图是无漏洞类型还是漏洞类型。

表 4-1 存在漏洞的 bad()函数代码

1:	static void bad(float Data){
2:	float data = Data;
3:	int result = (int)(100.0/ data);
4:	printIntLine (result);
5:	}

表 4-2 不存在漏洞的 good()函数

1:	static void good(float Data){
2:	float data = Data;
3:	if(fabs(data) > 0.000001){
4:	int result = (int)(100.0/ data);
5:	printIntLine (result);
6:	}else{
7:	printLine ("This would result in a divide by zero");
8:	}
9:	}

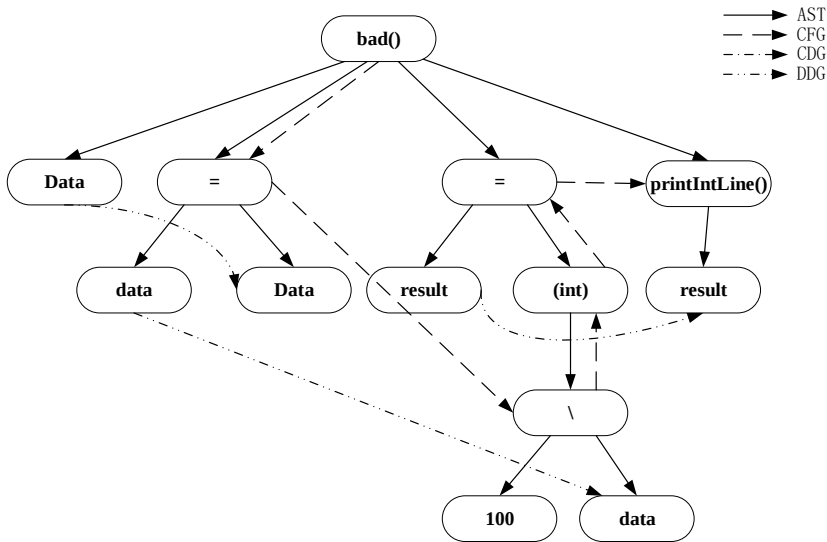


图 4-1 bad()函数的关联代码属性图

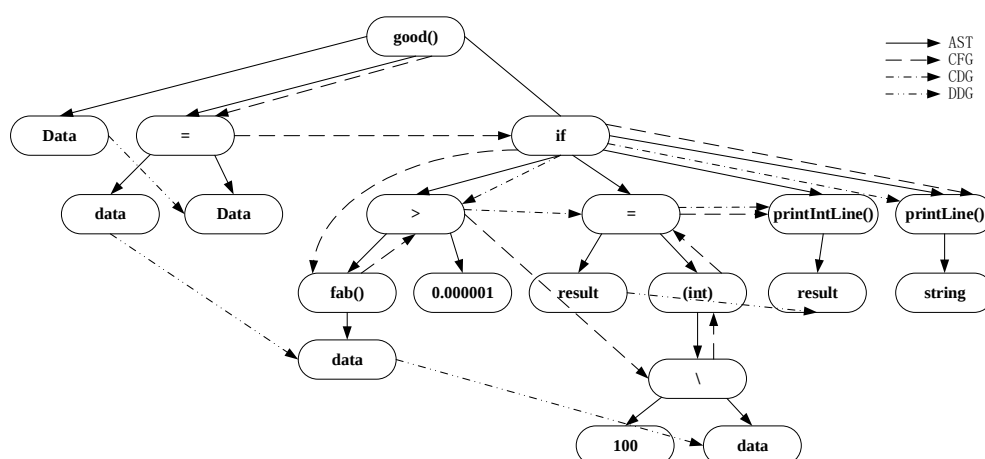


图 4-2 good()函数的关联代码属性图

4.1.2 基于注意力机制的源代码漏洞定位技术原理

注意力机制可以自动学习和优化节点间的连接关系^[53]。注意力机制使用注意力权重代替了原先非 0 即 1 的节点连接关系，即两个节点间的关系可以被优化为连续数值，从而获得更丰富的表达。并且 `attention` 值的计算是可以在节点间并行进行的，网络计算相当高效。

在使用基于图注意力网络的深度学习模型训练时，注意力机制会根据关联代码属性图中节点间的关系来分配相应的权重值，构成相应的注意力矩阵，如图 4-3 所示。



图 4-3 注意力矩阵色阶图

注意力矩阵中的权重值代表着模型对各个节点的注意力值。注意力值高的节点，代表着该节点对深度学习检测模型进行图分类时贡献度高。这些贡献度高的节点最终成为图分类的依据，极有可能是包含漏洞信息的节点。基于此思想，本文尝试使用基于注意力机制的注意力矩阵进行漏洞定位。

4.2 基于深度学习的源代码漏洞检测框架

基于深度学习的源代码漏洞检测技术的关键在于如何提取函数对应关联代码属性图的特征信息进行向量化，并选择合适的深度学习模型进行训练验证，实现漏洞检测与漏洞定位。基于关联代码属性图，本文提出了一种基于深度学习的源代码漏洞检测框架，如图 4-4 所示。

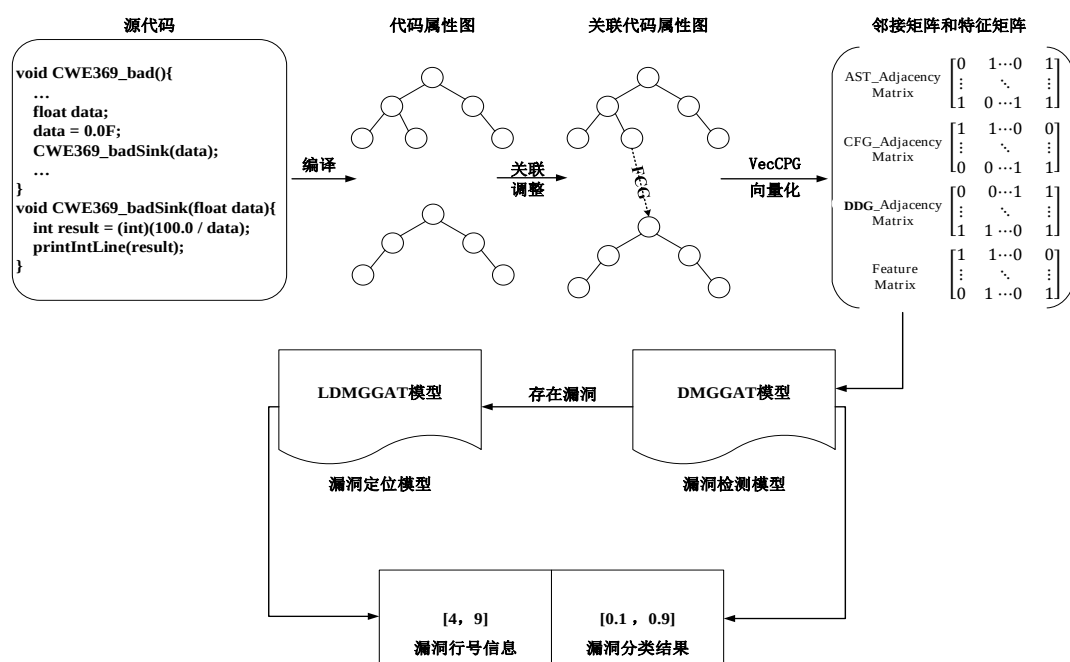


图 4-4 源代码漏洞检测框架图

从图 4-4 可以看出，基于深度学习的源代码漏洞检测技术首先需要将源代码通过词法分析、语法分析等编译操作生成代码属性图，添加 FCG 关联调用函数和被调用函数，调整 CFG、DDG 构建关联代码属性图，并基于数据集添加漏洞标签，为后续的深度学习检测模型提供图分类数据基础。其次，为尽可能提取出代码属性图中的词法、语法、语义等信息，使用 VecCPG 向量化方案对关联代码属性图进行特征抽取和向量化，得到关联代码属性图对应的特征矩阵和三种邻接矩阵。然后将函数对应的特征矩阵和邻接矩阵输入到 DMGGAT 漏洞检测模型中去。通过参数调试和训练验证，该模型能够对图数据进行二分类，以此判断函数中是否存在漏洞。若该函数存在漏洞，则将函数对应的特征矩阵和邻接矩阵输入到 LDMGGAT 漏洞定位模型中。LDMGGAT 漏洞定位模型基于注意力矩阵和 IQR 算法对节点进行分类，结合节点对应的行号信息来实现漏洞定位效果。

基于深度学习的源代码漏洞检测技术可以分为三个方面：基于关联代码属性

图的 VecCPG 向量化技术、基于图注意力网络的 DMGGAT 漏洞检测模型构建和基于图注意力网络的 LDMGGAT 漏洞定位模型构建。

4.3 基于关联代码属性图的 VecCPG 向量化技术

向量化的好坏直接影响到后续漏洞检测及定位模型的效果。源代码不是简单的文本信息，其中蕴含着较多的语法信息、语义信息、控制信息、调用信息、依赖信息等要素。如何将这信息要素抽取出来，形成较为全面的源代码向量化信息是漏洞检测的重要基础。因此，本文提出一种基于关联代码属性图的 VecCPG 向量化技术，对节点和边进行向量化构建对应的特征矩阵和邻接矩阵，用以全面反映关联代码属性图的主要特征。

如图 4-5 所示，基于关联代码属性图的 VecCPG 向量化技术从 Label、函数、常数和类型四个方面来对节点进行向量化，所有节点的向量化形成了函数的特征矩阵。基于抽象语法树 AST、控制流程图 CFG、数据依赖图 DDG 对节点间的关系进行向量化，形成三种邻接矩阵。

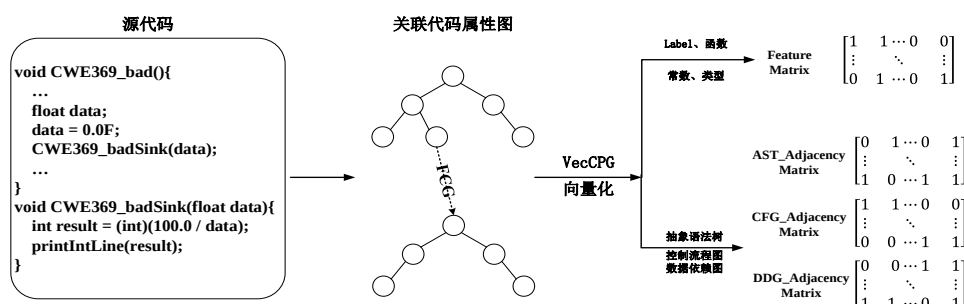


图 4-5 VecCPG 向量化过程

4.3.1 特征矩阵

特征矩阵（Feature Matrix）表示节点的信息，它捕获源代码的语法特征。对于给定的关联代码属性图 $G_{CPG}=(V_{CPG},E_{CPG},\lambda_{CPG},\Gamma_{CPG},\mu_{CPG})$ ，其特征矩阵定义如公式 4-1 所示。

$$X=R^{|V_{CPG}|\times|F|} \quad (4-1)$$

其中， $|V_{CPG}|$ 表示属性图中所有节点的数目， $|F|$ 表示每个节点的特征向量维度。每个节点的向量从标签信息、函数、常量及变量型四个方面进行源代码语法结构及语义信息的抽取，其结构如公式 4-2 所示：

$$\text{VecNode} = (\text{label}, \text{function}, \text{literal}, \text{type}) \quad (4-2)$$

代码属性图节点的特征向量化共有 143 位，如表 4-3 所示，下文将对各部分展开详细描述。

表 4-3 节点特征向量各部分构成

Label	函数	常量	类型	总
12	101	12	18	143

(1) **label**: 描述代码属性图中的节点类型信息。每个节点有且仅有一种类型，故采用独热方式进行编码。本方法考虑了 12 种节点类型，故 Label 共占 12 位。其标示符号及具体含义如表 4-4 所示：

表 4-4 标签及含义

名称	含义
BLOCK	块
CALL	函数调用
CONTROL_STRUCTURE	控制语句结构
FIELD_IDENTIFIER	命名空间的引用
IDENTIFIER	变量名
JUMP_TARGET	跳转
LITERAL	常量
LOCAL	局部变量
METHOD	方法定义
METHOD_RETURN	方法返回
RETURN	函数返回
UNKNOWN	未知类型

(2) **函数 (function)**: 函数包含操作符,系统函数和用户自定义函数,均使用独热编码进行编码,这三种类型均包含 1 位标志位，共有 101 位，如表 4-5 所示：

表 4-5 函数各部分构成

操作符	系统函数	用户自定义函数
1+59	1+39	1

a) **操作符 (operator)**: 软件源代码中变量或语句的具体操作，主要包括算数运算、关系运算、逻辑运算、位操作及指针操作。每个节点有且仅有一种运算类型，故采用独热方式进行编码。本方案考虑了 59 种运算符号，编码长度为 60 位，即 59 个操作符、1 位标志位。其标示符号如表 4-6 所示：

表 4-6 操作符

操作符	操作符
<operator>.addition	<operator>.logicalAnd
<operator>.addressOf	<operator>.logicalNot
<operator>.and	<operator>.logicalOr
<operator>.arithmeticShiftRight	<operator>.logicalShiftRight
<operator>.assignment	<operator>.memberAccess
<operator>.assignmentDivision	<operator>.minus
<operator>.assignmentMinus	<operator>.modulo
<operator>.assignmentMultiplication	<operator>.multiplication
<operator>.assignmentPlus	<operator>.not
<operator>.cast	<operator>.notEquals
<operator>.compare	<operator>.or
<operator>.computedMemberAccess	<operator>.plus
<operator>.conditional	<operator>.pointerShift
<operator>.delete	<operator>.postDecrement
<operator>.division	<operator>.postIncrement
<operator>.equals	<operator>.preDecrement
<operator>.exponentiation	<operator>.preIncrement
<operator>.fieldAccess	<operator>.shiftLeft
<operator>.getElementPtr	<operator>.sizeOf
<operator>.greaterEqualsThan	<operator>.subtraction
<operator>.greaterThan	<operator>.xor
<operator>.indexAccess	<operators>.assignmentAnd
<operator>.indirectComputedMemberAccess	<operators>.assignmentArithmeticShiftRight
<operator>.indirectFieldAccess	<operators>.assignmentExponentiation
<operator>.indirectIndexAccess	<operators>.assignmentLogicalShiftRight
<operator>.indirection	<operators>.assignmentModulo
<operator>.indirectMemberAccess	<operators>.assignmentOr
<operator>.instanceOf	<operators>.assignmentShiftLeft
<operator>.lessEqualsThan	<operators>.assignmentXor
<operator>.lessThan	

b)系统函数（function）：function 为调用的系统 API 函数名称或其它自定义函

数名称，由于属性图中每个函数节点的名称有且仅有一个，故采用独热方式进行编码。本课题共选取了 39 个系统 API 函数，function 部分的编码长度为 41 位，其中每个 API 函数占据一位编码，加一位标志位，所有用户自定义函数占用一位编码。由于系统 API 函数众多，这里不再一一列举。

c)用户自定义函数（User Define Function, UDF）：用户自定义函数与系统函数相同，都是为了实现某些功能。但用户自定义函数不在代码通用标准库中，是由用户自己编写的。在节点信息中，除操作符和系统函数，其余的函数节点均代表用户自定义函数。

（3）常量（literal）：一些常数也是导致代码存在漏洞的因素，例如除零的漏洞。将代码属性图中的常量分为三类：字符，字符串，数值类型(如整数)，每种类型包含 1 位标志位，3 位内容位。

（4）变量类型（type）：本方案共考虑 10 种基本类型及 6 种修饰类型。10 种基本类型和 6 种修饰类型均独自采用独热编码方式。加上未知类型，type 部分的编码长度为 18 位，即 10 位基本类型、6 种修饰类型、1 种其它类型和空值。16 种类型名称及含义如表 4-7 所示：

表 4-7 类型及含义

名称	含义
int	基本类型，整数
float	基本类型，浮点数
double	基本类型，双精度浮点数
long	基本类型，长整数
char	基本类型，字符
string	基本类型，字符串
void	基本类型，任意
struct	基本类型，结构体
union	基本类型，联合体
signed	修饰类型，符号
unsigned	修饰类型，无符号
*	修饰类型，指针
array	修饰类型，数组
map	修饰类型，映射
vector	修饰类型，向量
other	其它

4.3.2 邻接矩阵

邻接矩阵（Adjacent Matrix）是表示节点之间相邻关系的矩阵。邻接矩阵的行数和列数由关联代码属性图中的节点数目确定。邻接矩阵 Adj 的定义公式如下：

$$\text{Adj} = \mathbf{R}^{|V_{\text{CPG}}| \times |V_{\text{CPG}}|} \quad (4-3)$$

在关联代码属性图中，任意两个节点间可能存在多种边关系。不同的边关系有着不同的涵义，因此在对节点间边关系构造邻接矩阵时，不能对各种边不进行区分。不区分边是指只要两个节点间存在任何边关系就将邻接矩阵中对应位置的元素置为 1，或者对边的数量进行简单相加。无论是全部置为 1 还是边数量相加，都无法对边所代表的语法结构、控制结构、依赖结构进行区分。因此，VecCPG 向量化技术选择将边关系单独进行向量化。

关联代码属性图中包含六种边关系：AST、CFG、DDG、CDG、PDG、FCG。

抽象语法树 AST 中蕴含了源代码的抽象语法结构，包含多种语法信息，是对源代码进行漏洞检测的基础，需要对其进行向量化构造 AST 邻接矩阵。

控制流图 CFG 则表示了程序的执行过程中可以遍历的所有路径。程序语句的执行过程十分重要，不能打乱顺序。例如在图 4-2 中，if(fabs(data) > 0.000001)语句与 int result = (int)(100.0/ data)的前后执行顺序不能颠倒。假如先做除法运算再进行非 0 判断，则程序还是存在漏洞。因此在漏洞检测时需要控制流图 CFG 信息，需要对其进行向量化构造 CFG 邻接矩阵。

数据依赖图 DDG 是数据间的依赖关系，代表着数据的传递与数值的流向。大部分漏洞都是因为数据赋值产生的，例如，除 0 漏洞的除数为 0 和缓冲区溢出的数据越界。因此数据依赖图的边信息也十分重要。控制依赖图 CDG 表示判断语句对于变量值的影响，是程序控制流导致的一种约束。因此需要对其进行向量化构造 DDG 邻接矩阵。

控制依赖图 CDG 和控制流图 CFG 在判断语句处的部分边信息重合，并且 CFG 包含更多的节点关系信息，因此可以不使用 CDG 来进行漏洞检测。程序依赖图 PDG 由数据依赖图 DDG 和控制依赖图 CDG 构成。因为在漏洞检测中选用了 DDG，不选用 CDG，所以针对 PDG 的使用实质上是对 DDG 的使用。

函数调用图 FCG 用以关联调用函数和被调用函数，并且关联代码属性图通过 FCG 实现了对 CFG、DDG 的调整，可以不选用 FCG。

因此，本文基于关联代码属性图的 VecCPG 向量化技术则选用了 AST、CFG、DDG 分别进行向量化，构造对应的邻接矩阵。邻接矩阵构造方法为若两个节点在关联代码属性图中存在一条边，则在邻接矩阵中对应的元素为 1。

4.4 基于图注意力网络的 DMGGAT 漏洞检测模型构建

为了更好地处理图结构类型的数据，基于图注意力网络，本文提出了一种名为 DMGGAT（Directed Multi Graph GAT）的深度学习模型来完成图的二分类，实现源代码的漏洞分析。DMGGAT 的结构如图 4-6 所示，包括三个多头 GAT 层、MLP 层和 softmax 层。

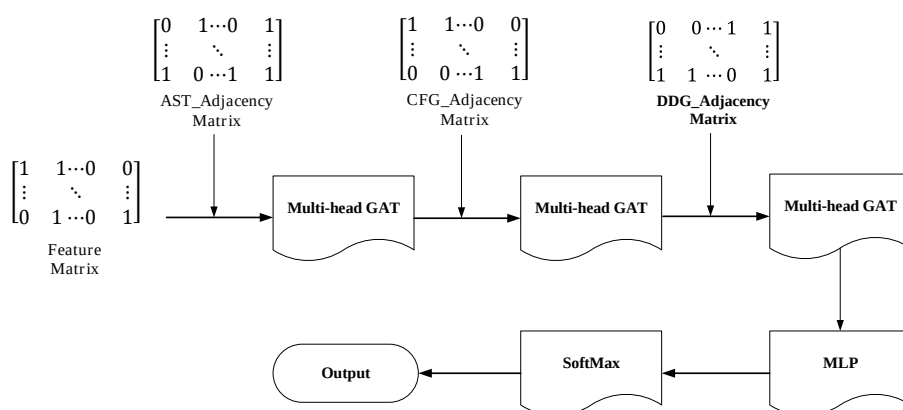


图 4-6 DMGGAT 结构

在传统的应用场景中，GAT（Graph Attention Networks，图注意力网络）的输入是一个特征矩阵和一个邻接矩阵，采用多头注意力机制提取潜在特征后，输出所有节点的特征向量并对不同的节点进行分类。然而，漏洞分析任务需要对图分类，而不是对图的节点进行分类。如果将 GAT 的输出特征矩阵直接展开为向量，输入到 MLP 中进行分类，可能会导致 MLP 参数过多，从而引发“高维诅咒”的问题。因此使用三个多头 GAT 层，输入不同的邻接矩阵，对特征向量进行更新降维。

向模型中输入邻接矩阵的先后顺序分别是 AST、CFG、DDG。在第一个多头 GAT 层使用 AST 邻接矩阵，是为了更新语句对应根节点的向量。例如 $a + b$ 语句，将变量 a 和变量 b 的信息融合到操作符 $+$ 中去，这样操作符 $+$ 节点的特征向量就代表着语句 $a + b$ 的语义信息。在第二个多头 GAT 层使用 CFG 邻接矩阵，是为了体现程序代码的真实控制流。控制流图 CFG 在关联代码属性图中的控制流向都是从当前语句的根节点流向下一语句的根节点。因此在第二层使用 CFG 邻接矩阵，可以根据程序的上下文关系来更新节点对应的特征向量。在第三个多头 GAT 层使用 DDG 邻接矩阵，是为了使用数据流信息来更新特征向量。

最后，特征向量再经过一个 MLP 和 Softmax 层即可完成图分类。接下来将对各个模块进行详细说明。

4.4.1 GAT 模块

传统的图神经网络在聚合节点特征的过程中，对于所有的节点都一视同仁，赋予相同的权重值。然而源代码中一些不必要的函数或者一些无关紧要的信息在源代码漏洞检测中意义不大。因此，在特征提取过程中，本文应用到了注意力机制，将注意力集中在需要重视的部分以获取更高的效率。

注意力机制借鉴了人类的思维方式，把有限的思维资源集中在更能够提取特征的部分，近年来在 CV, NLP 领域中注意力机制的引入，给深度学习的模型提供了巨大的性能提升，获得了显著的成果，使模型获得了更高的准确度。因此本文选用了图神经网络中的 GAT(Graph Attention Network)作为提取节点特征的主要网络结构，把注意力机制引入到图神经网络中，让图神经网络能够聚焦于图中真正重要的节点与特征信息，取得了更好的效果。GAT 的工作原理如下^[53]：

GAT 的输入为图的特征矩阵 $\mathbf{h} \in \mathbb{R}^{n \times f}$ ，邻接矩阵 $\mathbf{A}_{n \times n}$ 。其表示形式分别如下：

$$\mathbf{h} = \{\vec{h}_1, \vec{h}_2, \vec{h}_3, \vec{h}_4, \dots, \vec{h}_n\}, \vec{h}_i \in \mathbb{R}^f \quad (4-4)$$

$$\mathbf{A}_{n \times n} = (a_{ij}) \quad (4-5)$$

其中 n 是节点个数， f 是每个节点的特征向量长度。

GAT 在聚合邻居节点信息时，需要对每个节点施加不同的权重。GAT 网络引入了注意力机制，经过一个注意力函数（attention function）来计算出节点 i 与节点 j 之间的权重系数 e_{ij} ，计算公式为：

$$e_{ij} = \text{attention_function}(\mathbf{W}\vec{h}_i, \mathbf{W}\vec{h}_j) \quad (4-6)$$

其中 $\mathbf{W}$ 为图注意力网络的权重矩阵，它是可以被学习的参数。在自然语言处理中，传统的注意力函数的实现方式通常是计算特征向量之间的余弦相似度或相关性。在源代码漏洞检测中，把特征向量之间的相似度作为注意力的评判依据的意义不大。因此，本文将两个特征向量进行拼接，经过一个单层的前向神经网络来实现注意力函数，该单层网络的权重矩阵表示为 $\vec{a} \in \mathbb{R}^{2 \times f}$ 。为了保留大部分的梯度信息，中间的激活函数采用 LeakyRelu 函数。LeakyRelu 函数和权重系数的计算公式如下：

$$\text{LeakyRelu}(x) = \begin{cases} x, & x \geq 0 \\ \alpha * x, & x < 0, \alpha \in (0, 1) \end{cases} \quad (4-7)$$

$$e_{ij} = \text{LeakyRelu}(\vec{a}^T [\mathbf{W}\vec{h}_i \parallel \mathbf{W}\vec{h}_j]) \quad (4-8)$$

其中 $\parallel$ 代表拼接操作， $\cdot^T$ 表示转置操作。

在实际应用注意力机制时，本文采用 masked attention 的方式，即只计算节点

与其一阶邻居节点之间的注意力系数，不关注非邻居节点。为了能够更好的分配权重，计算出所有邻居节点的权重系数后，采用 softmax 函数对原始的权重系数进行归一化处理：

$$\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in N_i} \exp(e_{ik})} \quad (4-9)$$

N_i 表示节点 i 的所有邻居节点的集合。

将公式 4-9 展开，可以得到最终的注意力公式：

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\vec{a}^T [\vec{W}\vec{h}_i \parallel \vec{W}\vec{h}_j]))}{\sum_{k \in N_i} \exp(\text{LeakyReLU}(\vec{a}^T [\vec{W}\vec{h}_i \parallel \vec{W}\vec{h}_k]))} \quad (4-10)$$

注意力权重系数的整个计算过程如图 4-7 所示。

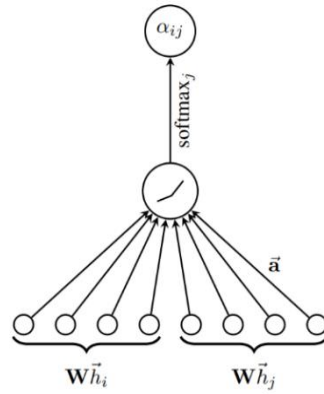


图 4-7 注意力权重系数的计算过程

得到最终的注意力权重系数矩阵后，将其和对应节点相乘求和，经过一个激活函数 σ 就可以得到节点 i 聚合其邻居节点信息后的新的表示，这里选择 softmax 作为激活函数。对于单个节点 i ，其输出的特征 $\vec{h}_i'$ 为：

$$C = \sigma \left(\sum_{j \in N_i} \alpha_{ij} \vec{W}\vec{h}_j \right) \quad (4-11)$$

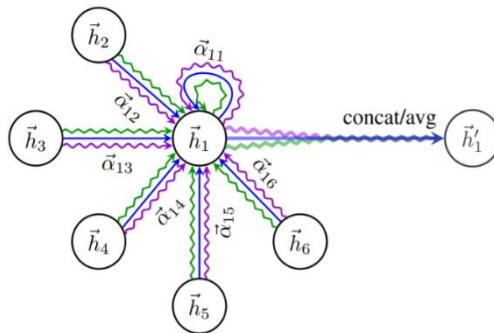


图 4-8 多头注意力机制示意图

为了使注意力机制的学习过程更加稳定，提高模型的拟合能力，GAT 引入了多头的注意力机制，可以提取到节点向量在不同表示空间中的特征，从而识别出更加丰富的信息，如图 4-8 所示。多头注意力机制同时使用多个神经网络的权重矩阵 $\mathbf{W}^k$ ，分别计算出不同的聚合结构，最终将 K 个计算结果合并。

在使用多头注意力机制的情况下，本文没有选择 GAT 论文中的求输出特征的代表方法：

$$\vec{h}_i'' = \sigma\left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in N_i} \alpha_{ij} \mathbf{W}^k \vec{h}_j\right) \quad (4-12)$$

其中 K 代表多头注意力的层数， σ 代表激活函数。将多个特征平均化容易丢失一部分信息，而本文的方法则是：

$$\vec{h}_i'' = \text{GAT}(\mathbf{A}, [\vec{h}^1 // \vec{h}^2 // \dots // \vec{h}^k]) \quad (4-13)$$

再单独使用一层 GAT 来对特征进行整合，最后得到整个图的表达信息。

4.4.2 分类模块

得到图的特征向量后，本文将其输入分类模块中实现图的二分类。首先将池化后得到的图特征向量输入到具有较少层数和参数的 MLP 结构中实现对特征信息的整合，MLP 的输入层节点数为图特征向量的维数，输出层节点数为 2，表示结果分为两类。最后，经过了一个 Softmax 层对输出进行归一化，以满足二分类的概率要求。Softmax 的定义如下：

$$S_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (4-14)$$

其中 z_i 代表第 i 个元素， S_i 是 Softmax 层对应的输出。分类模块示意图如图 4-9 所示。

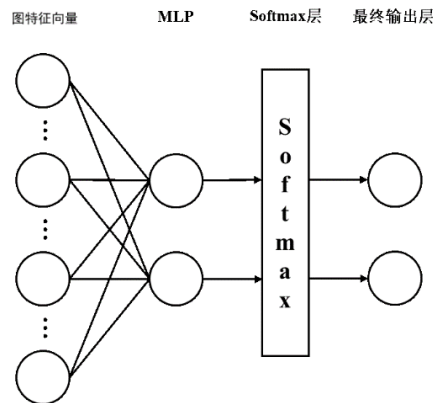


图 4-9 分类模块示意图

4.5 基于图注意力网络的 LDMGGAT 漏洞定位模型构建

DMGGAT 模型可以在函数级别，准确的检测出代码是否包含漏洞，但并不能给出漏洞的具体位置，无法达到漏洞定位效果。图注意力机制通过给不同邻居节点分配不同的注意力权重，用以更新节点的特征向量。注意力权重值越高，则模型对节点的关注度越高，这些节点则成为 DMGGAT 模型进行图分类的重要依据。在 DMGGAT 模型中，注意力机制主要用于对特征向量的中间操作，不会输出注意力矩阵，因此，本文提出 LDMGGAT 模型，通过提取 DMGGAT 模型的推理阶段的各层注意力权重矩阵，来进行漏洞定位。

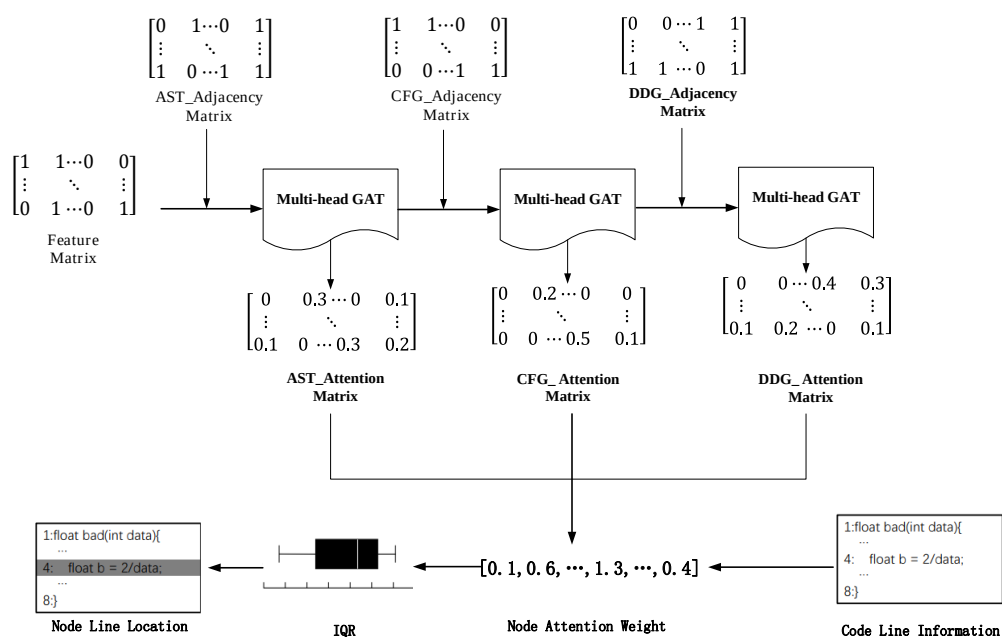


图 4-10 LDMGGAT 模型结构示意图

4.5.1 LDMGGAT 漏洞定位模型结构

LDMGGAT 模型的漏洞定位方法如图 4-10 所示：基于 DMGGAT 模型中的三层多头图注意力层，将每一层对应不同邻接矩阵的节点注意力权重矩阵归一化输出，获取每个节点对应的邻居节点对该节点的注意力权重之和，求取平均值，得到该节点的平均注意力权重。然后以 1: 1: 1 的比例将 AST、CFG、DDG 对应的节点平均注意力权重相加，获得 LDMGGAT 模型对所有节点的注意力向量。引入 IQR 算法对节点进行分类，并基于代码属性图中的节点行号信息进行定位。

下文将结合表 4-8 中 CWE369 除零漏洞示例代码、图 4-11 示例代码的代码属性图、表 4-9 代码属性图中的节点内容对 LDMGGAT 模型漏洞定位检测方法进行详细描述。在 CWE369 除零漏洞示例代码中，变量 data 的数据来源为 RAND32()

产生的随机数，data 可能为 0。因此在对变量 data 进行除法时，可能会导致除零漏洞。

表 4-8 CWE369 除零漏洞示例代码

```
1: void CWE369_Divide_by_Zero_bad(){
2:   int data;
3:   data = RAND32();
4:   printIntLine(100 / data);
5: }
```

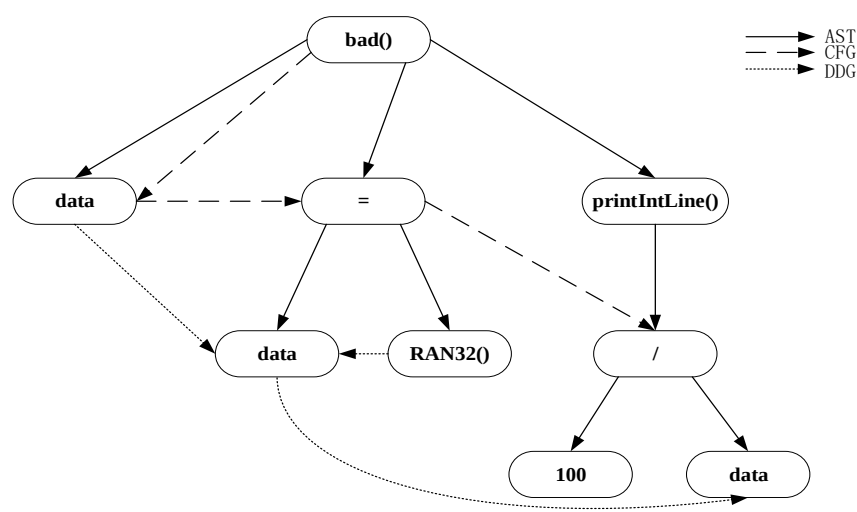


图 4-11 示例代码对应的代码属性图

表 4-9 示例代码中节点内容

节点 ID	节点	类型	行号
1	CWE369_Divide_by_Zero_bad()	函数名	1
2	data	变量	2
3	=	操作符=	3
4	data	变量	3
5	RAND32()	系统函数	3
6	printIntLine()	用户自定义函数	4
7	/	操作符/	4
8	100	常量	4
9	data	变量	4

4.5.2 注意力矩阵

LDMGGAT 模型对节点的注意力机制与邻接矩阵有关。邻接矩阵中的元素 a_{ij} ，表示节点 i 和节点 j 间是否存在边关系，值为0或1。注意力矩阵中元素 b_{ij} 的值是依据邻接矩阵中的元素 a_{ij} 进行分配。若节点 i 与节点 j 之间存在邻接关系，即 a_{ij} 为1时，则注意力矩阵中对应位置元素 b_{ij} 为 $(0,1]$ ，表示节点 i 对节点 j 的注意力权重为 b_{ij} 。若节点 i 与节点 j 之间不存在邻接关系，即 a_{ij} 为0时，则注意力矩阵中对应位置元素 b_{ij} 值为0。并且在注意力矩阵中，若节点 i 无邻接关系，则节点 i 对应的行 b_i 中所有元素之和为0。若节点 i 有诸多边关系，则节点 i 对应的行 b_i 中与所有邻接位置的元素之和为1。因此，邻接矩阵 $\text{Adj} = R^{(|V_{\text{cpg}}| \times |V_{\text{cpg}}|)}$ 在LDMGGAT模型中会被转化为注意力矩阵 $\text{Att} = A^{(|V_{\text{cpg}}| \times |V_{\text{cpg}}|)}$ 。注意力矩阵 $\text{Att} = A^{(|V_{\text{cpg}}| \times |V_{\text{cpg}}|)}$ 中值的计算方法如公式4-15：

$$b_{ij} = \alpha(W h_i^{\rightarrow}, W h_j^{\rightarrow}) \quad (4-15)$$

其中 b_{ij} 表示每个特征向量 i 和特征向量 j 之间的注意力， α 表示计算两个向量之间注意力的注意力方法， W 表示权重矩阵。

在注意力矩阵 Att 中， b_{ij} 和 b_{ji} 分别代表了节点 i 对节点 j 的注意力程度和节点 j 对节点 i 的注意力程度。每一行代表着该节点与其他节点的注意力值，行和为1。每一列代表着其他节点对该节点的注意力值。值得注意的是，虽然注意力矩阵 Att 的元素位置上是对称的，但是元素数值不同。这说明一个节点对另一个节点的注意度高但并不意味着另一个节点需要对这个节点付出同样注意力。因此注意力矩阵 Att 不是对称矩阵。

	bad()	data	=	data	RAND32()	printlnLine()	/	100	data
AST	0.64	0.63	0.75	0.13	0.63	0.27	0.77	0.29	0.39
CFG	0.74	0.91	0.39	0.00	0.00	0.48	0.60	0.00	0.00
DDG	0.00	0.00	0.00	1.00	0.21	0.00	0.00	0.00	0.78
总	1.38	1.54	1.14	1.13	0.84	0.75	1.37	0.29	1.17

图 4-12 节点平均注意力

节点的平均注意力值代表着邻居节点对该节点的平均注意力，也代表着在LDMGGAT模型的当前多头图注意力层对该节点的注意力。方法是将注意力矩阵中的每一列非0元素相加，除以非0元素的个数以求得平均值。然后以1:1:1的比例，将每层的节点平均注意力相加，得到整个LDMGGAT模型对该节点的在总体图注意力，示例代码的节点平均注意力如图4-12所示。

4.5.2 IQR 算法

程序源代码中不同的节点有不同的作用，包含的节点数量及其贡献对图的分类也不同。此外，在不同的程序中，贡献较大的节点数量也不尽相同。所以，不能简单地设置一个阈值来识别这些对图有更大贡献的节点分类。因此，本文引入 IQR 算法来解决这个问题。

IQR 算法为四分位距法，是统计描述学中的一种方法，用以求取反映集中趋势的绝对中位差^[54]。本文使用 IQR 算法可以得到 LDMGGAT 模型集中注意的关键节点，并结合代码属性图中的节点行号信息进行漏洞定位。

IQR 算法首先对所有节点的注意力系数进行排序，然后选取第一四分位的数值 Q1 和第三四分位的数值 Q3，计算 Q3 与 Q1 之间的差值，得到四分位距 IQR。本文设置一个动态参数 k ，用以表示异常值的容忍度，因此图中节点的注意力值正常区间为 $(Q1 - k \times IQR, Q3 + k \times IQR)$ 。 k 的值越大，则越少过滤对图分类贡献度越大的节点，反之亦然。因此，通过调整 IQR 异常值，并结合正常注意力值区间对应的节点和节点行号来进行漏洞定位。

id	代码	平均注意力值	行号
8	100	0.29	4
6	printlnLine()	0.75	4
5	RAND32()	0.84	3
4	data	1.13	3
3	=	1.14	3
9	data	1.17	4
7	/	1.37	4
1	bad()	1.38	1
2	data	1.54	2

图 4-13 示例代码对应的平均注意力值排序

图 4-13 表示示例代码的节点平均注意力值排序。按照本文使用的 IQR 算法，第一四分位数 Q1 为节点 5 对应的 0.84，第三四分位数 Q3 为节点 7 对应的 1.37，此时 $IQR=0.53$ 。图中节点的注意力值正常区间为 $(0.84 - k \times 0.53, 1.37 + k \times 0.53)$ 。此时设定动态参数 $k=0$ ，则正常区间值对应的节点 id 分别为 5、4、3、9、7。在正常区间值的节点中，LDMGGAT 模型对于节点 7 对应的除法注意力值最高，其次是除法的除数对象节点 9 变量 data。由此可以看出 LDMGGAT 模型在进行除零

漏洞检测时，最关注涉及除法和除数的节点。其次节点 3、4、5 对应着节点 9 除数 data 的数据来源，data 由 RAND32()随机产生，值可能为 0，存在漏洞风险。最终，根据 IQR 算法得到的正常区间值的节点对应的代码行号为 3、4。

4.6 实验验证

4.6.1 实验数据集

为了验证本章基于深度学习的源代码漏洞检测技术方案的有效性，本文采用公开数据集 Juliet Test SuiteV 1.3 作为实验数据集^[55]。

Juliet 数据集是由美国国家技术标准研究所（National Institute of Standards and Technology, NIST）于 2017 年针对 CWE 中的不同分类所创建的。该数据集由 C/C++编写，共包含了 121 种 CWE 漏洞类型，例如缓冲区溢出、整数溢出、整数下溢、空指针等。这些 C/C++程序可作为测试用例用以测试漏洞静态分析器和其他软件保障工具的有效性，目前已被广泛用于基于深度学习的源代码漏洞分析方法中。

如表 4-10 所示，在 Juliet 数据集中，每个源代码文件中的函数可以分为三类：bad 类、good 类和 main 类。

表 4-10 源代码文件中各函数含义

类型	函数名	含义	被调用函数
bad	bad()	存在漏洞	main()
	badSink()	无数据源的检查判断	bad()、goodG2B()
	badSource()	不正确的数据源	bad()、goodB2G()
good	good()	不存在漏洞	main()
	goodG2B()	使用 goodSource 和 badSink	good()
	goodB2G()	使用 goodSink 和 badSource	good()
	goodSource()	正确的数据源	goodG2B()
	goodSink()	有数据源的检查判断	goodB2G()
main	main()	主函数	无

bad 类包含 bad()函数、badSink()和 badSource()函数。bad()函数中存在漏洞，该函数可能会调用 badSink()函数或者 badSource()函数，也可能文件中不存在这两个函数，不进行调用。good 类中包含 good()函数、goodG2B()函数、goodB2G()函数、goodSource()函数和 goodSink()函数。good()函数只调用 goodG2B()函数和 goodB2G()函数，无其他代码。当源代码文件中存在 goodSource()函数和 goodSink()

函数时，goodG2B()函数和 goodB2G()函数才对其进行调用。main 类代表着 main() 函数，对 bad()函数和 good()函数进行调用，其余代码不影响这两个函数。

good()类中的函数对应着 bad 类中函数漏洞的多种修复方式，所以 bad 类函数与 good 类函数源代码差异较小，结构相似。因此本文将代码漏洞检测归结于 bad()类函数和 good()类函数的对应的关联代码属性图的图分类问题。

在本文提出的基于图注意力网络的漏洞检测定位方案中，是以函数为单位进行漏洞分析。因为漏洞不仅仅存在于单个函数内部，还存在于调用函数与被调用函数间，因此基于函数的漏洞分析不是简单的对每个函数都进行分析，而是会依据代码间的关联关系，将与漏洞相关的全部元素所对应的函数通过函数调用关系关联起来，单独作为一个检测对象。

存在真实漏洞的单个 bad()函数及其对应的修复函数 good 类函数可以直接将其转化为代码属性图，用来作为漏洞检测的图数据。由函数调用导致的真实漏洞的情况较为复杂，为此本文引入以下概念：敏感函数、非敏感函数、真实漏洞关联函数、无漏洞关联函数。

敏感函数：函数代码中存在不规范的代码操作或者不正确的数据来源等危险要素。这些危险要素是由语句本身或系统 API 函数调用引起的，例如除零漏洞中的 badSink()函数，没有对数据来源进行非 0 判断，直接对其进行除法操作。在代码的真实运行中，敏感函数与之相关联的函数可能会导致真实漏洞的出现，也有可能不会。

非敏感函数：函数代码本身不存在漏洞，该函数在被调用时可能会产生漏洞，也可能不产生漏洞。例如 badSource()函数，只是用来对某个变量值进行赋值返回操作，本身不存在漏洞或任何危险要素。但当 badSource()函数被 bad()函数调用，根据其返回值进行除法操作时，这就会导致漏洞的产生。又如 goodSource()函数在被任何函数调用进行除法操作时，都不会产生漏洞。

真实漏洞关联函数：由敏感函数引起的真实漏洞，与漏洞产生因素相关联的所有函数叫做真实漏洞关联函数。真实漏洞关联函数中可能会包含非敏感函数。

无漏洞关联函数：所有关联起来的敏感函数和非敏感函数不会导致真实漏洞的产生。例如 goodG2B()函数，调用了 goodSource()函数和 badSink()函数，虽然没有进行非 0 判断，但有非 0 的数据源，不存在漏洞。

因此提取出 Juliet 数据集数据集中的真实漏洞关联函数和无漏洞关联函数，并标记漏洞标签。将无漏洞关联函数作为负样本标注为“0”，代表该函数不存在漏洞；将真实漏洞关联函数作为正样本标注为“1”，代表该函数存在漏洞。

在模型训练方面，本文选择 Juliet 数据集中五个经典的子数据集，分别是 CWE-121 Stack-based Buffer Overflow（基于栈的缓冲区溢出），CWE-122 Heap-based Buffer Overflow（基于堆的缓冲区溢出），CWE-369 Divide-Zero（除零错误），CWE-416: Use After Free（释放后使用）和 CWE-476 NULL Pointer Dereference（空指针引用）。这五类数据集包含的漏洞都是 C\C++ 中较为常见的漏洞类型，其对应数据集标签分布如表 4-11 所示。

表 4-11 数据集的标签分布

CWE	漏洞类型	漏洞函数 1	无漏洞函数 0	总
121	Stack-based Buffer Overflow	6751	4638	11389
122	Heap-based Buffer Overflow	8112	5436	13548
369	Divide-Zero	2088	792	2880
416	Use After Free	1400	440	1840
476	NULL Pointer Dereference	849	334	1183

4.6.2 评价指标

4.6.2.1 漏洞检测效果评价指标

在代码漏洞分析领域，误报率（False Positive Rate, FPR）、漏报率（False Negative Rate, FNR）、查全率（True Positive Rate, TPR）、查准率（Precision, P）和 F1 分数（F1-Score）是常见的评价指标。定义描述如下：

$$FPR = \frac{FP}{TN+FP} \quad (4-16)$$

$$FNR = \frac{FN}{TP+FN} \quad (4-17)$$

$$TPR = \frac{TP}{TP+FN} \quad (4-18)$$

$$P = \frac{TP}{TP+FP} \quad (4-19)$$

$$F_1 = \frac{2 \times P \times TPR}{P + TPR} \quad (4-20)$$

其中，TP 为本身不存在漏洞同时也被模型预测为不存在漏洞的样本数量，FN 为本身不存在漏洞但被模型预测为存在漏洞的样本数量。FP 为本身存在漏洞但被模型预测为不存在漏洞的样本数量，TN 为本身存在漏洞且被模型预测为存在漏洞的样本数量。

误报率 FPR 表示存在真实漏洞的源代码中被判定为不存在漏洞的比例，误报

率越低，则检测效果越理想；漏报率 FNR 表示所有不存在真实漏洞的源代码中被判定为存在漏洞的比例，漏报率越低，则检测效果越理想；查全率 TPR 表示所有不存在真实漏洞的源代码中被正确判定为不存在漏洞的样本的比例，查全率越高，则检测效果越理想；查准率 P 表示所有被判定为不存在真实漏洞的源代码中准确检测出不存在漏洞的比例，查准率越高，则检测效果越理想；F1 分数表示查全率和查准率的调和平均值，其综合考虑了查全率和查准率，F1 分数越高，则检测效果越理想。

4.6.2.2 漏洞定位效果评价指标

在 Juliet 数据集中，每个漏洞文件都会有 POTENTIAL FLAW 潜在漏洞标识注释。根据这些注释，可以快速找到导致漏洞产生的语句行号。从图上看出，POTENTIAL FLAW 潜在漏洞标识注释分别在 29 和 32 行，对应漏洞代码的行号分别为 30 和 33 行。

```
24 void CWE369_Divide_by_Zero__float_rand_01_bad()
25 {
26     float data;
27     /* Initialize data */
28     data = 0.0F;
29     /* POTENTIAL FLAW: Use a random number that could possibly equal zero */
30     data = (float)RAND32();
31     {
32         /* POTENTIAL FLAW: Possibly divide by zero */
33         int result = (int)(100.0 / data);
34         printIntLine(result);
35     }
36 }
```

图 4-14 示例代码

根据对 Juliet 数据集中漏洞代码研究和 POTENTIAL FLAW 潜在漏洞标识注释的研究，本文提出两个概念：漏洞关联行和漏洞关键行。

漏洞关联行：漏洞产生原因所在行。当程序执行到当前语句所在行时，不会立即导致漏洞发生，但当前语句的某些操作，会导致后续漏洞的产生。

漏洞关键行：漏洞产生所在行。当程序执行到当前语句所在行时，就会产生漏洞。

本文以定位结果的准确率 Acc 来评价定位效果，只有满足以下定位条件才算定位成功：

- (1) 定位结果必须包含漏洞关键行；
- (2) 定位结果中的漏洞关联行数量占实际漏洞关联行数量的 80%以上；
- (3) 定位结果中的漏洞无关行数量低于定位结果总行数的 20%。

4.6.3 实验结果分析

4.6.3.1 漏洞检测实验结果分析

本文将表 4-11 中的五种 CWE 漏洞类型标签数据集以 8:2 的比例划分为训练集和测试集，具体训练集和测试集标签数据量如表 4-12 所示。

表 4-12 五种 CWE 漏洞类型的训练集和测试集

CWE	漏洞类型	训练集		测试集	
		漏洞 1	无漏洞 0	漏洞 1	无漏洞 0
121	Stack-based Buffer Overflow	5400	3710	1351	928
122	Heap-based Buffer Overflow	6490	4349	1087	5436
369	Divide-Zero	1670	634	418	158
416	Use After Free	1120	352	280	88
476	NULL Pointer Dereference	679	267	170	67

随后本文使用基于图注意力网络的 DMGGAT 漏洞检测模型分别对上述五种 CWE 漏洞类型对应的数据集进行了训练测试，得到的实验结果如表 4-13 所示。DMGGAT 模型在 CWE121、CWE122、CWE369、CWE416 和 CWE476 上的 F1 分数分别为 95.99%、90.36%、95.86%、96.00%和 96.62%。漏洞检测实验结果显示使用 DMGGAT 漏洞检测模型可以有效的检测出上述五种 CWE 漏洞类型中极大多数的漏洞。

表 4-13 DMGGAT 模型实验结果

	FPR%	FNR%	TPR%	P%	F1%
CWE121	5.86	4.0	96.0	95.97	95.99
CWE122	14.11	9.97	90.03	90.50	90.36
CWE369	6.44	5.70	94.30	97.48	95.86
CWE416	18.86	2.21	97.79	94.28	96.00
CWE476	11.38	2.36	97.64	95.62	96.62

DMGGTA 模型虽然在 CWE121、CWE122、CWE369、CWE416 和 CWE476 上 F1 分数较高，但只能证明模型在漏洞检测上的有效性，不能证明该模型比其他的漏洞检测方法具有更好的性能。

因此本文基于相同的训练集和测试集，分别使用漏洞检测工具 Flawfinder^[56]、双向长短时记忆 BiLSTM^[57]、机器学习 SVM^[58]和图卷积神经网络 GCN^[59]在五种 CWE 漏洞类型上进行漏洞检测实验，并与 DMGGAT 模型的实验结果进行了对比分析，具体对比实验结果如下表 4-14、4-15、4-16、4-17、4-18 所示。

表 4-14 CWE121 对比实验

CWE121	FPR%	FNR%	TPR%	P%	F1%
DMGGAT	5.86	4.00	96.00	95.97	95.99
FlawFinder	0.00	100.00	0.00	0.00	0.00
BiLSTM	9.91	4.74	95.26	93.33	94.29
SVM	10.56	15.26	84.73	92.11	86.65
GCN	7.42	8.50	91.50	94.72	93.08

表 4-15 CWE122 对比实验

CWE122	FPR%	FNR%	TPR%	P%	F1%
DMGGAT	14.11	9.97	90.03	90.50	90.36
FlawFinder	0.00	100.00	0.00	0.00	0.00
BiLSTM	26.56	12.88	87.12	83.03	85.03
SVM	27.98	22.08	77.92	80.60	79.24
GCN	26.42	11.45	88.55	83.32	85.86

表 4-16 CWE369 对比实验

CWE369	FPR%	FNR%	TPR%	P%	F1%
DMGGAT	13.17	5.16	94.84	94.02	94.43
FlawFinder	100.00	0.00	100.00	72.50	84.06
BiLSTM	19.86	8.35	91.65	90.94	91.30
SVM	29.62	13.75	86.25	86.36	86.30
GCN	18.77	6.85	93.15	91.52	92.33

表 4-17 CWE416 对比实验

CWE416	FPR%	FNR%	TPR%	P%	F1%
DMGGAT	18.86	2.21	97.79	94.28	96.00
FlawFinder	100.00	0.00	100.00	76.09	86.42
BiLSTM	25.00	13.57	86.43	91.67	88.97
SVM	27.95	14.29	85.71	90.70	88.14
GCN	20.00	4.26	95.74	93.22	95.32

Flawfinder 的优势是词法扫描和分析，基于内置的 C/C++ 函数漏洞数据库来工作，使用模式匹配来识别缓冲区溢出、格式化串漏洞等。表格中 FlawFinder 的检测数值都是基于表 4-11 进行计算的。从表 4-13 和表 4-14 可以看出，Flawfinder 在 CWE121 和 CWE122 上误报为 0，但漏报极高。通过研究其检测报告，本文发现 Flawfinder 在识别缓冲区溢出漏洞时主要是对 strcpy()、strcat()、gets()、sprintf()、

scanf()等函数进行识别，并不进行其他校验。因此不管是真实漏洞关联函数还是无漏洞关联函数，Flawfinder 都会认为这些函数中涉及缓冲区操作的语句存在漏洞。另外，Flawfinder 在 CWE369、CWE416、CWE476 上的检测结果全是误报，不能够进行相应漏洞识别。因此 Flawfinder 只能用于缓冲区溢出的敏感函数检测，再结合人工审计对代码进行一一排查。

表 4-18 CWE476 对比实验

CWE476	FPR%	FNR%	TPR%	P%	F1%
DMGGAT	7.16	3.90	96.10	96.50	96.30
FlawFinder	100.00	0.00	100.00	71.77	83.56
BiLSTM	8.29	6.68	93.32	95.85	94.57
SVM	35.30	7.1	92.86	84.40	88.43
GCN	8.71	6.82	93.18	95.64	94.40

双向长短时记忆 BiLSTM 是由前向 LSTM 和后向 LSTM 组合而成的，使用循环网络对节点的特征向量进行特征提取和训练。简单来讲，BiLSTM 可以看成两层的神经网络。前向 LSTM 使用从前往后的节点特征向量遍历顺序，后向 LSTM 从后往前对节点特征向量遍历，以此来获得前后节点间的关联。源代码对应的关联代码属性图前后节点间存在着 AST、CFG 等边信息。并且结合对比实验表信息可以看出，BiLSTM 在漏洞检测方面，具有一定的效果。但是 BiLSTM 中的 LSTM 主要用于自然语言处理中对上下文信息的建模，加之源代码不同于文本，具有更多的节点信息和节点间关系，因此其效果不如本文提出的 DMGGAT 模型。

SVM 是机器学习中的一种二分类模型，它的基本模型是定义在特征空间上的间隔最大的线性分类器。SVM 依据源代码对应的关联代码属性图结构进行分类，因此 SVM 针对图结构差异较大的图数据分类效果较为明显。但是在挑选的五种漏洞类型数据集上，部分真实漏洞关联函数和无漏洞关联函数对应的关联代码属性图图结构差别不大，此时其正确率会有所下降，误报率会有所上升。SVM 的检测效果不如 DMGGAT 模型与 BiLSTM。

图卷积神经网络 GCN 是图分类中的重要模型，通过对图数据进行特征提取达到分类效果。对比实验使用三层 GCN，分别输入 AST、CFG、DDG 邻接矩阵来进行漏洞检测。GCN 对每个节点求取其邻居节点对该节点的加权平均值，这里与 GAT 的差别在于，GCN 中邻居节点对该节点的权重值是相同的，而 GAT 中则是使用注意力机制，邻居节点对于该节点的权重值不相同。因此 GCN 模型的漏洞检测效果略低于 DMGGAT 模型。

DMGGTA 模型的优势在于注意力机制与训练过程中添加了多种节点间关系，

能够给予图中关键节点较大的注意度，因此模型效果较好。DMGGAT 模型在 CWE121、CWE369、CWE416 和 CWE476 上 F1 分数都在 95% 以上，而在 CWE122 上 F1 分数低一些为 90.36%。一方面是因为 CWE122 的数据集规模比其他四种漏洞类型要大一些。另一方面是因为在 CWE122 堆缓冲区溢出的代码中，定义了大量的结构体，真实漏洞关联函数与无漏洞关联函数间的差别仅在于对结构体中同类型的不同变量的引用。DMGGAT 模型在针对相同类型的不同变量区分度较低，因此误报较高，查全率和查准率较低一些。

根据上述对比实验结果可以看出，DMGGAT 漏洞检测模型的效果比 Flawfinder、BiLSTM、SVM、GCN 漏洞检测方法效果要更好。

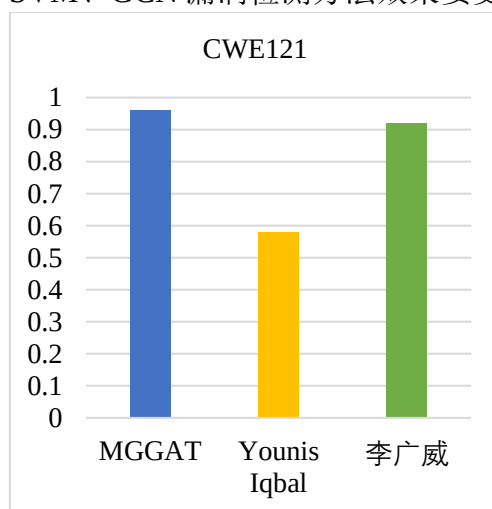


图 4-15 CWE121 各类方法 TPR 对比柱状图

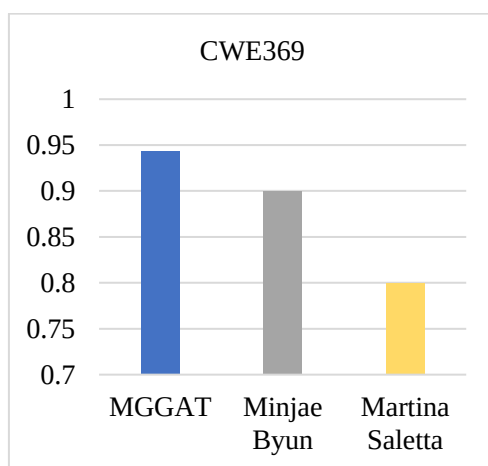


图 4-16 CWE369 各类方法 TPR 对比柱状图

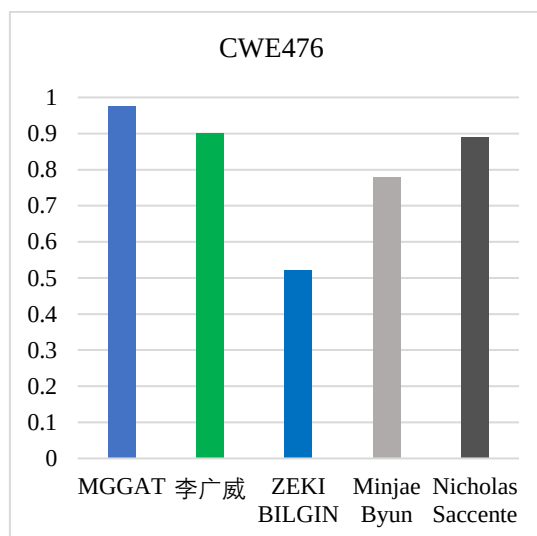


图 4-17 CWE476 各类方法 TPR 对比柱状图

目前漏洞检测相关技术研究使用的数据集和评判标准不尽相同。为验证本文提出的 DMGGAT 模型的有效性，基于相同的数据集和 TPR 评判标准，本文挑选了 Younis Iqbal^[61]、李广威^[62]、Minjae Byun^[63]、Martina Saletta^[64]、ZEKI BILGIN^[65]、Nicholas Saccente^[66]发表的论文进行对比，对比效果具体见图 4-15、4-16 和 4-17。从图中可以看出，最左列的 DMGGAT 模型的 TPR 值在 CWE121、CWE369、CWE476 上均比其他方法要高，说明 DMGGAT 模型效果较好。

4.6.3.2 漏洞定位实验结果分析

本文提出 LDMGGAT 漏洞定位模型，基于注意力矩阵和 IQR 算法来对漏洞进行定位。将图中节点的注意力值正常区间为($Q1 - k \times IQR$, $Q3 + k \times IQR$)与节点行号信息相结合，通过不断调整 k 值即可实现定位功能。基于 LDMGGAT 模型的漏洞定位数据来源于 DMGGAT 模型中判断存在漏洞的图数据，具体的漏洞定位实验结果如下表 4-19 所示，在五种漏洞类型测试数据上的定位准确率在 80%以上，验证了 LDMGGAT 模型的有效性。

表 4-19 漏洞定位实验结果

漏洞类型	k 值	准确率
CWE121	0.32	82.71%
CWE122	0.34	80.16%
CWE369	0.25	86.32%
CWE416	0.2	84.47%
CWE476	0.26	85.69%

4.7 本章小结

本章通过关联代码属性图间的差异性将源代码漏洞检测问题归纳为图分类问题。为尽可能包含源代码中的语法、语义、控制、依赖等信息，提出了 VecCPG 向量化方案，详细介绍了向量化过程，构造了关联代码属性图的特征矩阵和三种邻接矩阵。随后将关联代码属性图的特征矩阵和邻接矩阵输入到基于图注意力的 DMGGAT 漏洞检测模型中。依靠三层多头图注意力层网络层和分类模块，可以有效地对漏洞进行检测。其次，使用基于图注意力网络的 LDMGGAT 模型对漏洞实现了定位功能。最后，使用漏洞检测工具 Flawfinder、双向长短时记忆 BiLSTM、机器学习 SVM 和图卷积神经网络 GCN 与 DMGGAT 模型进行了对比实验分析和 LDMGGAT 模型定位实验结果分析，证明了这两个模型的有效性。

第五章 基于深度学习的源代码漏洞检测系统设计与实现

前文对基于关联代码属性图的生成技术和基于深度学习的模型检测技术两个关键技术研究进行了详细描述。基于上述方法，本章将详细介绍基于深度学习的源代码漏洞检测系统的设计与实现。

5.1 系统设计

5.1.1 需求分析

基于深度学习的源代码漏洞检测系统是基于源代码生成的关联代码属性图，通过使用不同类型的深度学习漏洞检测模型，对待检测的源代码进行漏洞检测和漏洞定位。本原型系统的功能需求主要分为两块内容，第一部分是基于待检测代码生成代码属性图，通过添加 FCG 关系和调整 CFG、DDG 边关系，关联调用函数和被调用函数；第二部分则是使用本文提出的 DMGGAT 模型和 LDMGGAT 模型对代码属性图进行漏洞检测及定位。这两个部分分别在本文的第三章和第四章中介绍了相应的技术原理。

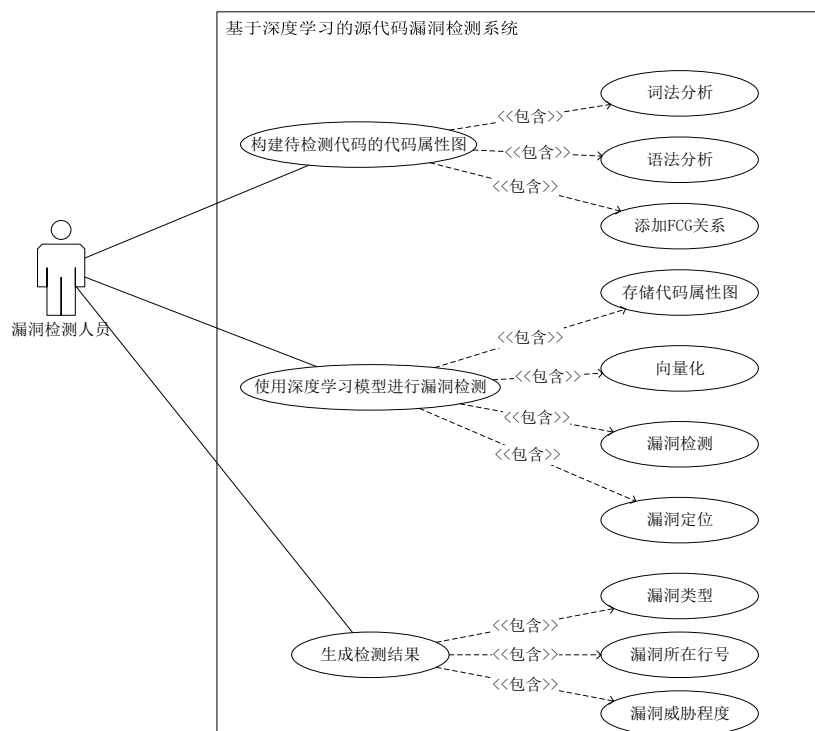


图 5-1 系统用例图

如图 5-1 所示，本系统的面向用户是人工审计或漏洞研究的代码漏洞检测人员。该系统旨在高效快速的筛查源代码中的漏洞，辅助人工审计，减少人工审计中大量的代码检测工作，使漏洞检测人员可以将更多的精力放到对漏洞检测结果的判定中。用户可以将待检测部分的源代码或者整个系统源代码上传系统，通过对源代码进行词法分析、语法分析等编译操作，生成代码属性图，并对函数进行关联和存储。然后使用系统中的深度学习检测模型来对待检测代码进行漏洞检测。系统会预设几种类型的漏洞检测模型，用户可以使用预设模型或者自行上传模型，增强了系统的可扩展性，扩展了漏洞检测类型的覆盖面。

5.1.2 总体框架设计

如图 5-2 所示，基于深度学习的源代码漏洞检测系统采用三层架构。在表示层使用 B/S 架构，用户可以使用电脑或者移动端对系统页面进行访问，进行源代码上传、漏洞检测和检测结过查看等操作。前端通过 Nginx 将用户请求的数据转发到业务逻辑层，根据不同的请求调用相应的接口。业务逻辑层通过后台功能模块在数据访问层中对数据库进行增删改查等操作，最终将用户请求的数据返回给前端页面，进行可视化展示。

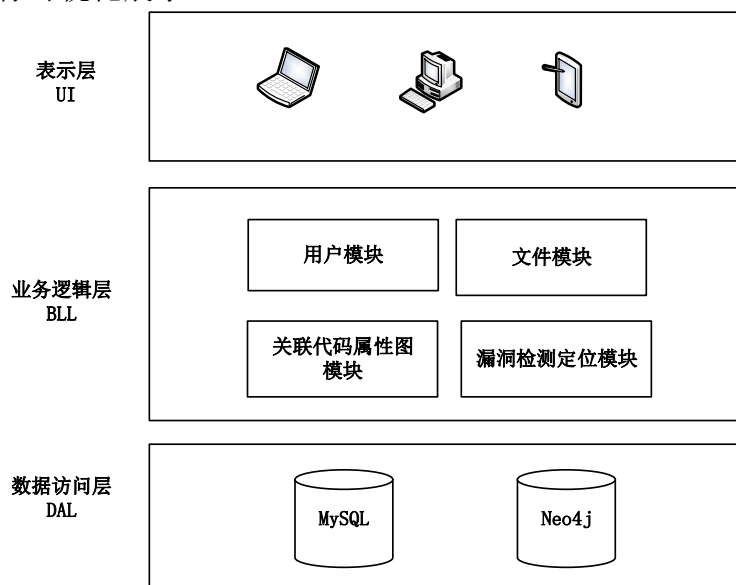


图 5-2 系统架构图

5.1.3 功能模块设计

基于深度学习的源代码漏洞检测系统根据不同的功能和业务逻辑，可以分为四个模块：用户管理模块、文件管理模块、关联代码属性图模块和漏洞检测模块，如图 5-3 所示。

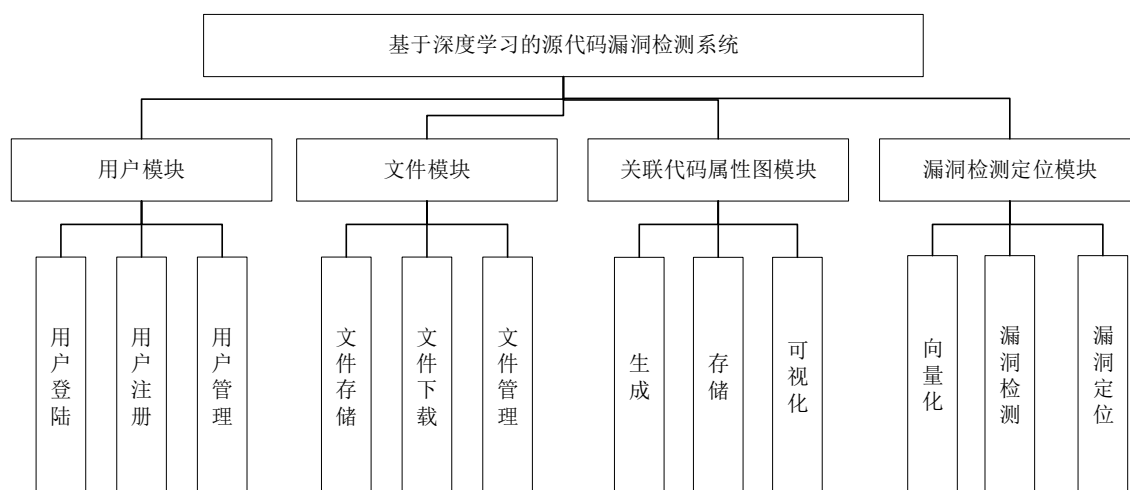


图 5-3 系统功能模块

用户管理模块是任何系统都必有的，用于保障系统的安全性、可靠性和数据库内容的准确性。用户管理模块包含三个功能：用户登陆、用户注册和用户管理。本系统将用户分为两类，普通用户和系统管理员。普通用户第一次使用该系统时，需要在系统注册界面进行账号密码注册，随后即可使用账号密码登陆。系统用户可以对用户信息进行管理，包括删除用户、修改密码操作。

文件模块包含三个功能：文件存储、文件下载、文件管理。文件存储是将用户上传的源代码文件和生成的检测报告存储在 MySQL 数据库中。文件下载是当用户需要离线查看检测报告或进行相应研究时，系统提供源代码文件和检测报告下载功能。文件管理是对上述源代码文件和检测报告进行有序管理，包括删除、查看等操作。

关联代码属性图模块包括三个功能：关联代码属性图的生成、关联代码属性图的存储、关联代码属性图的可视化。当用户上传待检测的源代码后，系统会将源代码进行编译，生成对应的关联代码属性图。然后将关联代码属性图存储到 Neo4j 图数据库中，方便后续可视化和漏洞检测定位模型中对其进行向量化。

漏洞检测定位模块包括三个功能：关联代码属性图的向量化、漏洞检测和漏洞定位。漏洞检测定位模块首先将待检测代码生成的代码属性图进行向量化，并预设了五种漏洞类型对应的漏洞检测模型和漏洞定位模型，用户可以选择预设的模型或自行上传相应的漏洞检测和漏洞定位模型进行检测，得到漏洞检测和定位结果。

5.2 基于深度学习的源代码漏洞检测系统实现

基于深度学习的源代码漏洞检测系统使用 B/S 架构和 MVC 架构进行开发实现。前端页面使用 Vue 框架，结合 HTML、CSS 和 JavaScript 语言进行开发，开发工具采用 VSCode。功能模块使用 Java 和 Scala 语言实现，后端接口使用基于 Springboot 框架进行开发，开发工具采用 IDEA，数据库采用 Neo4j 和 MySQL。本节将主要介绍系统关联代码属性图模块和漏洞检测定位模块两大核心功能模块的开发。

5.2.1 关联代码属性图模块的实现

该模块的主要功能是关联代码属性图的生成、存储、可视化，以便为后续的漏洞检测提供基础数据。模块整体流程如图 5-4 所示，首先使用 codeToCpg 代码分析工具对源代码进行编译，生成包含抽象语法树 AST、控制流图 CFG、程序依赖图 PDG 中所有信息的 cpg.bin 文件。其次使用代码属性图转 CSV 格式工具 cpgToCsv 对 cpg.bin 文件进行内容解析，得到所有节点的节点信息文件 node.csv 和节点间边关系文件 edge.csv。随后将 node.csv 和 edge.csv 通过 neo4j-admin import 命令导入到 Neo4j 图数据库中。此时在 Neo4j 图数据库中形成的是源代码对应的代码属性图。之后，使用 AssCPG 工具对代码属性图进行函数关联，增添 FCG 关系，调整 CFG 和 DDG 边，最终形成关联代码属性图。最后，基于 Neo4j 图数据库自带的 Neo4j Browser，对关联代码属性图进行可视化展示。关联代码属性图模块的整个流程实现封装在后端程序中，下面对每一步进行介绍。

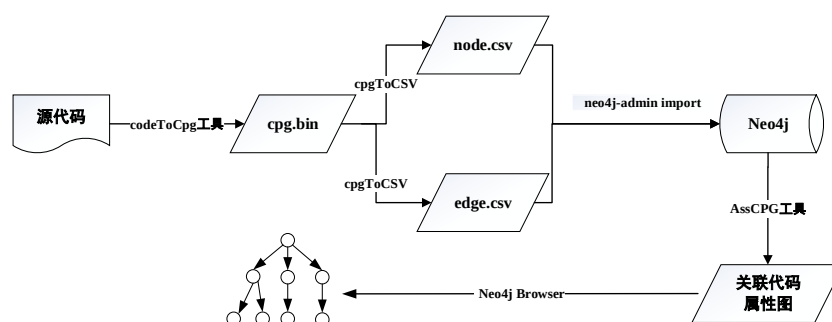


图 5-4 关联代码属性图模块流程

代码分析工具 codeToCpg 封装了开源的 Joern 软件，实现了对源代码进行词法分析、语法分析等编译操作。codeToCpg 工具能够得到源代码对应的 AST、CFG、DDG、CDG、PDG 所有信息。

codeToCpg 工具生成的 cpg.bin 文件不符合 Neo4j 图数据库支持的文件格式，因此需要使用代码属性图转 CSV 格式工具 cpgToCsv 进行结果解析，将对应的节点信息和边信息存储为 Neo4j 图数据库支持的 csv 文件。cpgToCsv 工具使用 scala 语言进行编写，部分源码如下所示。

cpgToCsv 工具部分源码

```
object newThread extends App {  
  //加载 cpg.bin 文件  
  var cpgFileName: String = "cpg.bin"  
  val cpg = CpgBasedTool.loadFromOdb(cpgFileName)  
  addDataFlowOverlayIfNonExistent(cpg)  
  //获取 nodes 迭代器  
  val nodeIt: util.Iterator[overflowdb.Node] = cpg.graph.nodes  
  //迭代 node 获取 node 所有信息  
  ...  
  //加载边  
  val context = new LayerCreatorContext(cpg)  
  val cpgedges = context.cpg  
  val edgeIt = cpgedges.method.iterator  
  //获取所有边信息  
  ...  
}
```

图 5-5 示例代码对应的节点信息文件 node.csv 与边信息文件 edge.csv 内容如图 5-6、图 5-7 所示。

```
C: > Users > ZHJ > Desktop > C CWE369_Divide_by_Zero.c  
1 void CWE369_Divide_by_Zero_Data()  
2 {  
3     float Data;  
4     Data = 0.0F;  
5     CWE369_Divide_by_Zero(Data);  
6 }  
7  
8 void CWE369_Divide_by_Zero(float Data)  
9 {  
10    float data;  
11    data = Data;  
12    int result = (int)(100.0 / data);  
13    printIntLine(result);  
14 }
```

图 5-5 示例代码

1	id:ID,,LABEL,ALIAS_TYPE_FULL_NAME,ARGUMENT_INDEX,AST_PARENT_FULL_NAME,AST_PARENT_TYPE,BINARY_SIGNATURE,CANONICAL_NAME,CLOSURE_BINDING_ID,CODE,C	7
2	1,META_DATA,,,,,,,,C,,,,,,,,,"List(semanticcp, dataflowDss)",List(),,,,,,	
3	2,NAMESPACE_BLOCK,,,,,,,,,<global>,,,,,,,,,<global>,-1,,,,,,,,,	
4	1000104,LOCAL,,,,,,,,Data,,,,,,,,List(),,,,,,,,,Data,1,,,,,,,,float,	
5	1000108,CALL,,3,,,,,CWE369_Divide_by_Zero(Data),4,,,,STATIC_DISPATCH,List(),,,,,,6,,CWE369_Divide_by_Zero,,CWE369_Divide_by_Zero,3,,,,,TODO assign	
6	1000117,IDENTIFIER,,2,,,,,Data,11,,,,List(),,,,,,,,,12,,,,Data,2,,,,,,,,float,	
7	1000122,UNKNOWN,,1,,,,,int,10,,,,List(),,,,,,,,,13,,,,,1,,CastTarget,,,,,,,,,	
8	1000128,METHOD_RETURN,,,,,,,,RET,0,,,,List(),BY_VALUE,,,,,,,,9,,,,,3,,,,,Void,	
9	1000134,METHOD,,<global>,NAMESPACE_BLOCK,,,,,,,,<operator>.assignment,,true,,,,<operator>.assignment,0,,,,,TODO assignment signature,,,,,	
10	1000135,METHOD_PARAMETER_IN,,,,,,,,p1,,,,List(),BY_VALUE,,,,,,,,p1,1,,,,,ANY,	
11	1000138,BLOCK,,1,,,,,List(),,,,,,,,,1,,,,,,,,ANY,	
12	1000142,METHOD_RETURN,,,,,,,,RET,,,,List(),BY_VALUE,,,,,,,,-1,,,,,ANY,	
13	1000147,BLOCK,,1,,,,,List(),,,,,,,,,1,,,,,,,,ANY,	
14	1000158,METHOD_PARAMETER_OUT,,,,,,,,float Data,27,,,,BY_VALUE,,,,,,,,9,,,,Data,1,,,,,,,,float,	
15	102,TYPE,,,,,,,,int,,,,,,,,int,,,,,,,,,	
16	1000103,BLOCK,,1,,,,,0,,,,List(),,,,,,,,,3,,,,,1,,,,,void,	
17	1000107,LITERAL,,2,,,,,0.0f,11,,,,List(),,,,,,,,,5,,,,,2,,,,,float,	
18	1000112,METHOD_PARAMETER_IN,,,,,,,,float Data,27,,,,List(),BY_VALUE,,,,,,,,9,,,,Data,1,,,,,,,,float,	
19	1000153,METHOD_PARAMETER_OUT,,,,,,,,p1,,,,BY_VALUE,,,,,,,,p1,1,,,,,ANY,	
20	1000157,METHOD_PARAMETER_OUT,,,,,,,,p2,,,,BY_VALUE,,,,,,,,p2,2,,,,,ANY,	

图 5-6 节点 csv 文件部分信息

1	id:START_ID,id:END_ID,type:TYPE,label
2	1000144,1000146,AST,
3	1000144,1000145,AST,
4	1000144,1000147,AST,
5	1000144,1000146,DDG,
6	1000144,1000145,DDG,
7	1000134,1000137,AST,
8	1000134,1000135,AST,
9	1000134,1000138,AST,
10	1000134,1000136,AST,
11	1000134,1000137,DDG,
12	1000134,1000135,DDG,
13	1000134,1000136,DDG,
14	1000139,1000142,AST,
15	1000139,1000140,AST,

图 5-7 边 csv 文件部分信息

Neo4j 图数据库常用的两种数据导入方法为 load csv 和 neo4j-admin import，本文使用导入速度较快的第二种方法。在得到代码属性图对应的节点信息文件 node.csv 和边信息文件 edge.csv 后，使用 neo4j-admin import 命令对其进行导入。导入过程如图 5-8 所示，导入后生成的代码属性图展示效果如图 5-9 所示。

```
D:\Software\Neo4j\neo4j-community-3.5.22\bin>neo4j-admin import --mode=csv --database=graph.db --nodes C:\Users\ZHJ\Desktop\node.csv --multiline-fields=true --relationships C:\Users\ZHJ\Desktop\edge.csv --ignore-duplicate-nodes=true --ignore-missing-nodes=true
Neo4j version: 3.5.22
Importing the contents of these files into D:\Software\Neo4j\neo4j-community-3.5.22\data\databases\graph.db:
Nodes:
  C:\Users\ZHJ\Desktop\node.csv
Relationships:
  C:\Users\ZHJ\Desktop\edge.csv

Available resources:
  Total machine memory: 15.86 GB
  Free machine memory: 6.23 GB
  Max heap memory : 227.75 MB
  Processors: 6
  Configured max memory: 14.08 GB
  High-I/O: false
```

图 5-8 导入过程

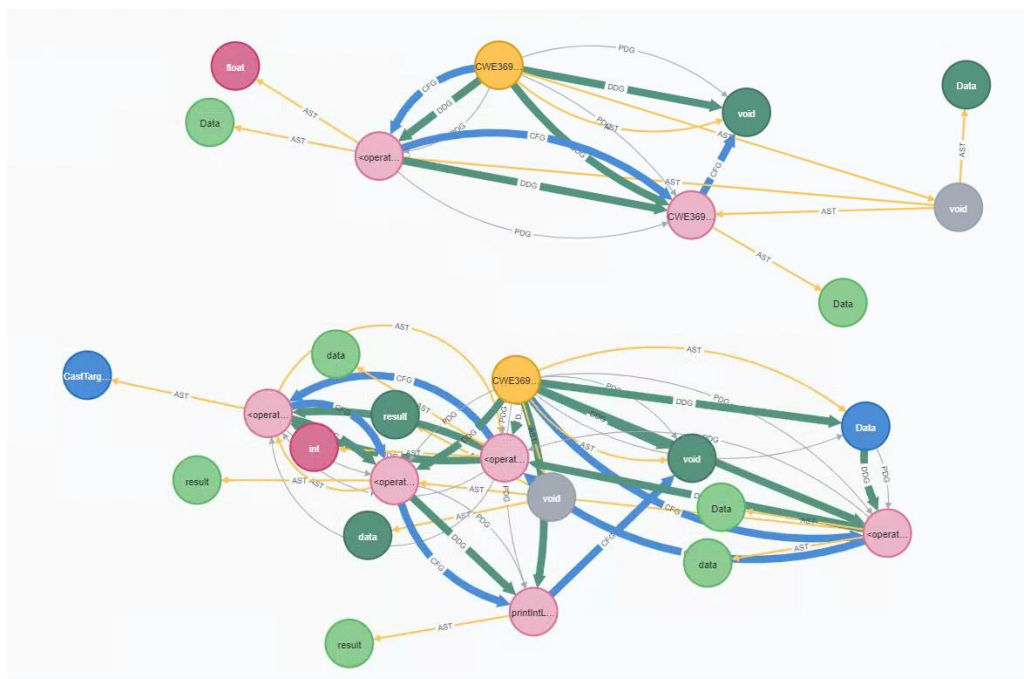


图 5-9 代码属性图

代码分析工具 `codeToCpg` 生成的代码属性图是单个函数对应的代码属性图，但是漏洞不仅存在于单个函数内，还可能存在于调用函数与被调用函数中，因此使用关联代码属性图构建工具 `AssCPG` 来关联和调整代码属性图。关联代码属性图构建工具 `AssCPG` 主要包含 `AddFCG`、`AssCFG` 和 `AssDDG` 三种算法，具体内容在第三章进行了详细介绍，最终生成的关联代码属性图如图 5-10 所示。

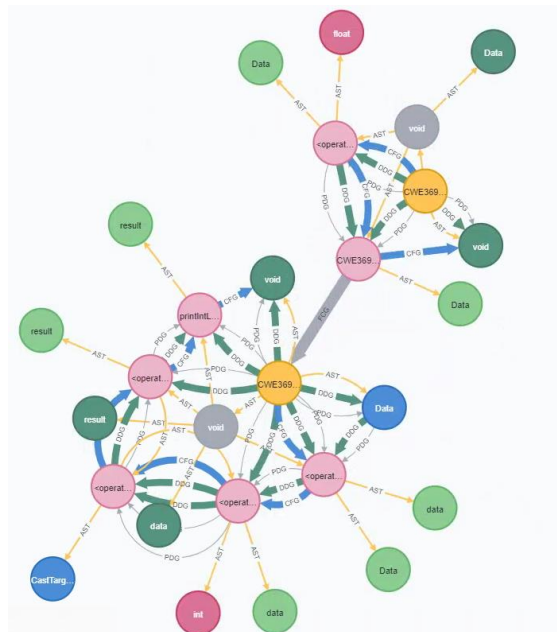


图 5-10 关联代码属性图

5.2.2 漏洞检测及定位模块的实现

漏洞检测及定位模块包括关联代码属性图的向量化、模型选择和上传，漏洞检测和定位。其中向量化技术、漏洞检测和漏洞定位技术在第四章进行了详细的研究。漏洞检测及定位模块流程如图 5-11 所示，首先使用基于 VecCPG 向量化方案的 cpgToVector 工具对关联代码属性图进行向量化，得到其对应的特征矩阵和邻接矩阵。然后输入到 DMGGAT 漏洞检测模型中，对上传的源代码文件进行是否存在漏洞的判定。若存在漏洞，则将存在漏洞的源代码文件再次输入到 LDMGGAT 模型中进行定位。最终将 DMGGAT 模型和 LDMGGAT 模型的检测结果合起来形成漏洞检测报告。

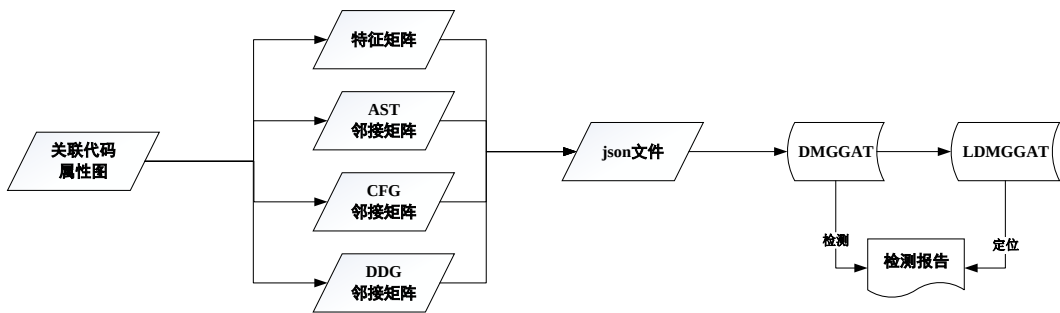


图 5-11 漏洞检测及定位模块流程

向量化工具 cpgToVector 将关联代码属性图中的节点信息转化为特征向量，节点间 AST、CFG、DDG 三种边关系转化为三种邻接矩阵，最终形成对应的 json 文件。cpgToVector 工具部分代码如下所示，对应的 json 文件内容如图 5-12 所示。

cpGToVector 部分代码

```
public void process(List<Integer> GraphIDs,String file,String label){  
    StringBuffer adjAndFeature = new StringBuffer("{}");  
    String adj = subGraphAdj(GraphIDs);  
    adjAndFeature.append(adj);  
    String feature = subGraphFeature(GraphIDs);  
    adjAndFeature.append(feature);  
    adjAndFeature.append(",");  
    adjAndFeature.append("\"label\": \"\");  
    adjAndFeature.append(label);  
    adjAndFeature.append("\",");  
    adjAndFeature.append("\"count\": \"\");
```

```
adjAndFeature.append(GraphIDs.size());

adjAndFeature.append("\",");

adjAndFeature.append("\func_name\":" );

adjAndFeature.append(file.replace("mcu_periph\\", ""));

adjAndFeature.append("\");

adjAndFeature.append("{}");

createAndWrite(adjAndFeature.toString(),file);
```

[illegible]

图 5-12 json 格式数据

漏洞检测模块则是预设了五种 DMGGAT 检测模型，将上述 json 文件输入到某种漏洞类型的模型中去，即可进行漏洞检测，并保留检测结果等待合并。

漏洞定位模块也预设了对应的五种 LDMGGAT 漏洞定位模型，其会根据 DMGGAT 漏洞检测模型的输出结果对对应的源代码文件进行定位检测。LDMGGAT 漏洞定位模型会根据根据每层的权重值和输出的特征向量，使用 IQR 算法进行定位。

在检测完成后，系统会给出相应的漏洞检测报告。漏洞检测报告包含报告名称、检测源码文件名称、漏洞类型、漏洞行号、漏洞威胁程度等信息。用户可以通过漏洞检测报告进行漏洞的核查。

5.3 基于深度学习的源代码漏洞检测系统测试

5.3.1 基于深度学习的源代码漏洞检测系统的运行环境

本文设计的源代码漏洞检测系统运行在装有 NVIDIA TITAN RTX 24GB GPU,

Intel(R) Xeon(R) CPU E5-2678 v3 @ 2.50GHz 和 96GB 内存的服务器上，相关信息如表 5-1 所示。系统测试使用 Google Chrome v99.0.4844.51。

表 5-1 服务器相关信息

类别	名称
系统版本	Ubuntu 20.04
	JDK8
运行环境	Nginx v1.18.0
	Neo4j v3.5.5
	MySQL v8.0.21

5.3.2 基于深度学习的源代码漏洞检测系统的系统展示

5.3.2.1 用户管理界面

用户管理界面包含三个页面：用户登陆页面、用户注册页面和用户管理页面。

用户登陆页面用于用户使用本系统时，进行身份认证，确保系统的安全性。如图 5-13 所示，用户输入用户名和用户密码，系统通过对数据库中用户信息进行查询确认，返回一个有时长的 cookie 实现访问。普通用户和系统管理员的权限不同，普通用户访问权限更低。



图 5-13 用户登陆页面

当新用户使用本系统时，则需要先进行账号注册，待管理员审核之后才能使用，如图 5-14 所示。新用户需要自定义用户名和密码，并进行重复密码操作达到确认目的。

管理员可以对所有本系统的用户进行管理。系统将后台用户数据返回前端展示，管理员可以执行删除和修改用户密码等操作，如图 5-15 所示。



图 5-14 用户注册页面

用户管理	文件管理	漏洞检测	结果查看	个人设置
用户名	密码	用户权限	操作	
张三	123456	管理人员	删除 修改密码	
李四	456123	普通用户	删除 修改密码	
王五	123456	普通用户	删除 修改密码	
赵六	125634	普通用户	删除 修改密码	

图 5-15 用户管理页面

5.3.2.2 文件管理界面

文件管理页面则是对用户上传的源码，检测结果进行管理，如图 5-16 所示。此页面包含了用户上传的源码、生成的报告、检测时间、所属用户和相应操作。操作包含了对检测信息的删除、查看和下载。

用户管理 文件管理 漏洞检测 结果查看					个人设置
序号	报告名称	报告源码	检测时间	所属用户	操作
1	CWE369_Divide_by_Zero__float_fgets_01	CWE369_Divide_by_Zero__float_fgets_01.c	2021.12.1	张三	删除 下载源码 下载报告 查看
2	CWE369_Divide_by_Zero	CWE369_Divide_by_Zero.zip	2021.12.2	张三	删除 下载源码 下载报告 查看
3	CWE476_NULL_Pointer_Dereference__binary_if_01	CWE476_NULL_Pointer_Dereference__binary_if_01.c	2021.12.12	李四	删除 下载源码 下载报告 查看
4	CWE122_Heap_Based_Buffer_Overflow_01	CWE122_Heap_Based_Buffer_Overflow_01.c	2021.12.31	王五	删除 下载源码 下载报告 查看

图 5-16 文件管理页面

5.3.2.3 漏洞检测定位界面

漏洞检测定位页面包含漏洞检测提交页面和漏洞检测结果页面。如图 5-17 所示，漏洞检测提交页面包含了源码文件上传，自定义报告名称，漏洞模型选择或者自定义漏洞上传。源码文件上传支持多种格式文件，可以上传单个文件，多个文件或者是源码压缩包。

用户管理文件管理漏洞检测结果查看

个人设置

漏洞检测提交

文件上传

选择文件

未选择文件

报告名称

输入检测报告名称，默认为源码文件名

检测模型选择

定位模型选择

自定义检测模型上传

选择文件

未选择文件

自定义定位模型上传

选择文件

未选择文件

检测

取消

图 5-17 漏洞检测提交页面

漏洞检测结果页面则包括检测结果查看，源代码查看和对应的代码属性图查看。如图 5-18 所示，检测结果页面展示了漏洞数量、漏洞对应的源码文件、定位信息和危害程度。对于检测结果可以通过操作中的查看源码和查看代码属性图来进行查看，源代码的查看也可以通过标签页中的源代码下拉菜单操作，如图 5-19。

用户管理文件管理漏洞检测结果查看

个人设置

检测结果

源代码

代码属性图

序号	源码文件	漏洞编号	漏洞定位	漏洞危害程度	操作
1	CWE369_Divide_by_Zero_float_fgets_01.c	CWE369除零漏洞	34、37、46	威胁	查看源码 查看代码属性图
2	CWE369_Divide_by_Zero_float_fgets_02.c	CWE369除零漏洞	36、39、51	威胁	查看源码 查看代码属性图

图 5-18 检测结果页面

用户管理文件管理漏洞检测结果查看

个人设置

检测结果

源代码

代码属性图

序号	源码文件	漏洞编号	漏洞定位	漏洞危害程度	操作
1	CWE369_Divide_by_Zero_float_fgets_01.c	CWE369除零漏洞	34、37、46	威胁	查看源码 查看代码属性图
2	CWE369_Divide_by_Zero_float_fgets_02.c	CWE369除零漏洞	36、39、51	威胁	查看源码 查看代码属性图

图 5-19 源代码标签页面

源代码页面则展示了用户上传的检测源码，根据漏洞检测定位结果，对漏洞代码所在行进行了红色高亮显示，方便用户在源代码展示页面对漏洞进行快速定位，如图 5-20 所示。

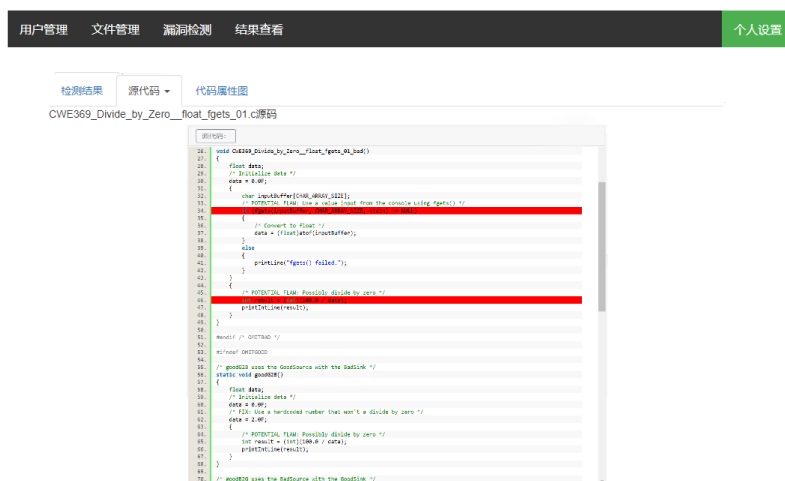


图 5-20 源代码页面

代码属性图页面则展示了上述源码对用的代码属性图，该页面内嵌了 **neo4j** 的前端页面，用户也可以使用 **neo4j** 的查询语言进行全面细致的节点和边查看，如图 5-21 所示。



图 5-21 代码属性图界面

5.4 本章小结

本章介绍了基于深度学习的源代码漏洞检测系统的设计与实验。对整个系统进行了详细的需求分析和框架设计，并对用户管理模块、文件管理模块、关联代码属性图模块和漏洞检测定位模块进行了详细的描述和具体实现。

第六章 全文总结与展望

6.1 全文总结

随着互联网的飞速发展和软件数量的爆炸式增长，漏洞数量呈现上升趋势。这些漏洞被不法分子所利用，用以窃取隐私、泄露数据、破坏资产，给个人和企业带来巨大的危害。因此漏洞检测成为当前安全发展中的重要一环。如何引入新兴 AI 技术，解决目前软件源代码漏洞检测技术存在的漏报率高、严重依赖领域专家知识、以及检测粒度过粗等诸多问题，及时发现、定位软件漏洞，已成为当前亟须解决的问题。为了减轻源代码漏洞检测时人工审计的压力，提高检测效率，本文基于深度学习对源代码漏洞检测技术进行了研究。基于深度学习的源代码漏洞检测技术主要分为两个核心研究，关联代码属性图构建技术研究和基于深度学习的漏洞检测定位模型研究。本文主要完成以下三项工作。

(1) 改进代码属性图，构建关联代码属性图。Fabian Yamaguchi 提出的代码属性图能够对源代码进行词法、语法分析，用于构建 AST、CFG、DDG、CDG、PDG，但研究发现，该方法只能生成单个函数的代码属性图，缺少函数调用关系。因为漏洞不仅仅会出现在单个函数内部，还有可能存在于调用函数与被调用函数之间，因此对函数添加调用关系十分必要。本文在原有的代码属性图基础上，通过图遍历的方法，对比调用关系中的函数名，形参等内容，确立了函数调用关系。并根据函数调用关系，将两个函数的 CFG 和 DDG 进行了关联，最终形成关联代码属性图。

(2) 基于关联代码属性图间的差异性，将源代码漏洞检测归纳为图分类问题，并构建 DMGGAT 漏洞检测模型和 LDMGGAT 漏洞定位模型。源代码漏洞检测首先需要通过本文提出的 VecCPG 向量化方法，对待检测代码的关联代码属性图进行向量化，构建对应的特征向量和 AST、CFG、DDG 邻接矩阵。然后将其输入到基于图注意力网络的 DMGGAT 模型中进行检测。DMGGAT 模型使用三层 GAT，通过输入不同的邻接矩阵来对特征向量进行更新降维，并使用分类模块实现漏洞检测。在漏洞检测中，注意力机制赋予邻居节点不同的权重，平均权重大的节点为图分类的依据。基于此原理，LDMGGAT 漏洞定位模型将每一层中的注意力矩阵输出，获得每个节点的平均注意力值，使用 IQR 算法实现漏洞定位。

(3) 设计并实现了基于深度学习的源代码漏洞检测系统。根据系统需求，将整个系统分为四个模块：用户管理模块、文件管理模块、关联代码属性图模块和漏洞检测定位模块。基于该系统，用户可以将漏洞检测代码以单个、多个文件

或者压缩包的形式上传，使用系统预设的漏洞检测定位模型或者自行上传模型，对上传代码进行漏洞检测。用户可以在系统对检测结果进行查看或者下载存储。

6.2 后续工作展望

本文基于关联代码属性图和图注意力网络，完成了源代码漏洞检测技术相关研究，并设计实现了相应的源代码漏洞检测系统。本文提出的研究方法在五种 CWE 漏洞类型上具有较好的检测效果，但仍存在一些不足，需要进行更加深入的探索和研究，主要包括以下几个方面：

（1）构建多分类的漏洞检测模型

本文提出的研究方法在 CWE121、CWE122、CWE369、CWE416、CWE476 上检测效果良好，初步验证了方法的可行性和模型的有效性。但 Juliet 数据集共有 121 种漏洞类型，还需要在其他漏洞类型上进行进一步的技术研究和漏洞检测。后续研究可以构建一种能够实现多分类的漏洞检测模型，用以检测更多的 CWE 漏洞类型，提高模型的普适性。

（2）提升漏洞定位准确性

本文使用基于注意力机制的 LDMGGAT 模型和 IQR 算法实现了漏洞定位，但有些定位结果还存在着无法定位到漏洞关键行、包含较多漏洞无关行、没有完全包含漏洞关联行问题，因此还需要进一步的研究来提升漏洞定位的准确性。

（3）基于真实软件漏洞进行检测

本文提出的研究方法在 Juliet 数据集上效果良好，但还需应用到真实软件上进行漏洞检测，检验能否发现一些已知漏洞和未知漏洞。

致 谢

在攻读硕士学位期间，首先衷心感谢李玉军老师。在毕业设计的选题、研究和写作等方面，李老师都悉心指导，多次进行详细讨论，肯定我近阶段的研究工作并指出研究方法的误区，因此我才能在源代码漏洞检测领域中做出一些有价值的研究。从入学到现在，从学习到工作，李老师都给予了我莫大的支持和鼓励。感谢李老师的辛勤付出和悉心教导，在今后的学习生活中，我定当谨记李老师的谆谆教诲，勤奋努力，不忘初心，积极进取。

感谢电子科技大学分布式存储与计算实验室的老师們和学校里我的所有任课老师，他们教会了我许多知识，夯实了理论基础，提升了理论素养。

感谢王一璇师兄、霍智慧师姐、陈翔师兄、罗叶妮师姐和乔齐师兄。在我的学习生活中，他们都提供了许多帮助，为我排忧解难，出谋划策。感谢我的师弟师妹们，一起学习的日子我将永远铭记。

感谢我的父母，感谢你们一直以来的养育之恩与无私支持。不管我在人生的岔路口做出什么样的决定，是你们给予我一往无前的信心。

感谢所有关心我的同学和朋友们，曾经时光很快乐，遇见你们很幸运。

感谢评审此文付出辛勤劳动的专家们。

最后，希望我能够在未来继续努力，不断学习，不畏艰难，积极进取，早日实现人生理想。

参考文献

- [1] 中国互联网络信息中心 (CNNIC). 第 48 次中国互联网络发展现状统计报告[R]. 北京: 2020 年 8 月 27 日.
- [2] Ghaffarian S M, Shahriari H R. Software vulnerability analysis and discovery using machine-learning and data-mining techniques: A survey[J]. ACM Computing Surveys (CSUR), 2017, 50(4): 1-36.
- [3] 陈小全, 薛锐. 程序漏洞: 原因、利用与缓解——以 C 和 C++ 语言为例[J]. 信息安全学报, 2017, 2(04): 41-56. DOI: 10.19363/j.cnki.cn10-1380/tn.2017.10.004.
- [4] Sun H, Cui L, Li L, et al. VDSimilar: Vulnerability detection based on code similarity of vulnerabilities and patches[J]. Computers & Security, 2021, 110: 102417.
- [5] Cui L, Hao Z, Jiao Y, et al. VulDetector: Detecting Vulnerabilities Using Weighted Feature Graph Comparison[J]. IEEE Transactions on Information Forensics and Security, 2020, 16: 2004-2017.
- [6] Li Z, Zou D, Xu S, et al. Vulpecker: an automated vulnerability detection system based on code similarity analysis[C]//Proceedings of the 32nd Annual Conference on Computer Security Applications. 2016: 201-213.
- [7] Wu P, Yin L, Du X, et al. Graph-based Vulnerability Detection via Extracting Features from Sliced Code[C]//2020 IEEE 20th International Conference on Software Quality, Reliability and Security Companion (QRS-C). IEEE, 2020: 38-45.
- [8] 黄诚, 赵倩崇, 郭勇延. 一种基于函数级代码相似性的漏洞检测方法[P]. 中国, 发明专利 ZL202111071388.0, 2021 年 9 月 13 日
- [9] Pham N H, Nguyen T T, Nguyen H A, et al. Detecting recurring and similar software vulnerabilities[C]//2010 ACM/IEEE 32nd International Conference on Software Engineering. IEEE, 2010, 2: 227-230.
- [10] Li J, Ernst M D. CBCD: Cloned buggy code detector[C]//2012 34th International Conference on Software Engineering (ICSE). IEEE, 2012: 310-320.
- [11] Yamaguchi F, Lindner F, Rieck K. Vulnerability extrapolation: Assisted discovery of vulnerabilities using machine learning[C]//Proceedings of the 5th USENIX conference on Offensive technologies. 2011: 13-13.
- [12] Jang C, Kim J, Jang H, et al. Rule-based auditing system for software security assurance[C]//2009 First International Conference on Ubiquitous and Future Networks. IEEE, 2009: 198-202.

- [13] Shahriar H, Haddad H M, Vaidya I. Buffer overflow patching for C and C++ programs: rule-based approach[J]. ACM SIGAPP Applied Computing Review, 2013, 13(2): 8-19.
- [14] 刘春燕. 基于规则的 C/C++代码静态检测方法研究[D].大连理工大学,2010.
- [15] Darwin I F. Checking C Programs with Lint[M]. Sebastopol, CA: O'Reilly Media, Inc, 1988
- [16] Klocwork. Klocwork K7[EB/OL]. [2021-06-10]. <http://www.klocwork.com>
- [17] PMD Source Code Analyzer.PMD source code[CP/OL]. [2021-06-10]. <https://pmd.github.io/>
- [18] Fortify: Fortify Static Code Analyzer| Micro Focus [EB/OL]. <https://www.microfocus.com/zh-cn/products/static-code-analysis-sast/overview>
- [19] Chernis B, Verma R. Machine learning methods for software vulnerability detection[C]//Proceedings of the Fourth ACM International Workshop on Security and Privacy Analytics. 2018: 31-39.
- [20] Medeiros I, Neves N F, Correia M. Automatic detection and correction of web application vulnerabilities using data mining to predict false positives[C]//Proceedings of the 23rd international conference on World wide web. 2014: 63-74.
- [21] Lin G, Zhang J, Luo W, et al. Cross-project transfer representation learning for vulnerable function discovery[J]. IEEE Transactions on Industrial Informatics, 2018, 14(7): 3289-3297.
- [22] Zhong H, Zhang L, Xie T, et al. Inferring specifications for resources from natural language API documentation[J]. Automated Software Engineering, 2011, 18(3): 227-261.
- [23] Padmanabhuni B M, Tan H B K. Predicting buffer overflow vulnerabilities through mining light-weight static code attributes[C]//2014 IEEE International Symposium on Software Reliability Engineering Workshops. IEEE, 2014: 317-322.
- [24] Feng Q, Zhou R, Xu C, et al. Scalable graph-based bug search for firmware images[C]//Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. 2016: 480-491.
- [25] Xu X, Liu C, Feng Q, et al. Neural network-based graph embedding for cross-platform binary code similarity detection[C]//Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. 2017: 363-376.
- [26] Harer J A, Kim L Y, Russell R L, et al. Automated software vulnerability detection with machine learning[J]. arXiv preprint arXiv:1803.04497, 2018.
- [27] 段旭,吴敬征,罗天悦,杨牧天,武延军.基于代码属性图及注意力双向 LSTM 的漏洞挖掘方法[J].软件学报,2020,31(11):3404-3420.DOI:10.13328/j.cnki.jos.006061.
- [28] Li Z, Zou D, Xu S, et al. Vuldeepecker: A deep learning-based system for vulnerability detection[J]. arXiv preprint arXiv:1801.01681, 2018.

- [29] Zou D, Wang S, Xu S, et al. μ VulDeePecker: A Deep Learning-Based System for Multiclass Vulnerability Detection[J]. IEEE Transactions on Dependable and Secure Computing, 2019, 18(5): 2224-2236.
- [30] Russell R, Kim L, Hamilton L, et al. Automated vulnerability detection in source code using deep representation learning[C]//2018 17th IEEE international conference on machine learning and applications (ICMLA). IEEE, 2018: 757-762.
- [31] Zhou Y, Liu S, Siow J, et al. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks[J]. Advances in neural information processing systems, 2019, 32.
- [32] Wang S, Liu T, Tan L. Automatically learning semantic features for defect prediction[C]//2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE). IEEE, 2016: 297-308.
- [33] Khanh Dam H, Pham T, Ng S W, et al. A deep tree-based model for software defect prediction[J]. arXiv e-prints, 2018: arXiv: 1802.00921.
- [34] 文敏,王荣存,姜淑娟.基于关系图卷积网络的源代码漏洞检测[J/OL].计算机应用:1-8[2022-03-07].<http://kns.cnki.net/kcms/detail/51.1307.TP.20211209.0925.006.html>
- [35] 朱彪凯,李颖,张志强,曹敏,刘三满.基于动态模糊测试和机器学习的智能合约漏洞检测方法[J].警察技术,2021(06):56-60.
- [36] 倪萍,陈伟.基于模糊测试的反射型跨站脚本漏洞检测[J].计算机应用,2021,41(09):2594-2601.
- [37] 赵伟,张问银,王九如,王海峰,武传坤.基于符号执行的智能合约漏洞检测方案[J].计算机应用,2020,40(04):947-953.
- [38] 朱正欣,曾凡平,黄心依.二进制程序的动态符号化污点分析[J].计算机科学,2016,43(02):155-158+187.
- [39] 李鑫,张维伟,郑力新.动静结合的二阶 SQL 注入漏洞检测技术[J].华侨大学学报(自然科学版),2018,39(04):600-605.
- [40] 宁馨,易平.基于深度学习的动静结合的漏洞挖掘方法[J].通信技术,2021,54(02):430-436.
- [41] Zhang R, Huang S, Qi Z, et al. Static program analysis assisted dynamic taint tracking for software vulnerability discovery[J]. Computers & Mathematics with Applications, 2012, 63(2): 469-480.
- [42] Rawat S, Ceara D, Mounier L, et al. Combining static and dynamic analysis for vulnerability detection[J]. arXiv preprint arXiv:1305.3883, 2013.

- [43] Li Y, Chen Z, Cao J, et al. {ReDoSHunter}: A Combined Static and Dynamic Approach for Regular Expression {DoS} Detection[C]//30th USENIX Security Symposium (USENIX Security 21). 2021: 3847-3864.
- [44] 陈英,陈朔鹰. 编译原理: 清华大学出版社, 2009 年
- [45] 刘任任, 王婷, 周经野. 离散数学 第 2 版: 中国铁道出版社, 2015.08
- [46] 徐俊明. 图论及其应用[M]. 合肥: 中国科学技术大学出版社, 2019.8
- [47] 中国电子技术标准化研究院. 知识图谱标准化白皮书[EB/OL].
<http://www.cesi.cn/201909/5589.html>
- [48] Yamaguchi F, Golde N, Arp D, et al. Modeling and discovering vulnerabilities with code property graphs[C]//2014 IEEE Symposium on Security and Privacy. IEEE, 2014: 590-604.
- [49] Neamtiu I, Foster J S, Hicks M. Understanding source code evolution using abstract syntax tree matching[C]//Proceedings of the 2005 international workshop on Mining software repositories. 2005: 1-5.
- [50] Allen F E. Control flow analysis[J]. ACM Sigplan Notices, 1970, 5(7): 1-19.
- [51] Ferrante J, Ottenstein K J, Warren J D. The program dependence graph and its use in optimization[J]. ACM Transactions on Programming Languages and Systems (TOPLAS), 1987, 9(3): 319-349.
- [52] 孙贺,吴礼发,洪征,徐明飞,周胜利.基于函数调用图的二进制程序相似性分析[J].计算机工程与应用,2016,52(21):126-133.
- [53] Veličković P, Cucurull G, Casanova A, et al. Graph attention networks[J]. arXiv preprint arXiv:1710.10903, 2017.
- [54] Yule G U, Kendall M G. An introduction to the theory of Statistics Charles Griffin and Co[J]. Ltd, London, 1968: 66.
- [55] Information Technology Laboratory, "Juliet Documents," Software and Systems Division PRIVACY/SECURITY ISSUES, <https://samate.nist.gov/SARD/around.php>, accessed Mar. 3. 2021.
- [56] Flawfinder. Flawfinder Home Page (dwheeler.com) [EB/OL]. <https://dwheeler.com/flawfinder/>
- [57] Huang Z, Xu W, Yu K. Bidirectional LSTM-CRF models for sequence tagging[J]. arXiv preprint arXiv:1508.01991, 2015.
- [58] 张学工. 关于统计学习理论与支持向量机[J]. 自动化学报, 2000, 26(1):11.
- [59] Defferrard M, Bresson X, Vandergheynst P. Convolutional neural networks on graphs with fast localized spectral filtering[J]. Advances in neural information processing systems, 2016, 29: 3844-3852.

- [60] Joern, "Joern, Open-Source Code Querying Engine for C/C++," ShiftLeft, <https://joern.io>, accessed in Mar. 14. 2021.
- [61] Iqbal Y, Sindhu M A, Arif M H, et al. Enhancement in Buffer Overflow (BOF) Detection Capability of Cppcheck Static Analysis Tool[C]//2021 International Conference on Cyber Warfare and Security (ICCWS). IEEE, 2021: 112-117.
- [62] 李广威, 袁挺, 李炼. 开源 C/C++ 静态软件缺陷检测工具实证研究[J]. 软件学报, 2022, 33(6): 0-0.
- [63] Byun M, Lee Y, Choi J Y. Analysis of software weakness detection of CBMC based on CWE[C]//2020 22nd International Conference on Advanced Communication Technology (ICACT). IEEE, 2020: 171-175.
- [64] Saletta M, Ferretti C. A neural embedding for source code: Security analysis and CWE lists[C]//2020 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCCom/CyberSciTech). IEEE, 2020: 523-530.
- [65] Bilgin Z, Ersoy M A, Soykan E U, et al. Vulnerability prediction from source code using machine learning[J]. IEEE Access, 2020, 8: 150672-150684.
- [66] Saccente N, Dehlinger J, Deng L, et al. Project achilles: A prototype tool for static method-level vulnerability detection of Java source code using a recurrent neural network[C]//2019 34th IEEE/ACM International Conference on Automated Software Engineering Workshop (ASEW). IEEE, 2019: 114-121.

攻读硕士 学位期间取得的成果

- [1] Vulmg: A Static Detection Solution For Source Code Vulnerabilities Based On Code Property Graph and Graph Attention Network[C]//2021 18th International Computer Conference on Wavelet Active Media Technology and Information Processing (ICCWAMTIP). IEEE, 2021: 250-255.